CSCI3230 (ESTR3108)
Fundamentals of Artificial Intelligence

Lecture 6. Neural Network Basics

Prof. Qi Dou

Email: qidou@cuhk.edu.hk
Office: Room 911, 9/F, SHB

Dept. of Computer Science & Engineering
The Chinese University of Hong Kong

Outline

Part 1. Neural network architecture

Part 2. Backpropagation

Part 3. Network training and validation

Part 4. Summary

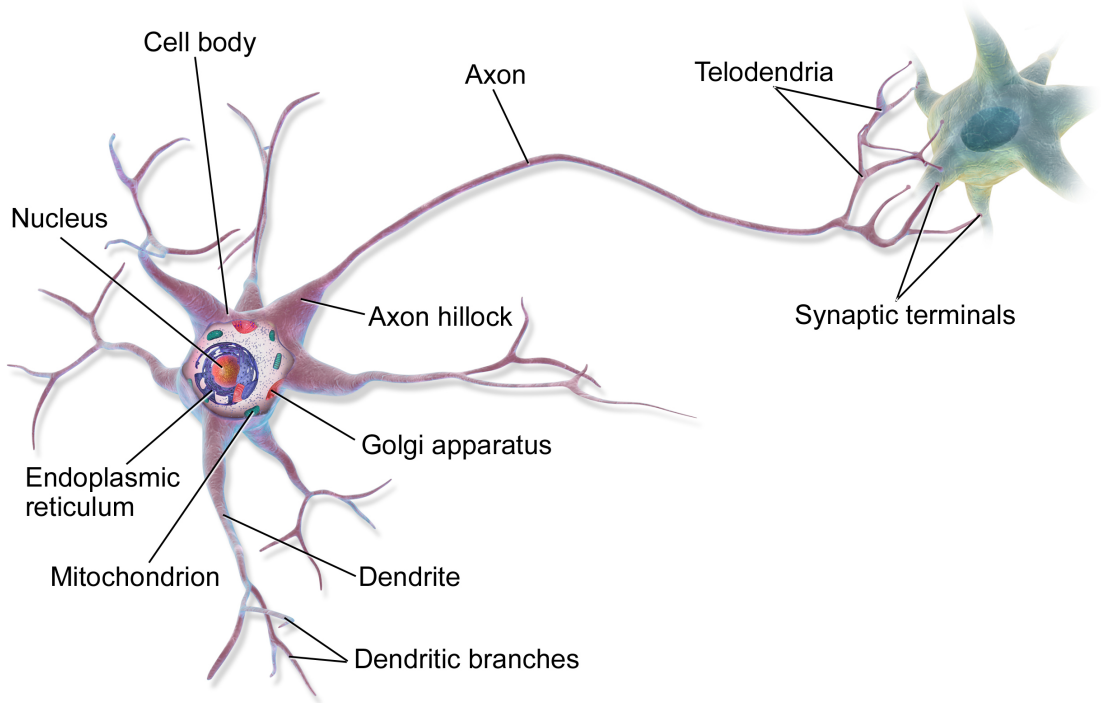
Part 1. Neural network architecture

Biological inspiration: modeling one neuron

- The basic computational unit of the brain is a neuron.

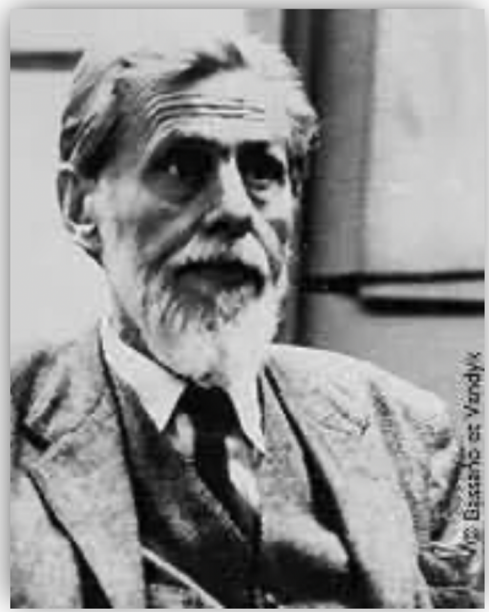
Biological inspiration: modeling one neuron

- The basic computational unit of the brain is a neuron.
- 86 billion neurons can be found in the human nervous system and they are connected with $10^{14} - 10^{15}$ approximately synapses.
- Each neuron receives input signals from its dendrites and produces output signals along its (single) axon connections.



Biological inspiration: modeling one neuron

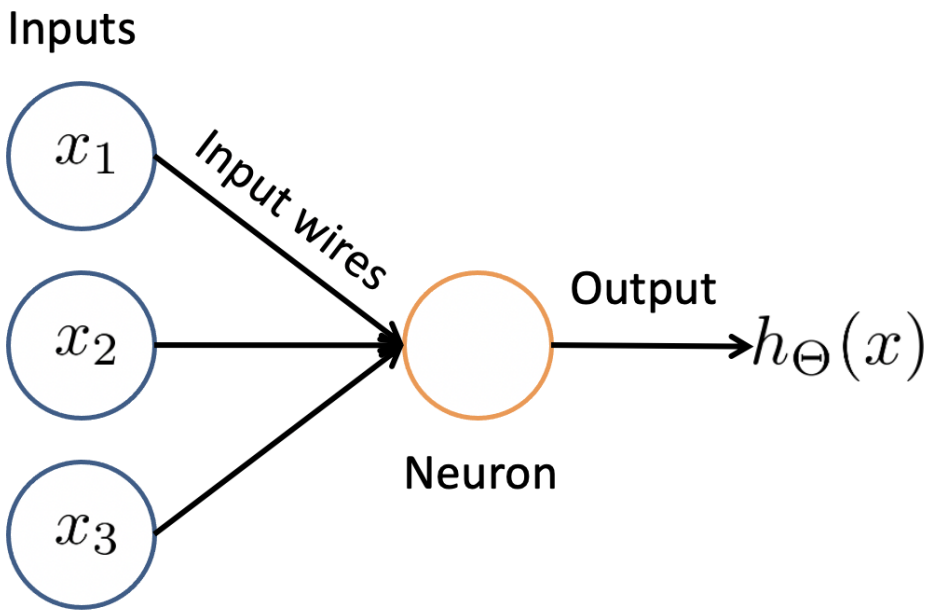
- McCulloch & Pitts (1943)[1] are generally recognized as the designers of the first neural network.
- Many of their ideas are still useful today, e.g.,
  - many simple units combine to give increased computational power.
  - the idea of a threshold.



[1] https://en.wikipedia.org/wiki/Artificial_neuron

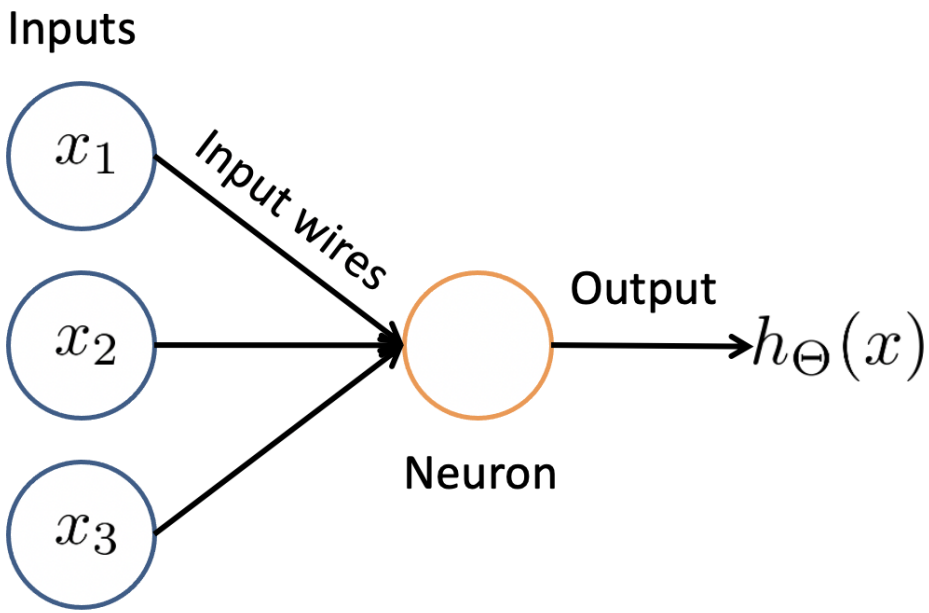
Neuron model: logistic unit

Here we model a neuron as a logistic unit.



Neuron model: logistic unit

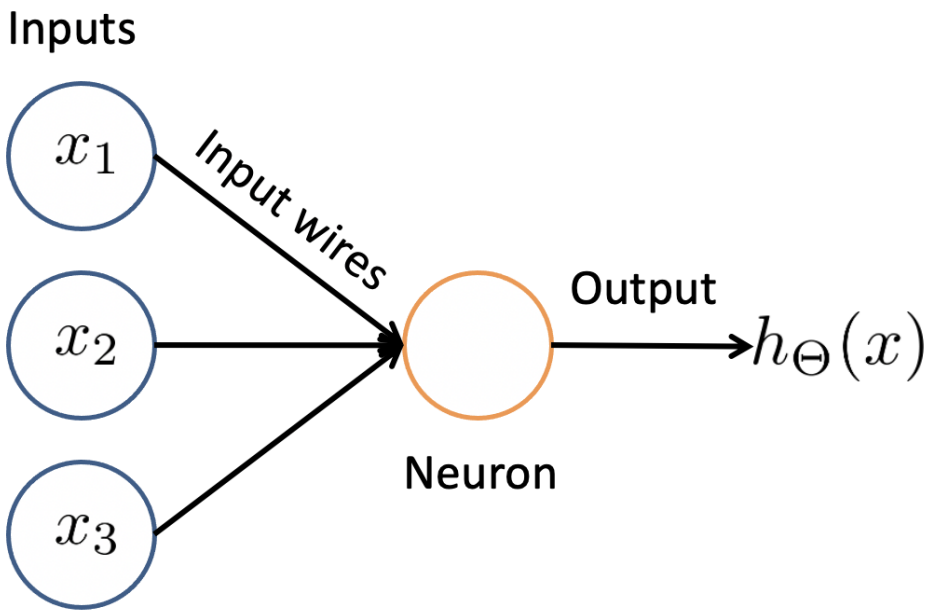
Here we model a neuron as a logistic unit.



- We have several inputs, e.g. x_1, x_2, x_3.

Neuron model: logistic unit

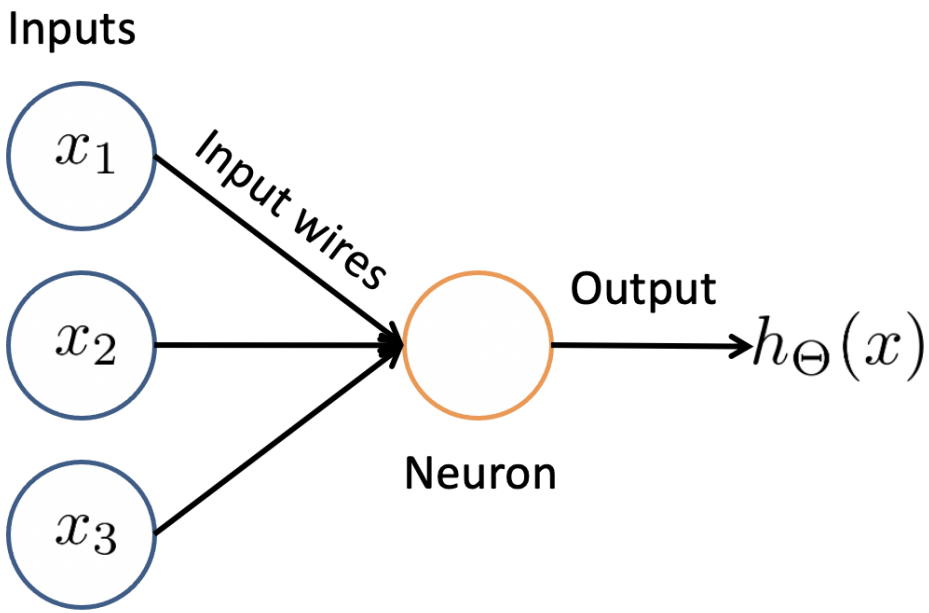
Here we model a neuron as a logistic unit.



- We have several inputs, e.g. x_1, x_2, x_3.
- We call the connection from the input to the neuron "input wires".

Neuron model: logistic unit

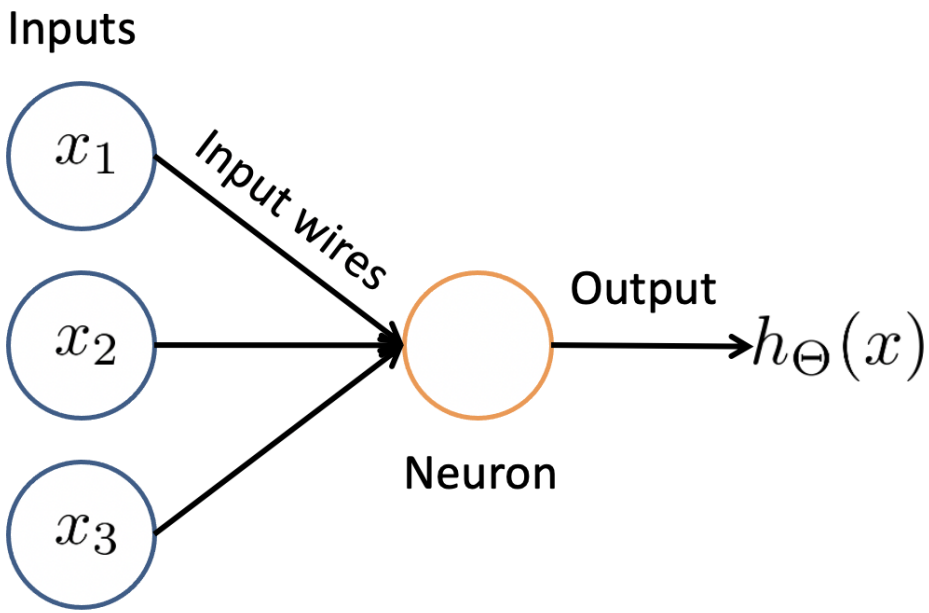
Here we model a neuron as a logistic unit.



- We have several inputs, e.g. x_1, x_2, x_3.
- We call the connection from the input to the neuron "input wires".
- We call the connection from the neuron to the output "output wire".

Neuron model: logistic unit

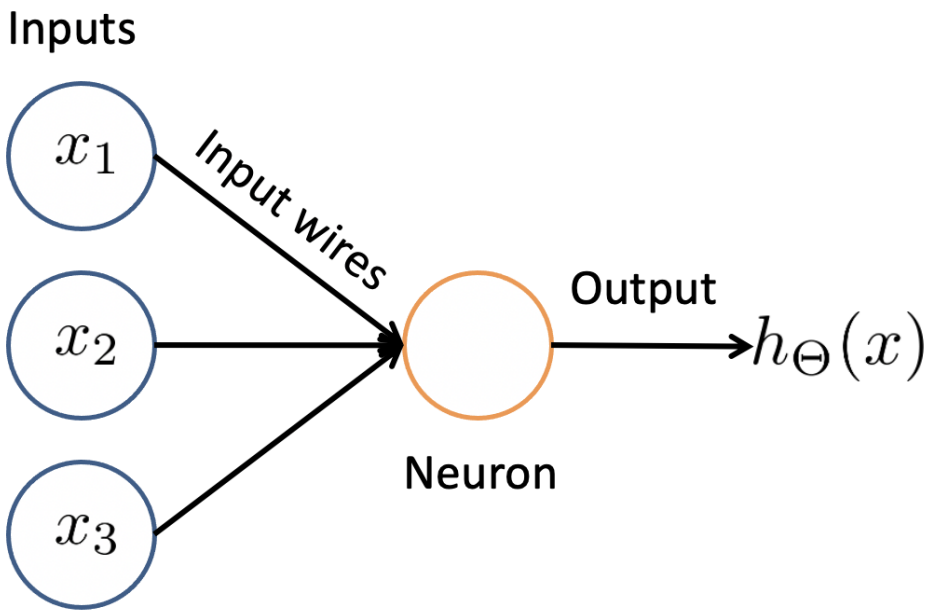
Here we model a neuron as a logistic unit.



- Here, $h_\Theta(x) = \frac{1}{1+e^{-x\Theta}}$ is an activation function, where $x = [x_1, x_2, x_3]$, $\Theta = [\theta_1, \theta_2, \theta_3]^T$.

Neuron model: logistic unit

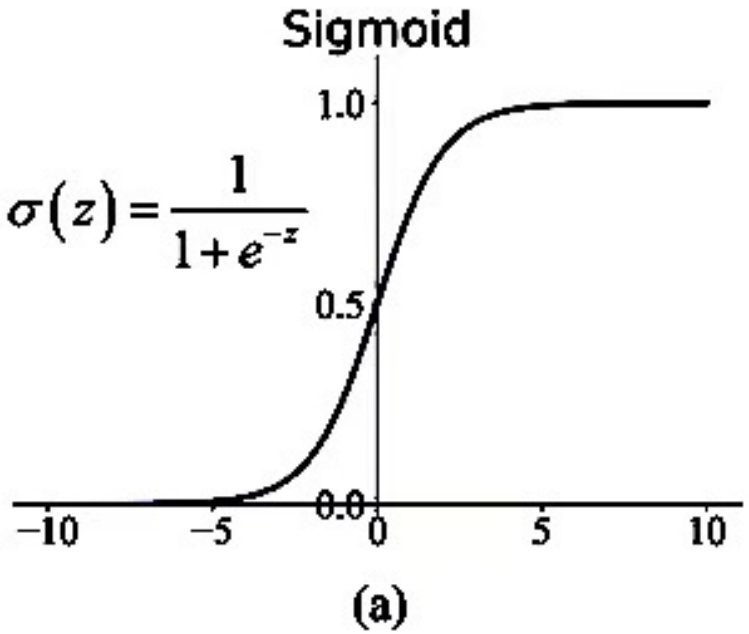
Here we model a neuron as a logistic unit.



- Here, $h_\Theta(x) = \frac{1}{1+e^{-x\Theta}}$ is an activation function, where $x = [x_1, x_2, x_3]$, $\Theta = [\theta_1, \theta_2, \theta_3]^T$.
- Θ is often called the "weights" of model or the "parameters" of model.

Activation functions (non-linear functions)

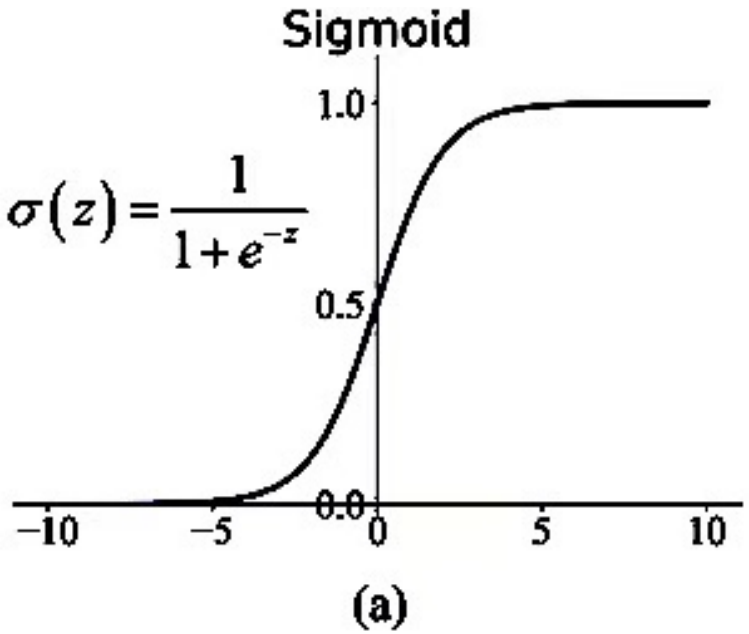
- Sigmoid



- Sigmoid with the mathematical form $\sigma(x) = \frac{1}{1+e^{(-x)}}$, which squashes real numbers to range between $[0, 1]$.

Activation functions (non-linear functions)

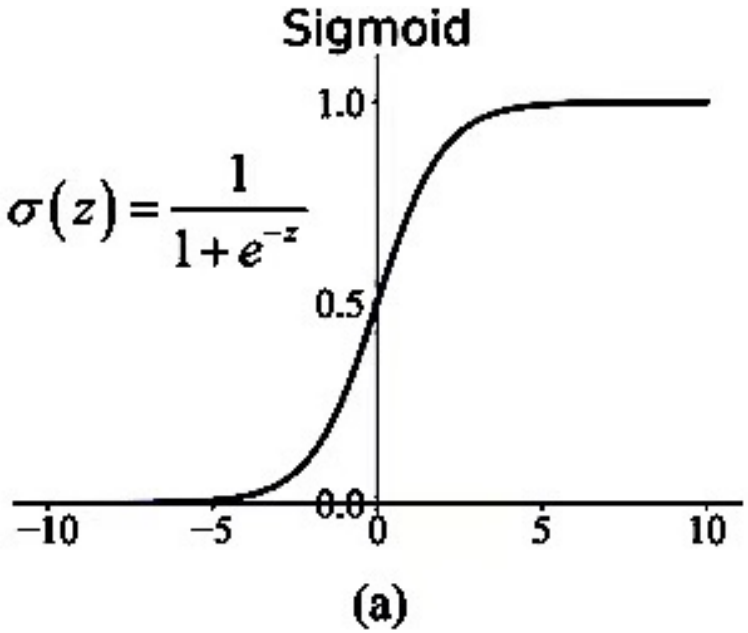
- Sigmoid



- Advantages:
  - Continuous function for easy derivation.
  - Limited output range, easy to optimize, can be used as output layer.

Activation functions (non-linear functions)

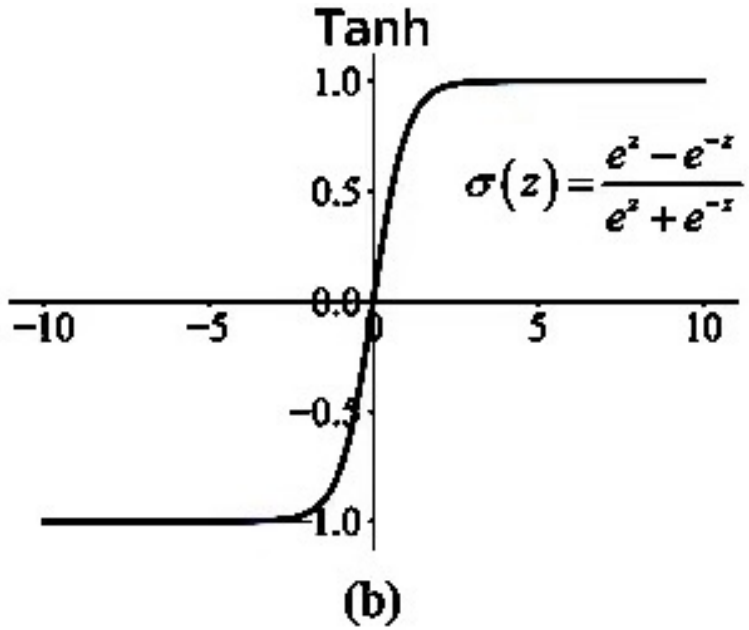
- Sigmoid



- Disadvantages:
  - When $x \to \pm\infty$, the gradient will be 0 (a.k.a. gradient vanish).
  - The computational complexity is high (exponential form).
  - The output mean value is not centered at 0, resulting in a decrease in the convergence speed.

Activation functions (non-linear functions)

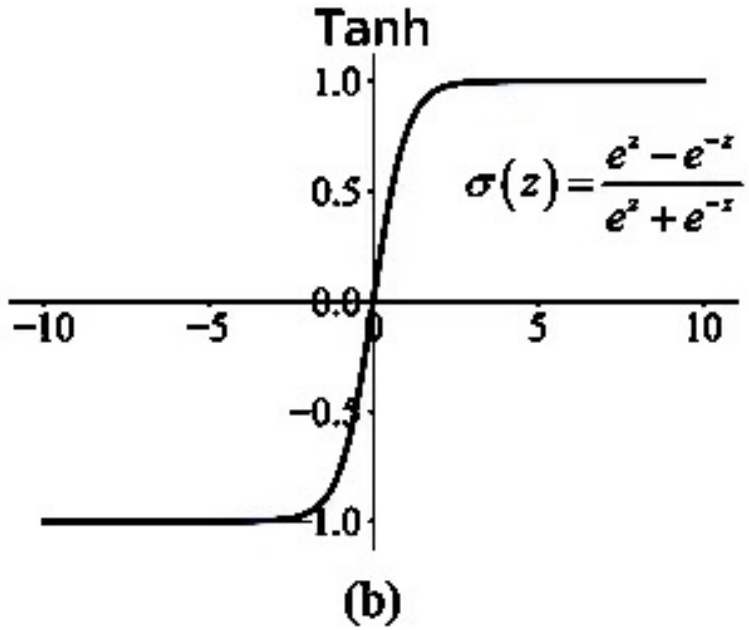
- Tanh



- Tanh with the mathematical form $tanh(x) = 2\sigma(x) - 1$, which squashes real numbers to range between $[-1, 1]$.

Activation functions (non-linear functions)

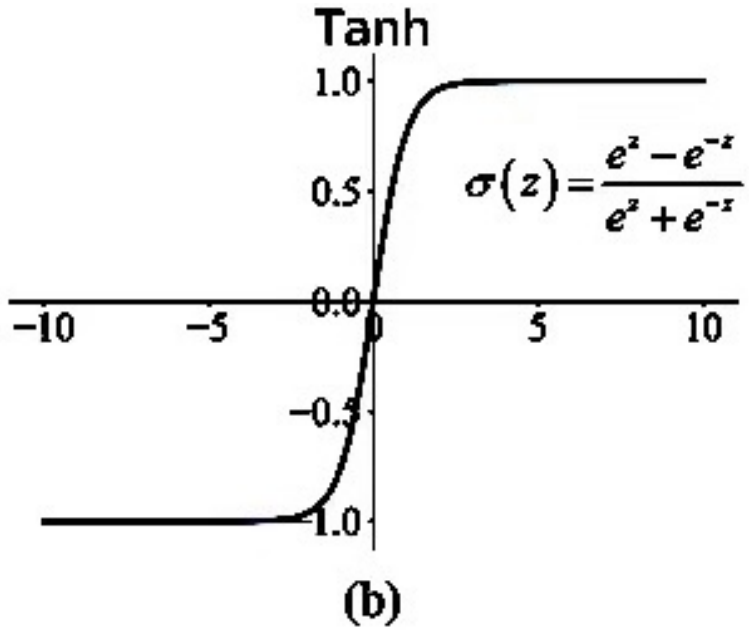
- Tanh



- Advantages:
  - The output mean value is centered at 0, resulting in converging faster than sigmoid.

Activation functions (non-linear functions)

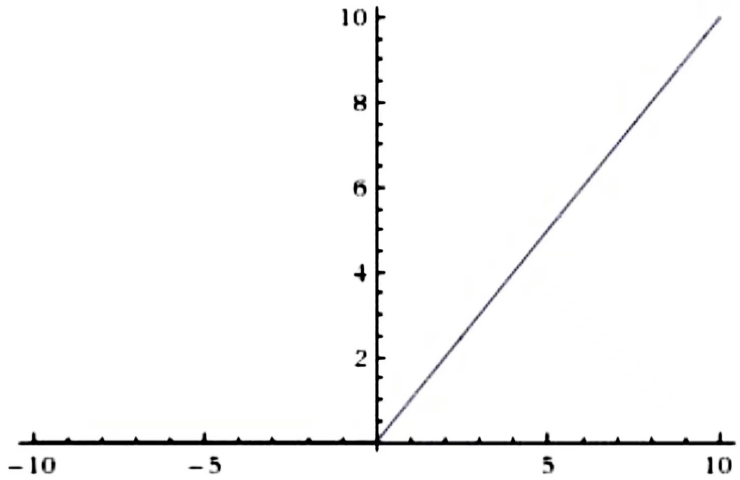
- Tanh



- Disadvantages:
  - When $x \to \pm\infty$, the gradient will be 0 (a.k.a. gradient disappearance).
  - The computational complexity is high (exponential form).

Activation functions (non-linear functions)

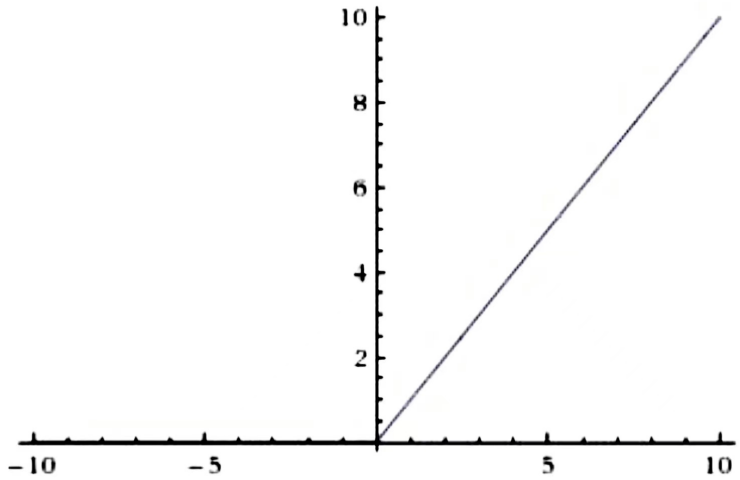
- ReLU (Rectified Linear Unit)



- ReLU with the mathematical form $f(x) = max(0, x)$, , which is zero when $x < 0$ and then linear with slope 1 when $x \geq 0$.

Activation functions (non-linear functions)

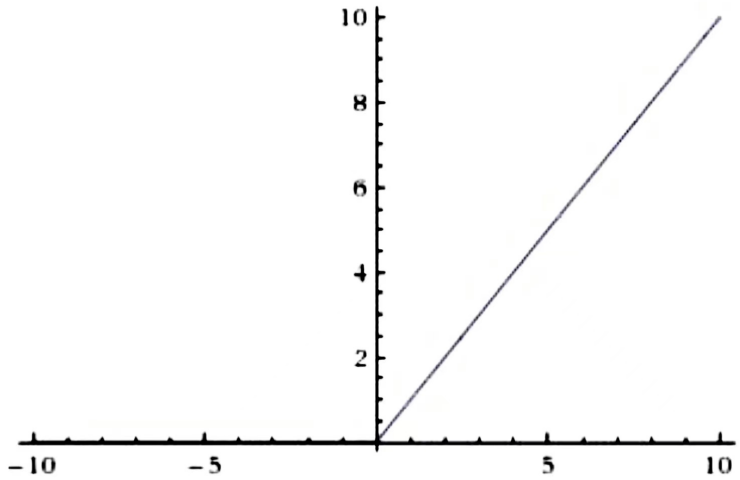
- ReLU (Rectified Linear Unit)



- Advantages:
  - The gradient is not saturated and the convergence speed is fast. Compared with sigmoid/tanh, it greatly improves the problem of gradient disappearance.
  - No exponential calculation is required, so the calculation speed is fast and the complexity is low.

Activation functions (non-linear functions)

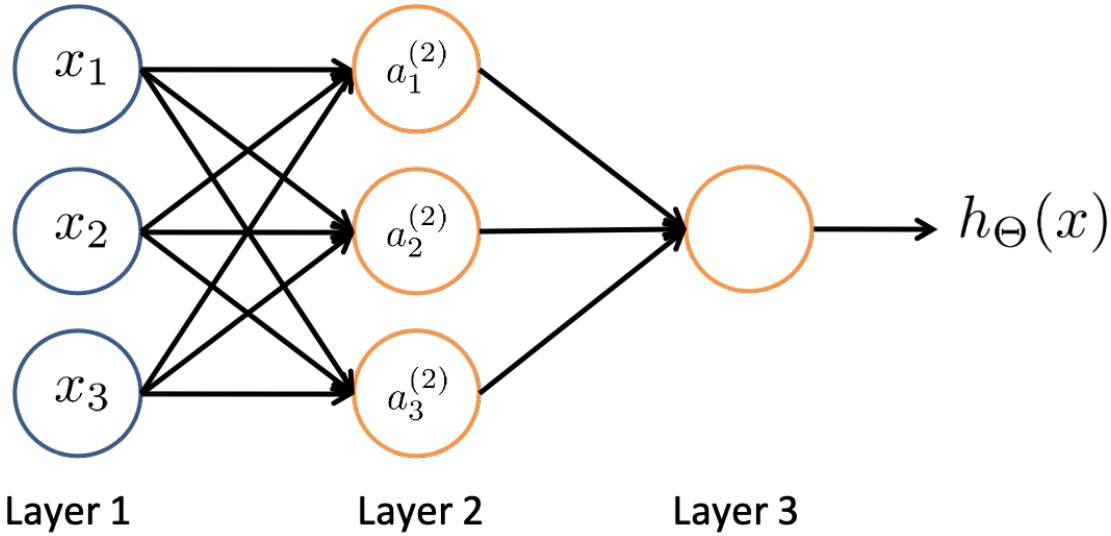
- ReLU (Rectified Linear Unit)



- Disdvantages:
  - When the input $x < 0$, the gradient is 0 and the parameters are not updated (a.k.a. neuron death).
  - The output mean value is greater than 0, which affects the convergence of the network.

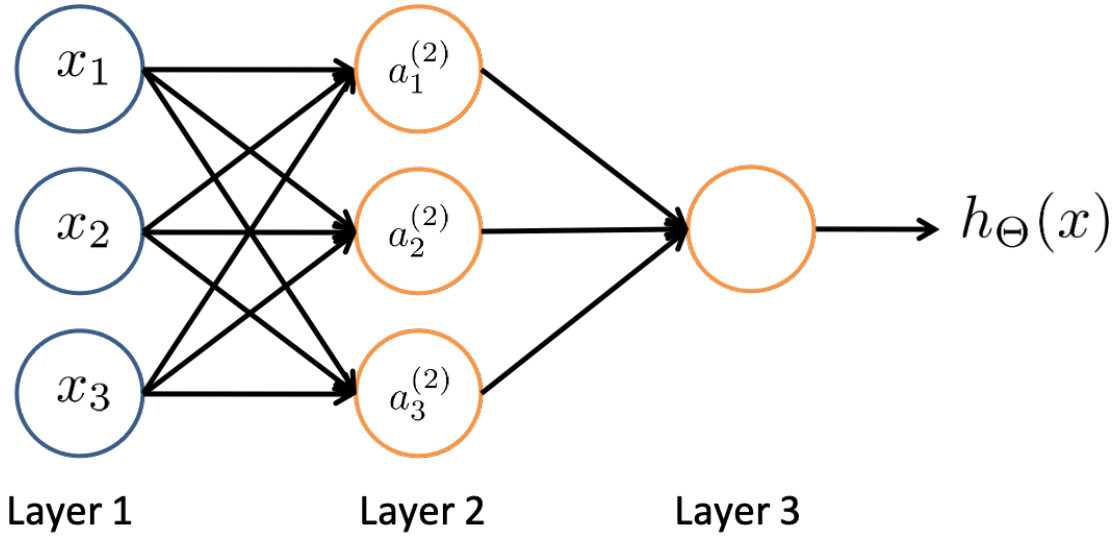
Feed forward neural network

A neuron network is just a groups of these neurons strung together.



Feed forward neural network

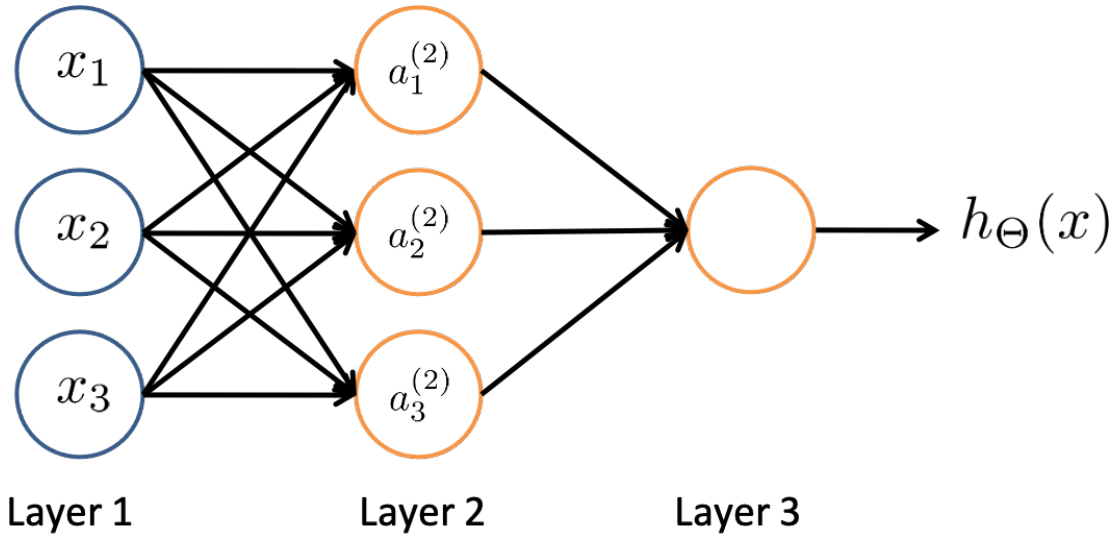
A neuron network is just a groups of these neurons strung together.



- We have input units x_1, x_2, x_3, three neurons, and the last neuron to compute $h_\Theta(x)$, where $a_i^{(j)}$ denotes the "activation" of unit i in layer j.

Feed forward neural network

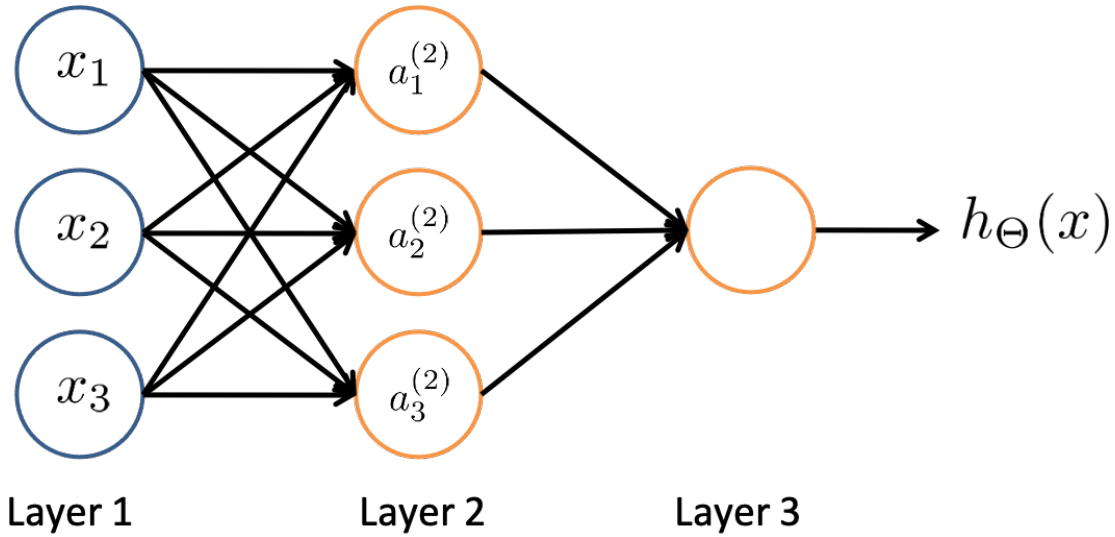
A neuron network is just a group of these neurons strung together.



- The first layer (Layer 1) is called the input layer because this is where the features x_1, x_2, x_3 are input.

Feed forward neural network

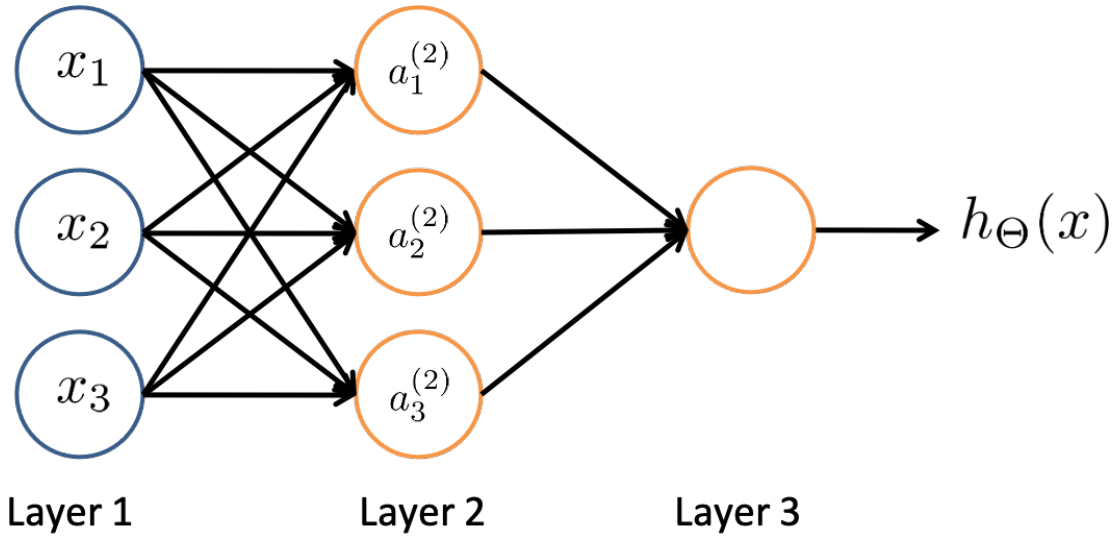
A neuron network is just a group of these neurons strung together.



- The first layer (Layer 1) is called the input layer because this is where the features x_1, x_2, x_3 are input.
- The final layer (Layer 3) is called the output layer because that layer has the neuron that outputs the final value computed.

Feed forward neural network

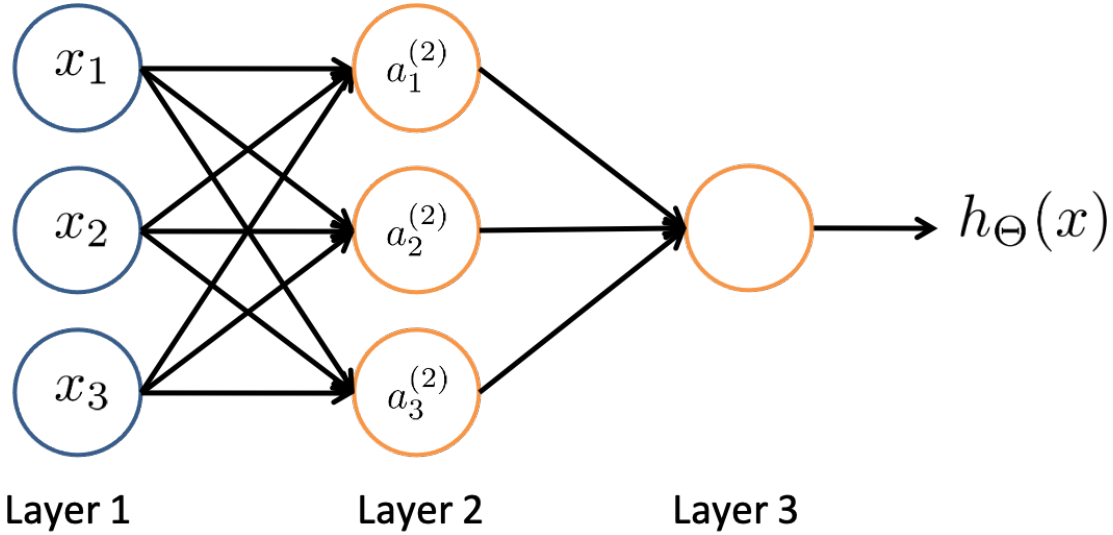
A neuron network is just a group of these neurons strung together.



- The first layer (Layer 1) is called the input layer because this is where the features x_1, x_2, x_3 are input.
- The final layer (Layer 3) is called the output layer because that layer has the neuron that outputs the final value computed.
- The layer (Layer2) between the input and output layer is call the hidden layer as its value can not be seen.

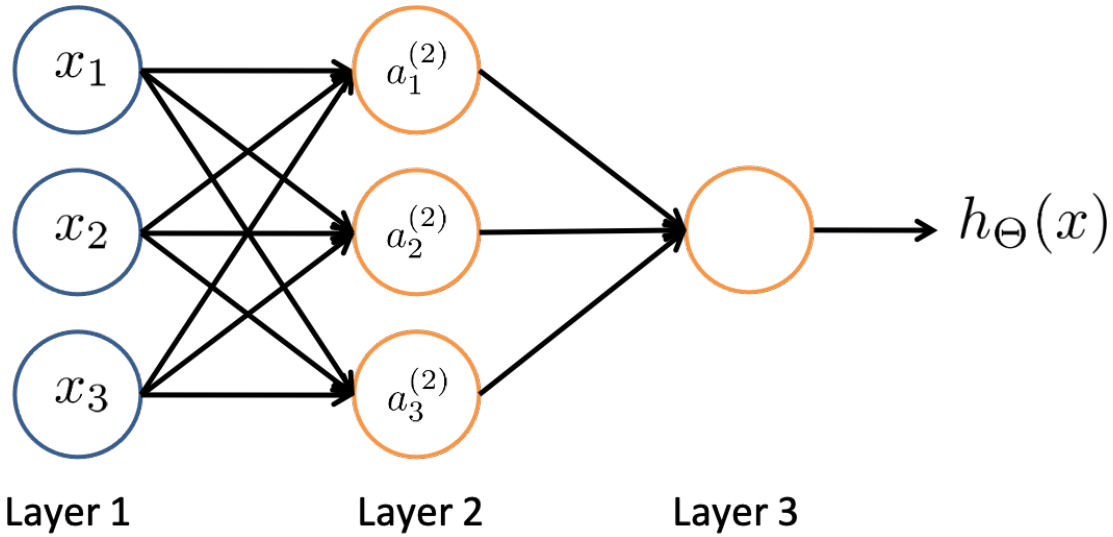
Feed forward neural network

The computational steps in the neural network.



Feed forward neural network

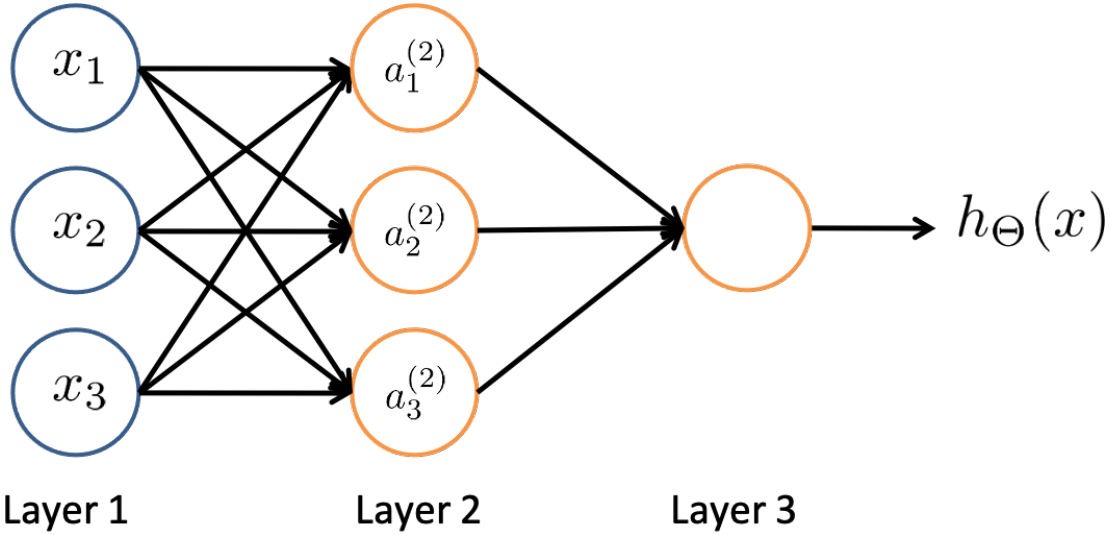
The computational steps in the neural network.



- $a_i^{(j)}$ = "activation" of unit i in layer j.
- $\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer $j+1$.
- $\Theta_{mn}^{(j)}$ = weight controlling function mapping from unit m in layer j to unit n in layer $j+1$.

Feed forward neural network

The computational steps in the neural network.

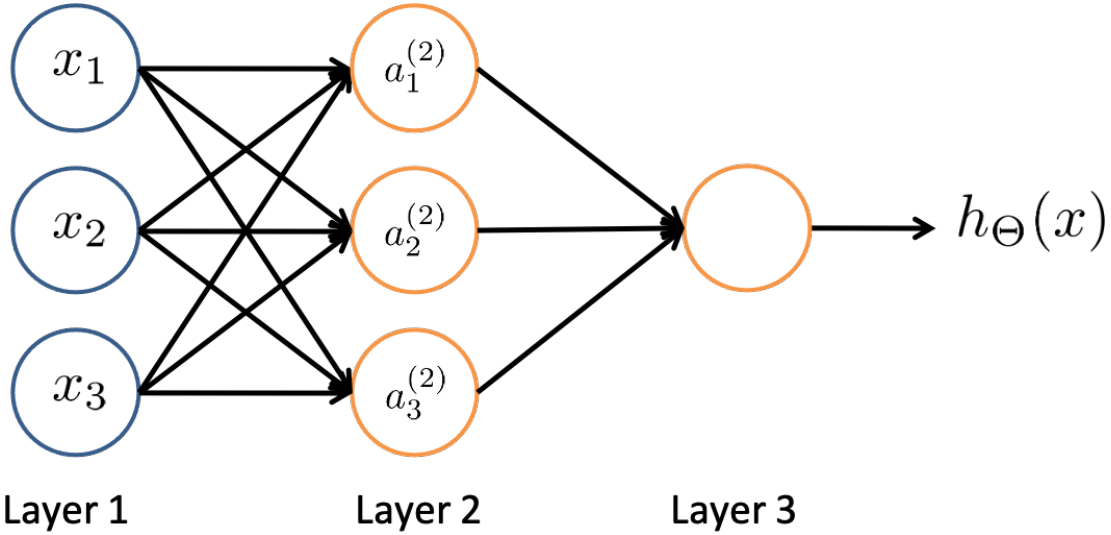


- $a_1^{(2)} = g(\Theta_{11}^{(1)} x_1 + \Theta_{21}^{(1)} x_2 + \Theta_{31}^{(1)} x_3)$
- $a_2^{(2)} = g(\Theta_{12}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{32}^{(1)} x_3)$
- $a_3^{(2)} = g(\Theta_{13}^{(1)} x_1 + \Theta_{23}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$

  where g is the sigmoid activation function.

Feed forward neural network

The computational steps in the neural network.

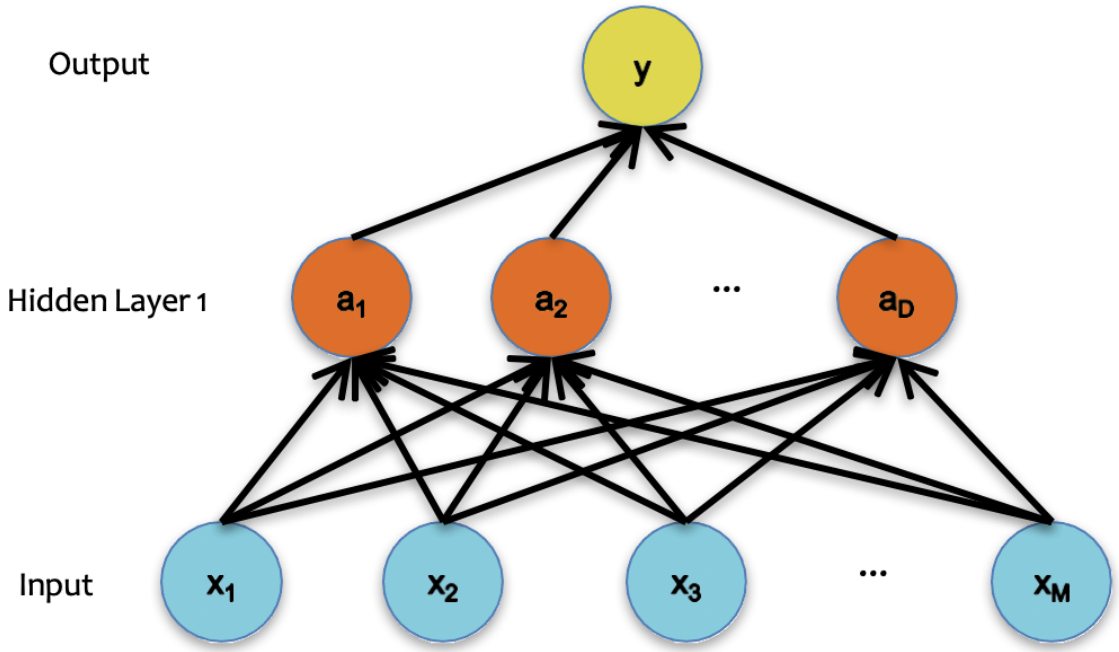


- $h_\Theta(x) = a_1^{(3)} = g(\Theta_{11}^{(2)} a_1^{(2)} + \Theta_{21}^{(2)} a_2^{(2)} + \Theta_{31}^{(2)} a_3^{(2)})$

  If network has s_j units in layer j, s_{j+1} units in layer $j+1$, then $\Theta^{(j)}$ will be of dimension $s_j \times s_{j+1}$.

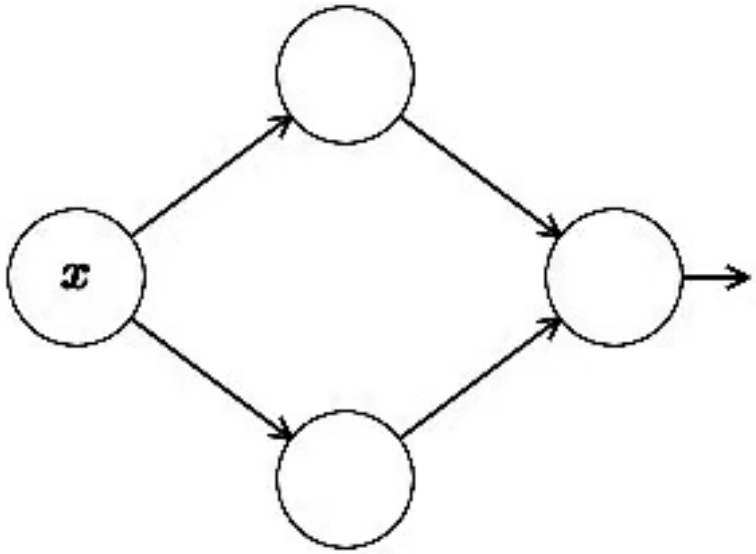
How many layers should we use?

- Single layer neural network is simple but can not learn a complex pattern. So, here comes a problem, how many layers should we stack in a neural network?
  (In other words, how many hidden layers do we need?)



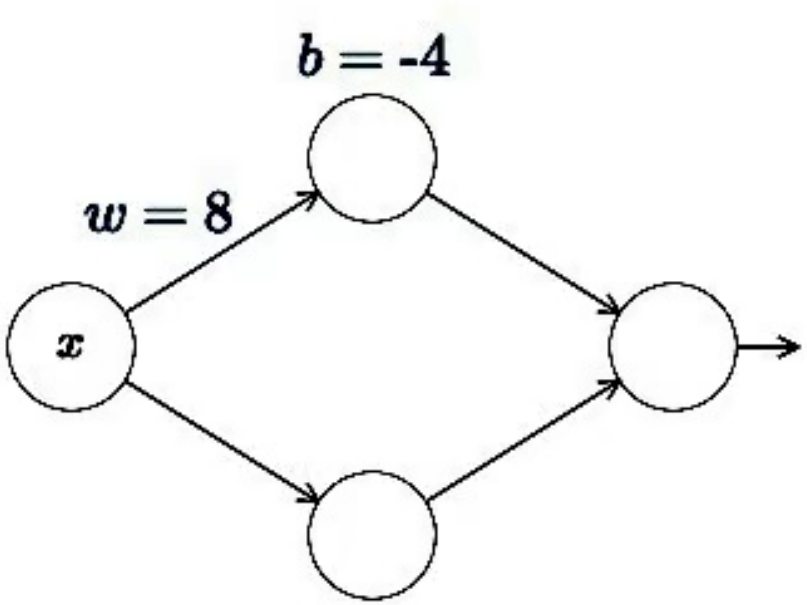
How neural networks model function?

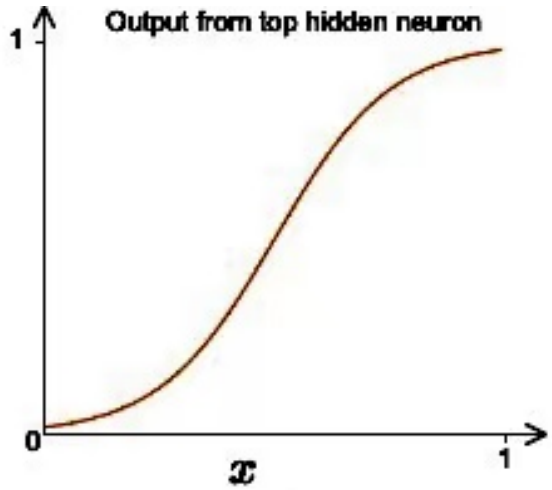
Let's first start with a simple network just has one input neuron, one hidden layer (with two neurons), and one output neuron:



How neural networks model function?

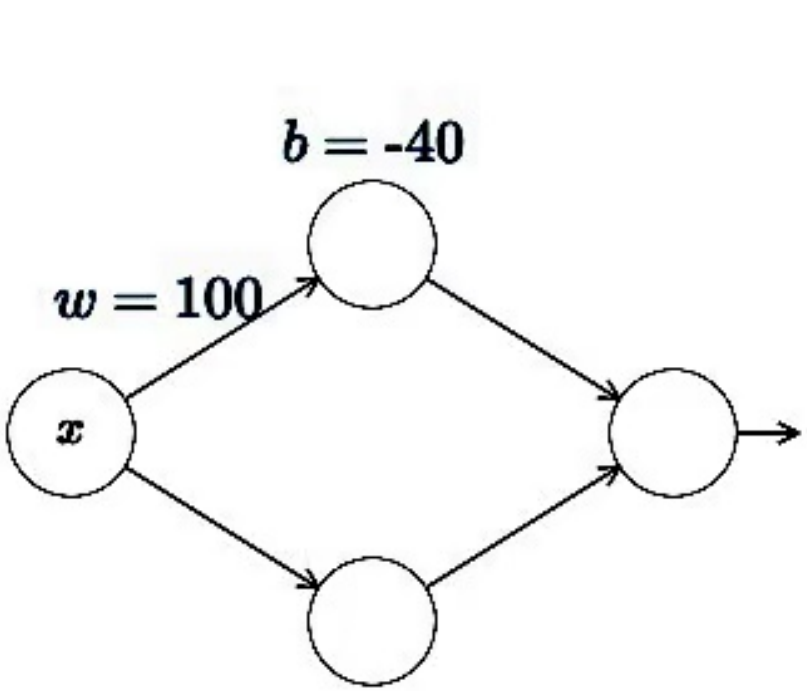
- Suppose manually setting $w = 8$ and $b = -4$, using Sigmoid function at output neuron, we will get the function curve of $y = f(x) = \frac{1}{1+e^{-(wx+b)}}$.

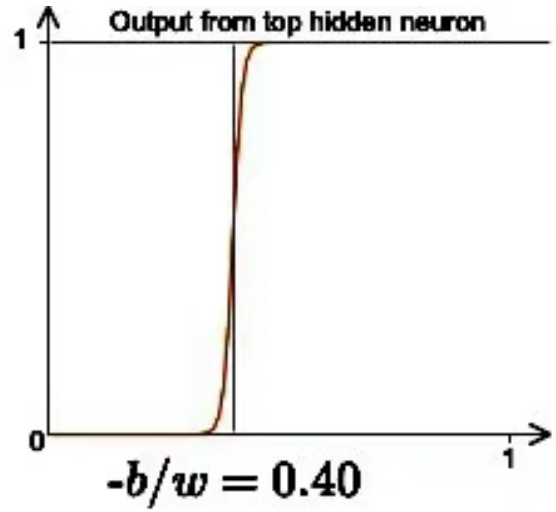




How neural networks model function?

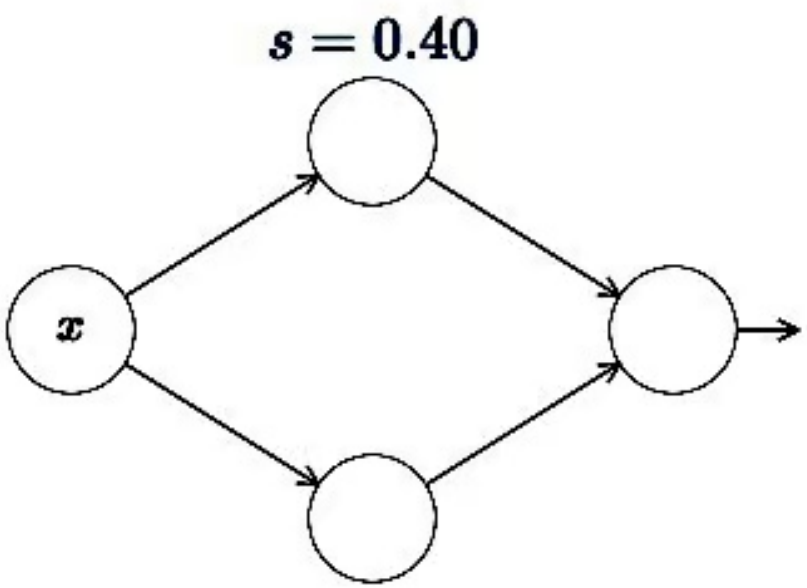
- If we increase w and decrease b, for example, $w = 100$, $b = -40$, then the curve $y = f(x)$ becomes steeper (just like a step function).

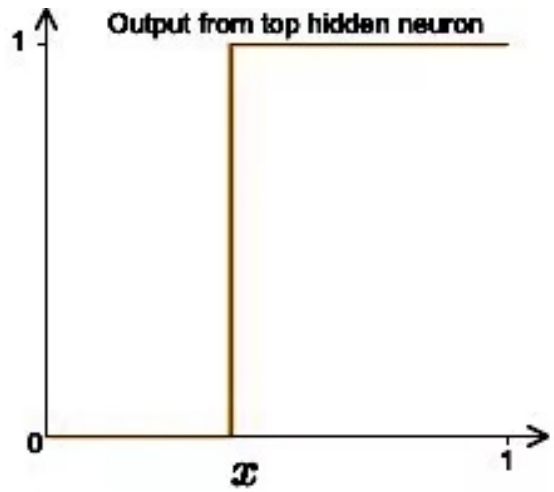




How neural networks model function?

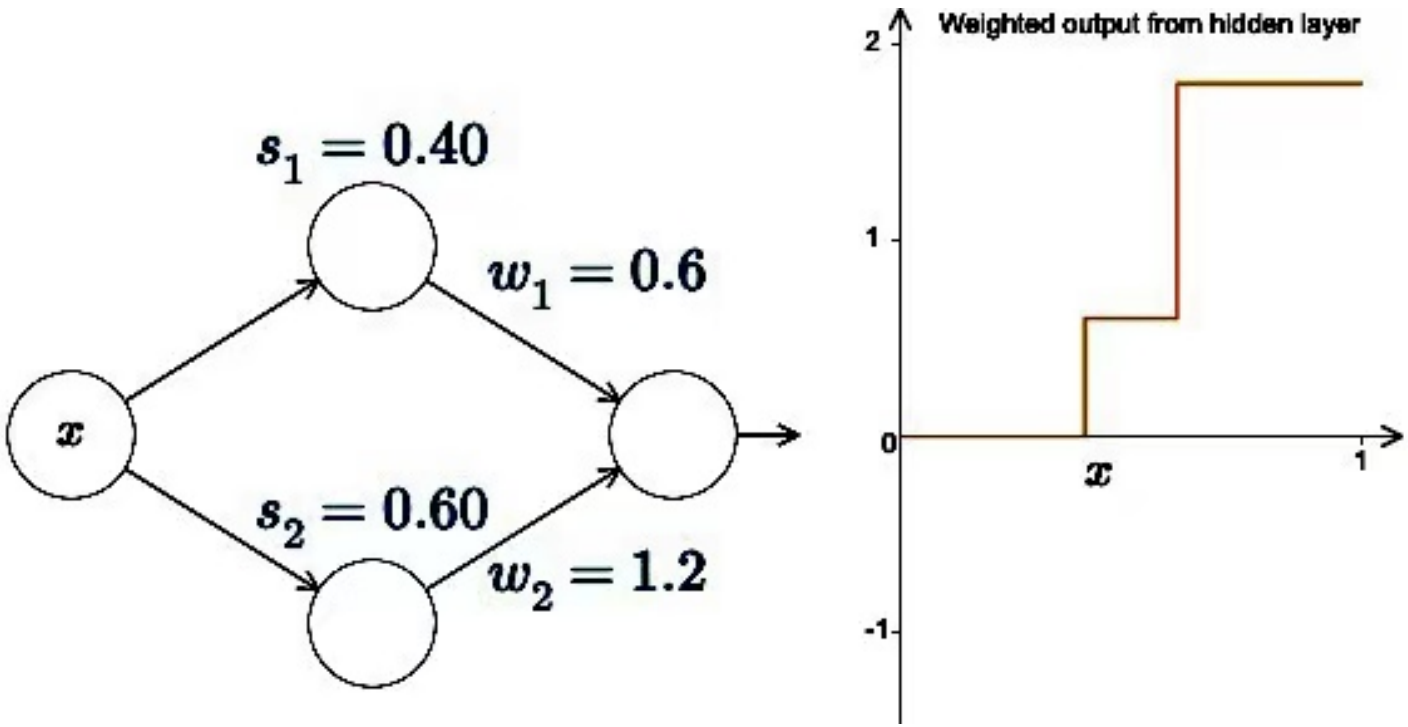
- To simplify our lives for describing hidden neurons using just a single parameter, s, which is the step position, $s = b/w$. Now, the previous curve can be approximated by the following:





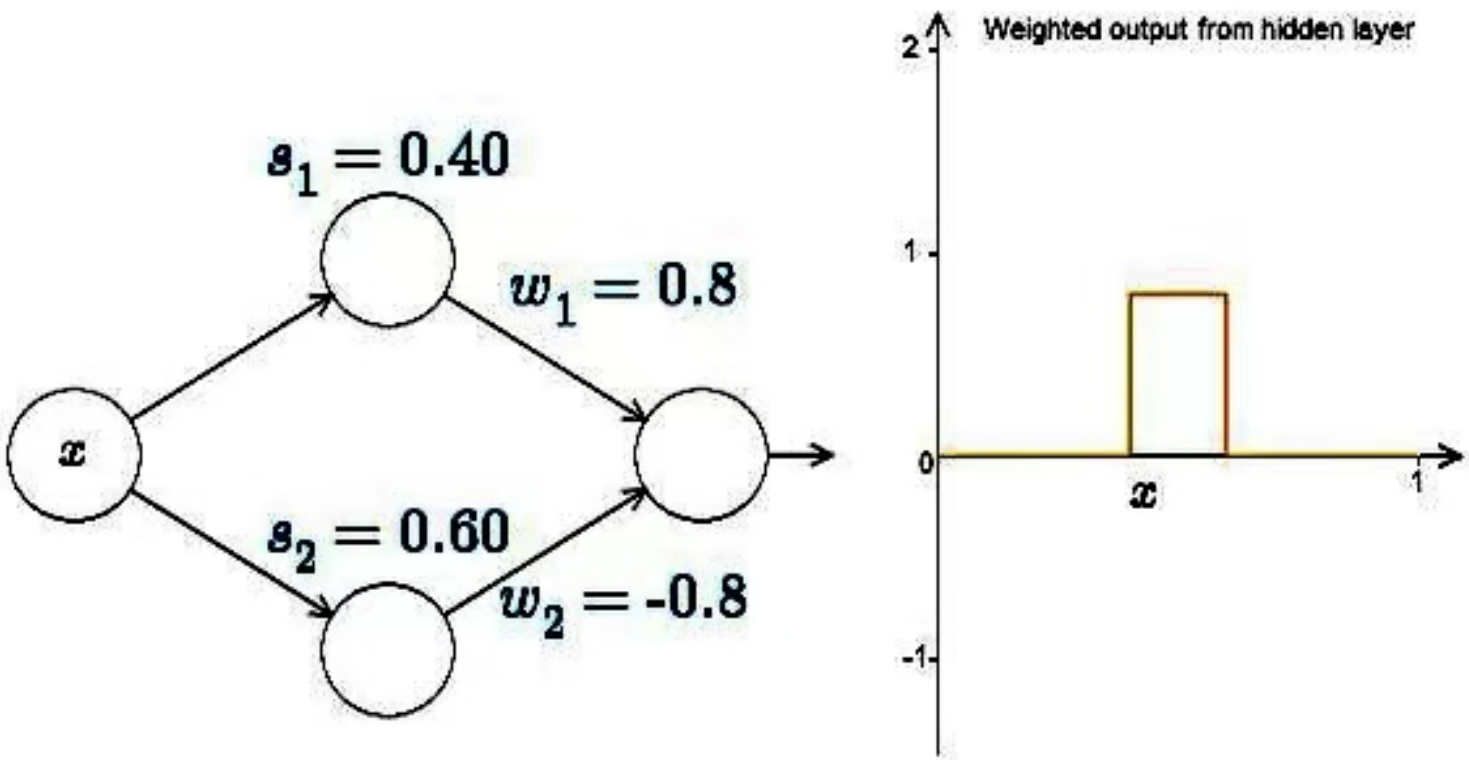
How neural networks model function?

- For the entire network, in particular, we'll suppose the hidden neurons are computing step functions parameterized by step points s_1 (top neuron) and s_2 (bottom neuron). And they'll have respective output weights w_1 and w_2. What's being plotted on the right is the weighted output $w_1 a_1 + w_2 a_2$ from the hidden layer. Here, a_1 and a_2 are the outputs from the top and bottom hidden neurons, respectively.



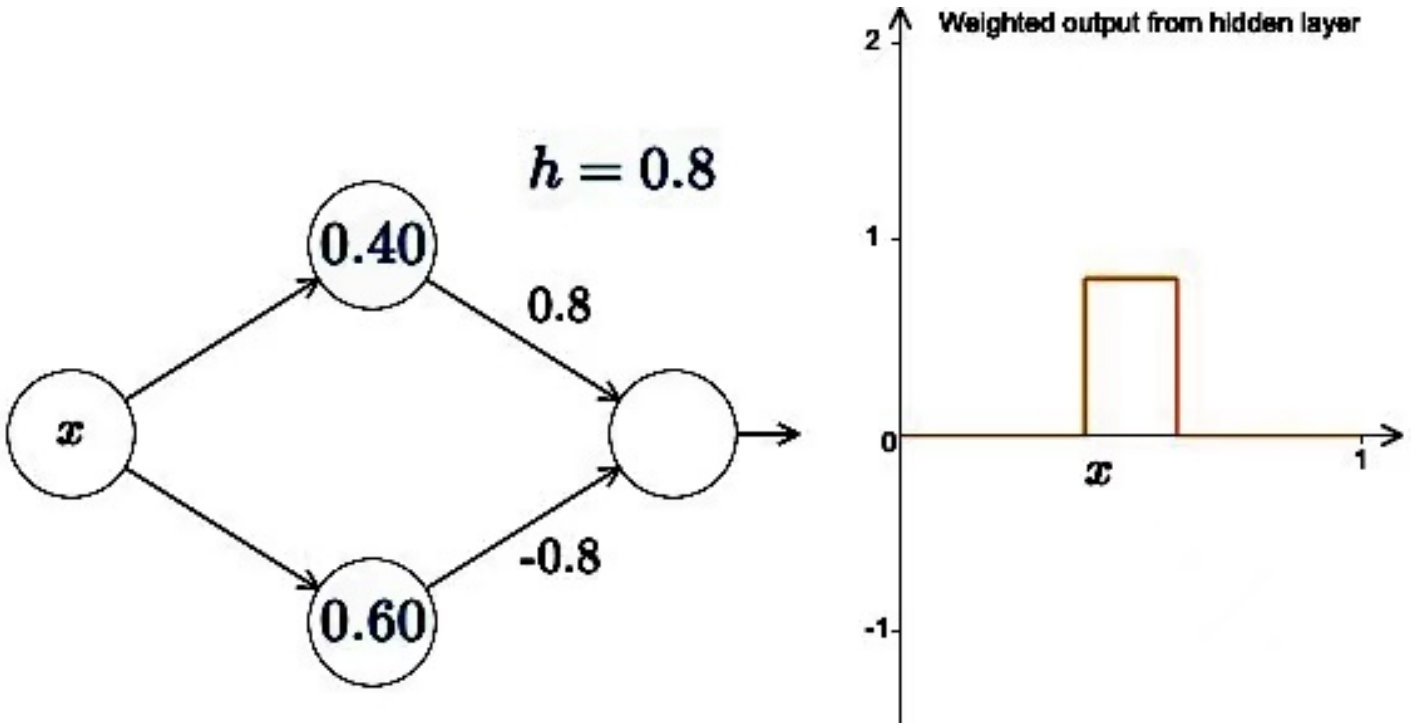
How neural networks model function?

- Besides, try setting w_1 to be 0.8 and w_2 to be -0.8. We get a "bump" function, which starts at point s_1, ends at point s_2, and has height 0.8. For instance, the weighted output might look like this:



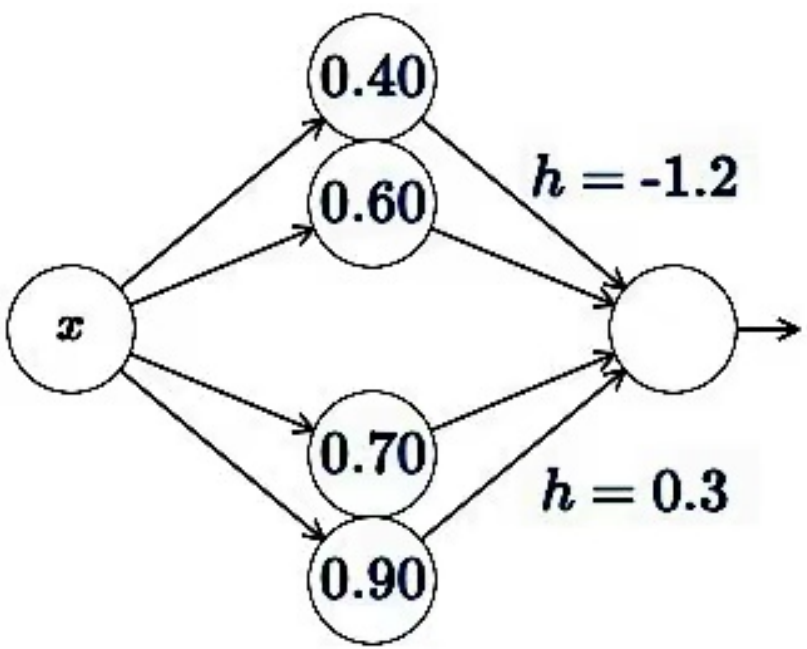
How neural networks model function?

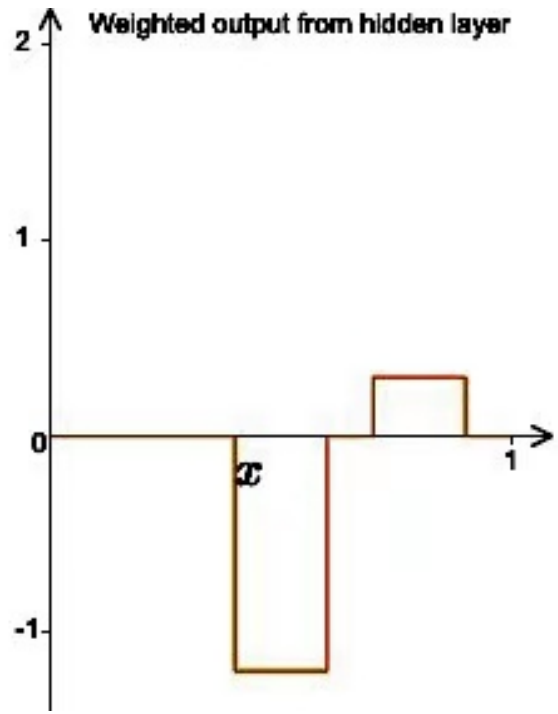
- For further simplification, let's use a single parameter, h, to denote the height.



How neural networks model function?

- Further, we can use our bump-making trick to get two bumps, by gluing two pairs of hidden neurons together into the same network with two pairs of h ($h = -1.2; h = 0.3$, respectively):





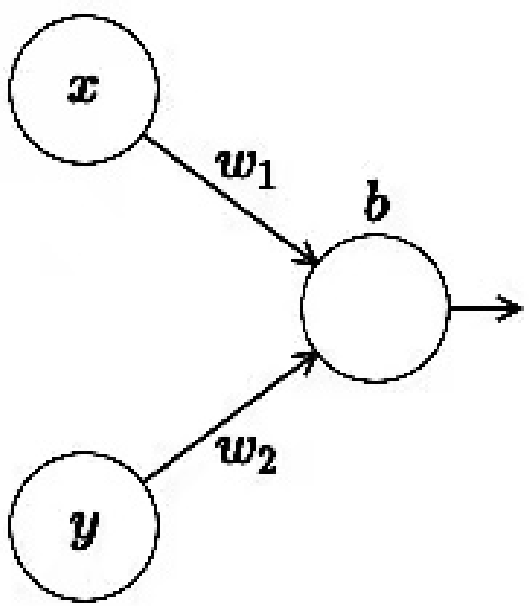
How neural networks model function?

Let‘s further consider what happens when we have two inputs to a neuron:

How neural networks model function?

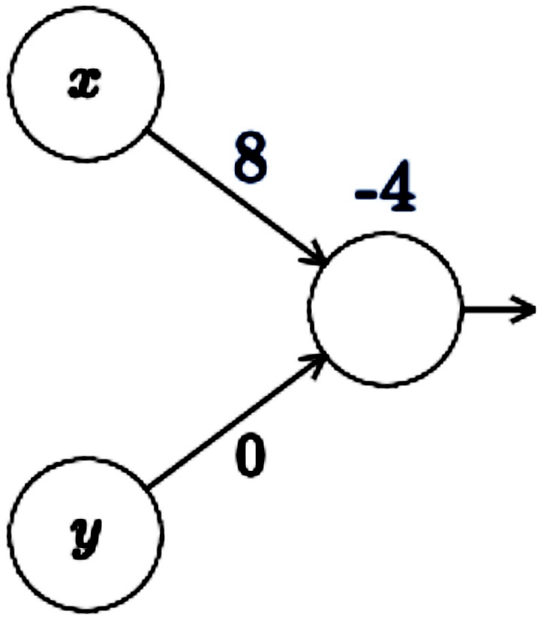
Let's further consider what happens when we have two inputs to a neuron:

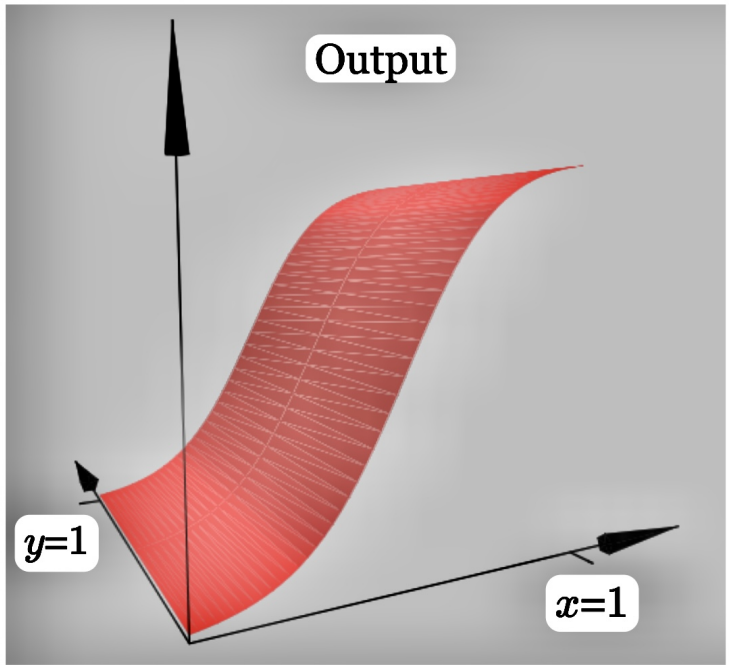
- Here we have inputs x and y, with corresponding weights w_1 and w_2, and a bias b on the neuron.



How neural networks model function?

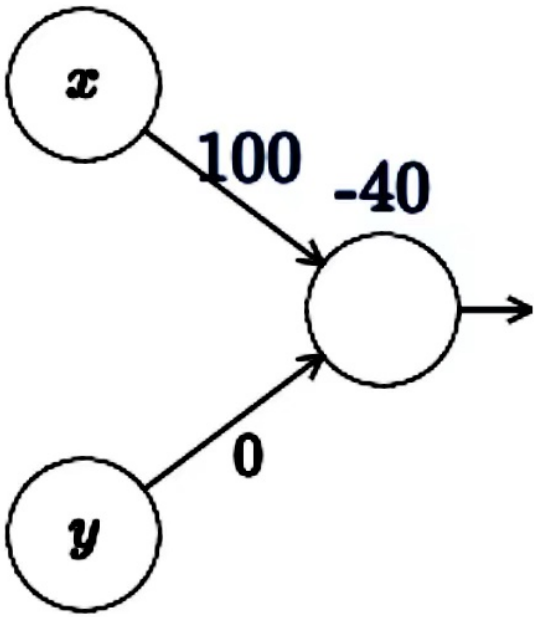
- Suppose manually setting $w_1 = 8$, $b = -4$ and $w_2 = 0$, using sigmoid function at output neuron, we will get the function curve of $y = f(x) = \frac{1}{1+e^{-(w_1 x + w_2 y + b)}}$.

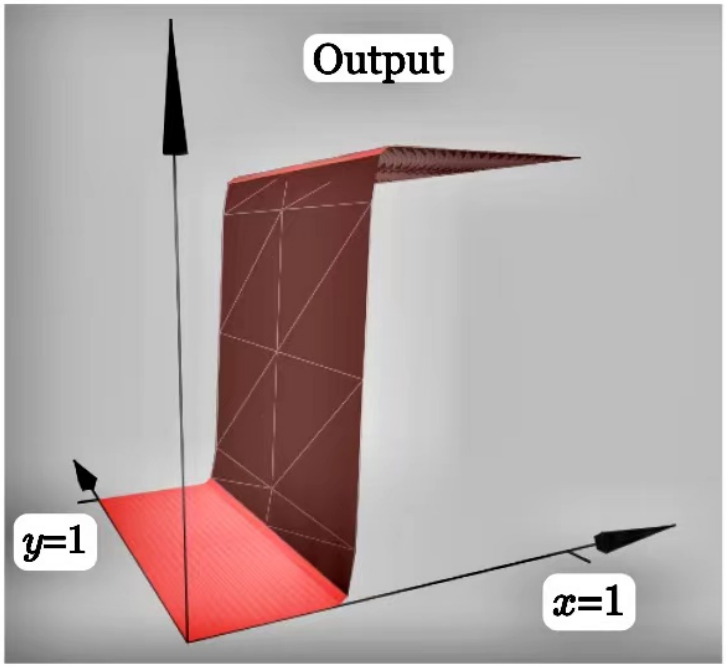




How neural networks model function?

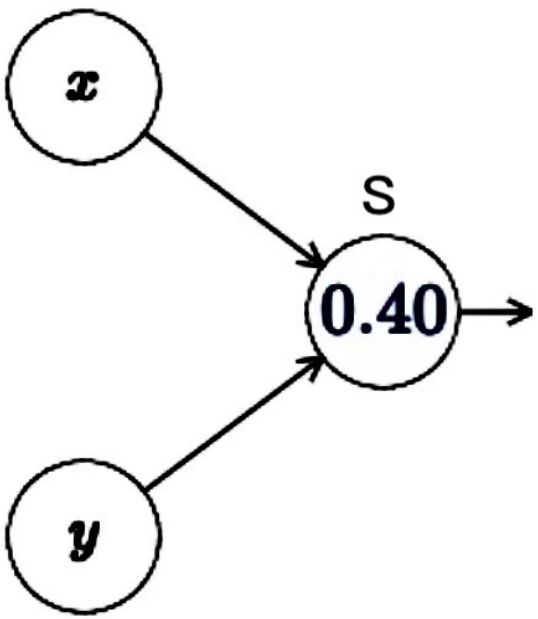
- If we increase w and decrease b, for example, $w = 100$, $b = -40$, then the curve $y = f(x)$ becomes steeper (just like a step function).

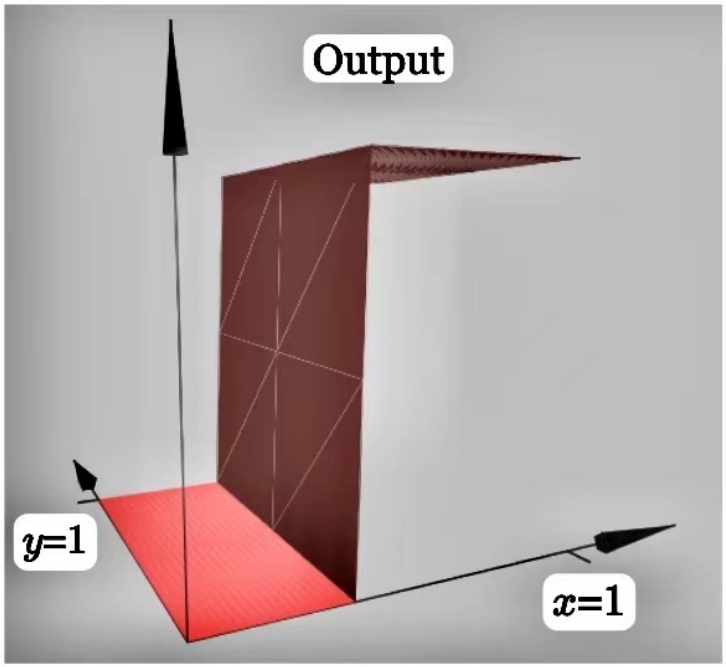




How neural networks model function?

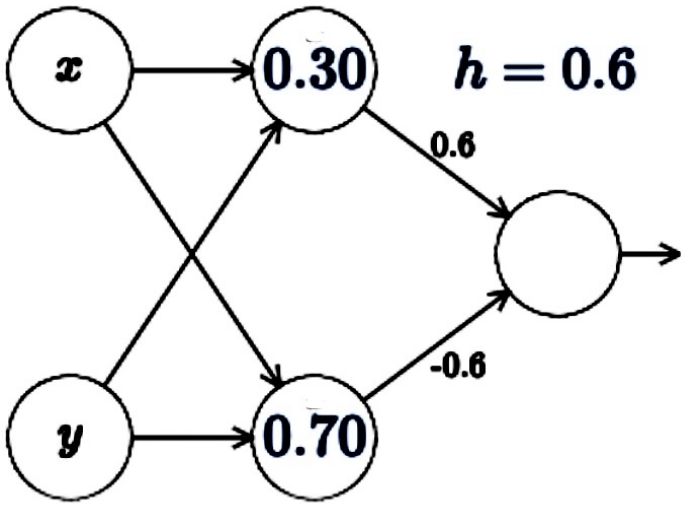
- To simplify our lives for describing hidden neurons using just a single parameter, s, which is the step position, $s = -b/w_1$. Now, the previous curve can be approximated by the following:

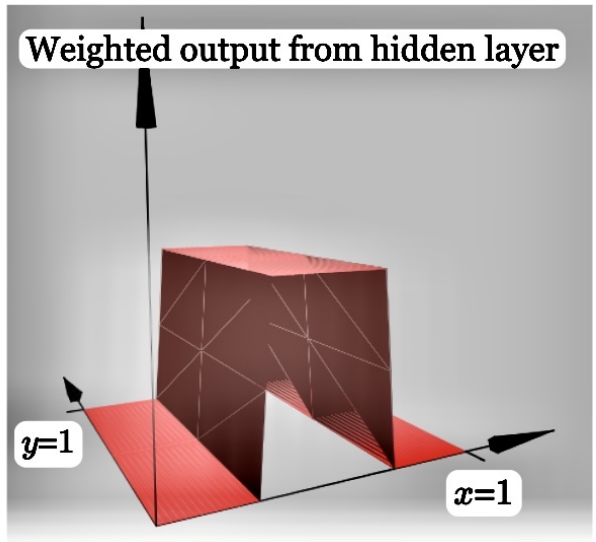




How neural networks model function?

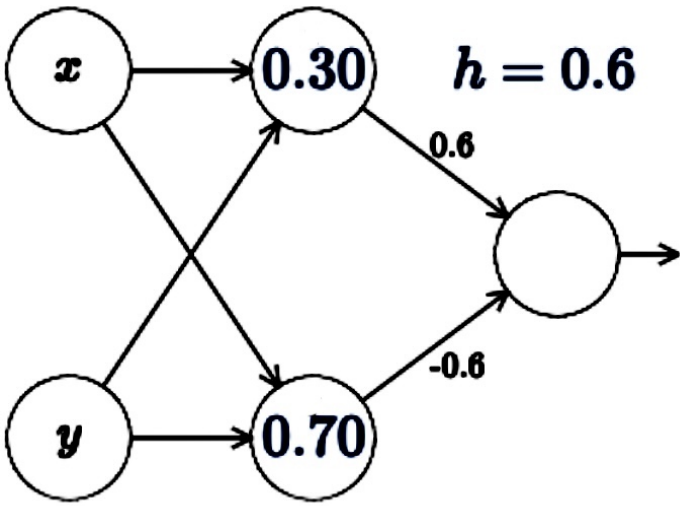
- Now we can use the step function we've just constructed to compute a three-dimensional bump function.
- We use two neurons, each computing a step function in the x direction.
- We combine those step functions with weight h, respectively, where h is the desired height of the bump. It's all illustrated in the following diagram:

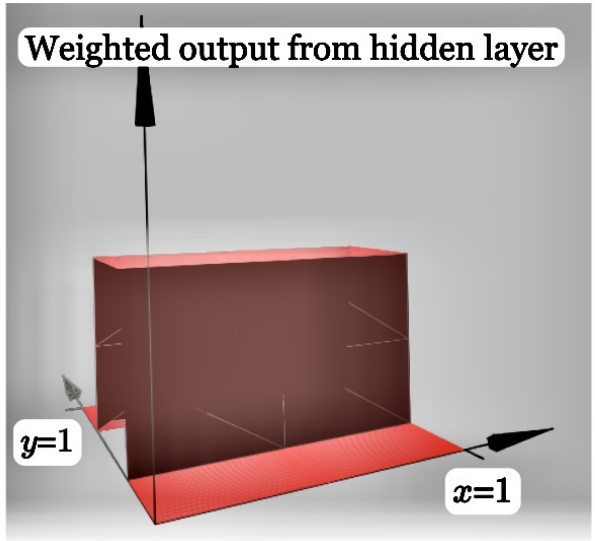




How neural networks model function?

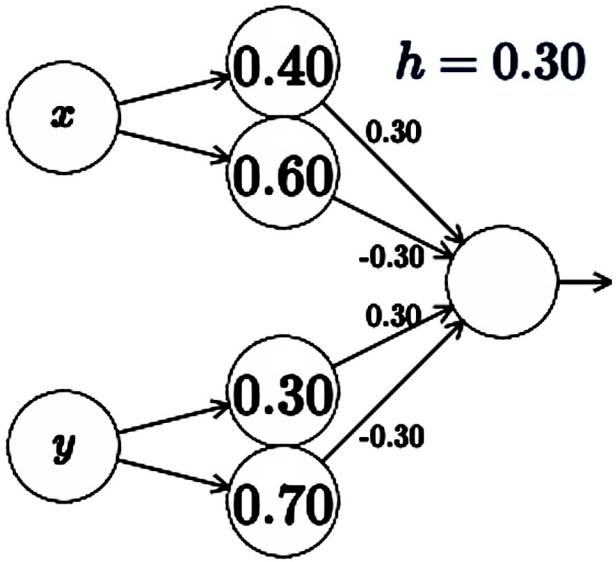
- Besides, we can easily make a bump function in the y direction, by using two step functions in the y direction. Recall that we do this by making the weight large on the y input, and the weight 0 on the x input. Here's the result:

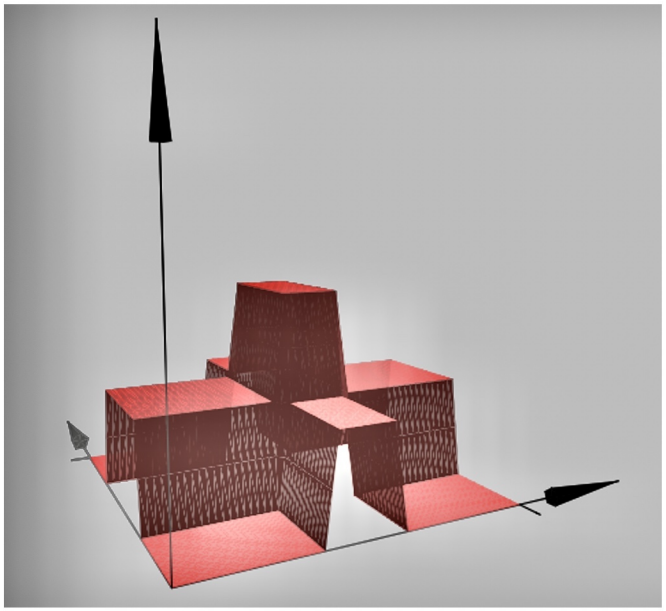




How neural networks model function?

- Let's consider what happens when we add up two bump functions, one in the x direction, the other in the y direction, both of height h:





Part 2. Backpropagation

Start to train a NN

A neural network basically consists of two components:

- The network architecture, which defines how many layers, how many neurons, and how the neurons are connected.

Start to train a NN

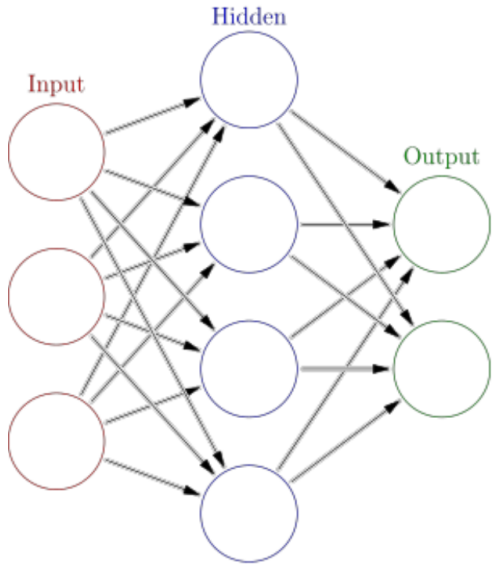
A neural network basically consists of two components:

- The network architecture, which defines how many layers, how many neurons, and how the neurons are connected.
- The parameters, values of the connections, model weights.

Start to train a NN

A neural network basically consists of two components:

- The network architecture, which defines how many layers, how many neurons, and how the neurons are connected.
- The parameters, values of the connections, model weights.
  - Before start actual training of a neural network, we need to select an initial value for these parameters.
  - In practice, randomly initialize the parameters to small values (e.g., normally distributed around zero; $\mathcal{N}(0, 0.1)$).
  - Then, we can begin training the NN to optimize parameters.

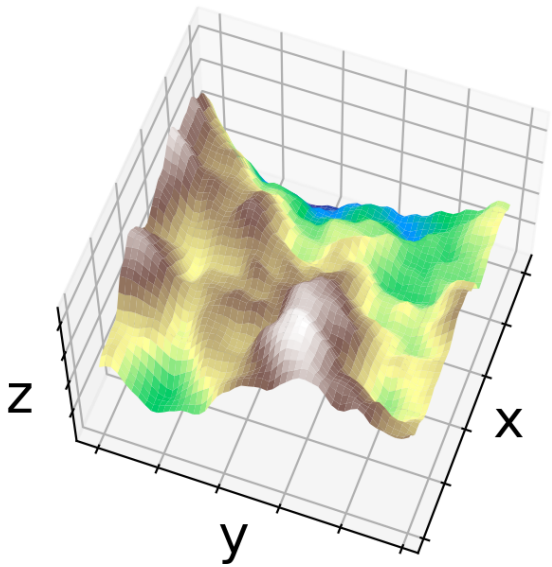


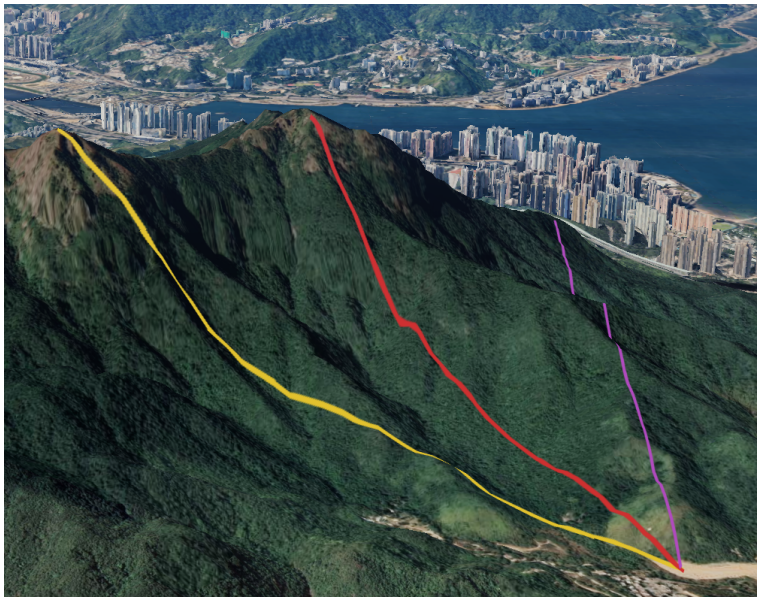
How to train a NN

- The **backpropagation** algorithm (Rumelhart et al., 1986) looks for the minimum of the cost function in weight space using the method of gradient descent. (recap?)
- Regarded as implementation of gradient descent in neural networks.
- A clever and efficient use of the Chain Rule for derivatives.
- Fundamental algorithm for training modern neural networks.

Recap: Gradient Descent

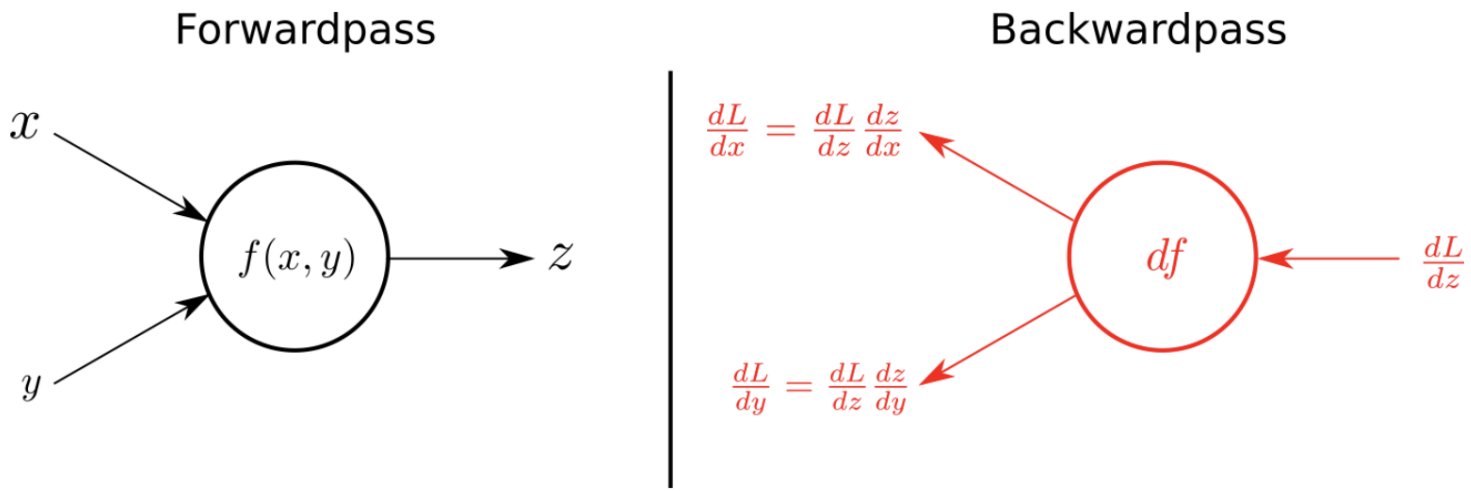
- Gradient descent moves opposite the gradient (the direction of steepest descent)





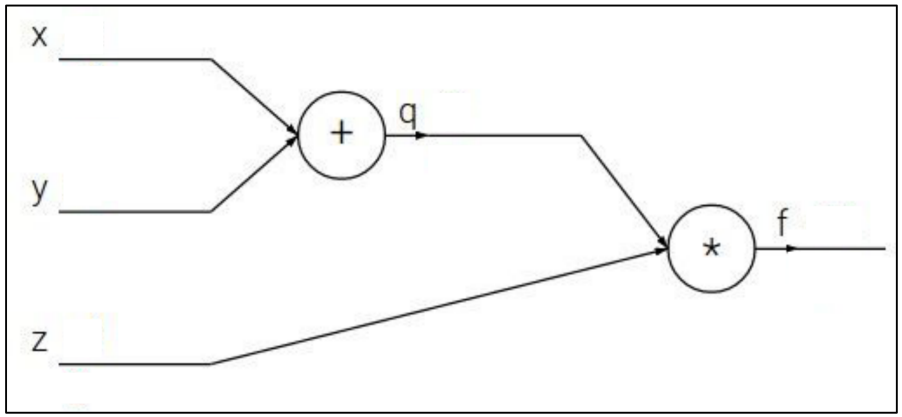
Overview of Backpropagation

- Initialize network with random weights
- For all training samples:
  - Input sample to the network and calculate the output (**forward pass**)
  - Compare the output with correct output to get loss term
  - For all layers (**backward pass** from output layer, to hidden, to inputs):
    - propagate the loss term back to previous layer
    - update the weights between the two layers
    (until the earliest hidden layer is reached)



Backpropatation - a simple example

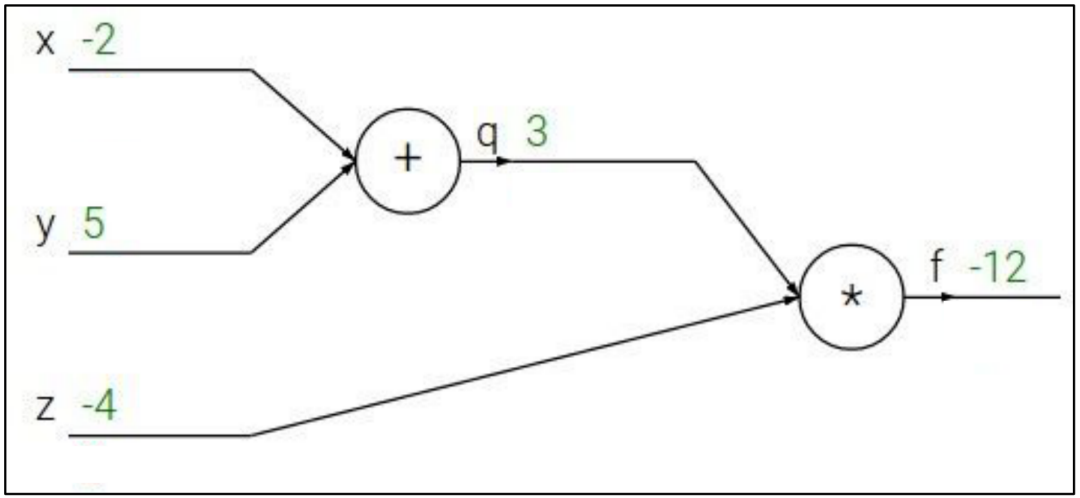
$$f(x, y, z) = (x + y)z$$



Backpropatation - a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

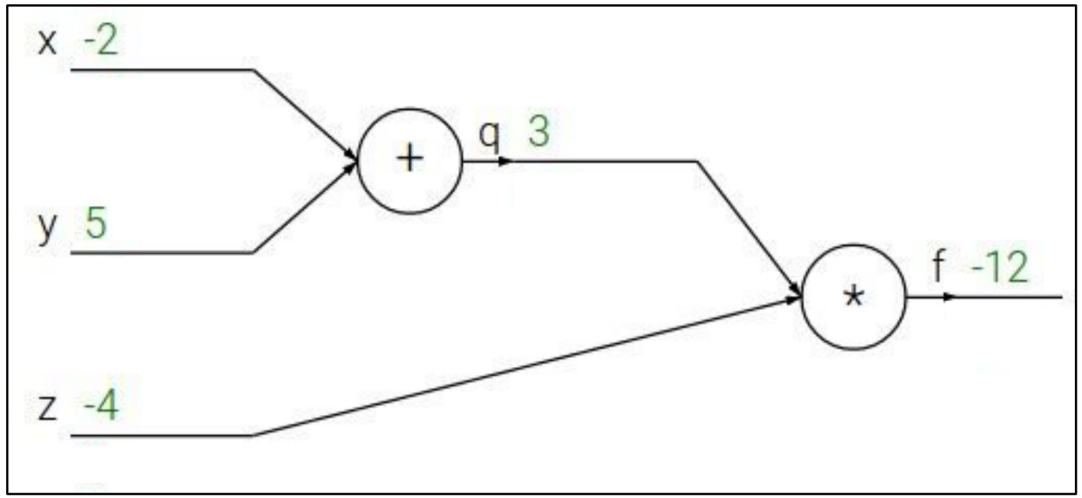


Backpropatation - a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



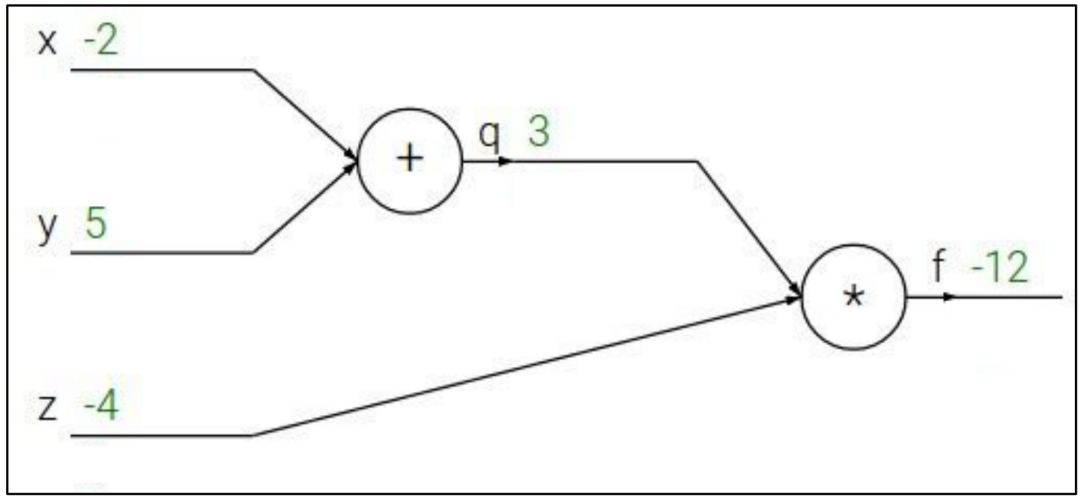
Backpropatation - a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

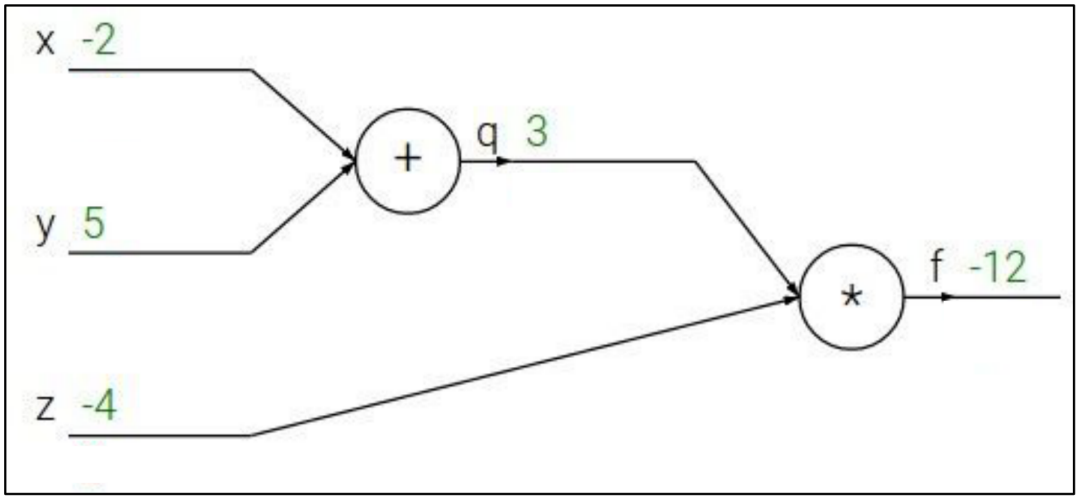
$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



Backpropatation - a simple example

$f(x, y, z) = (x + y)z$

e.g. $x = -2, y = 5, z = -4$



$q = x + y$
$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

$f = qz$
$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Backpropatation - a simple example

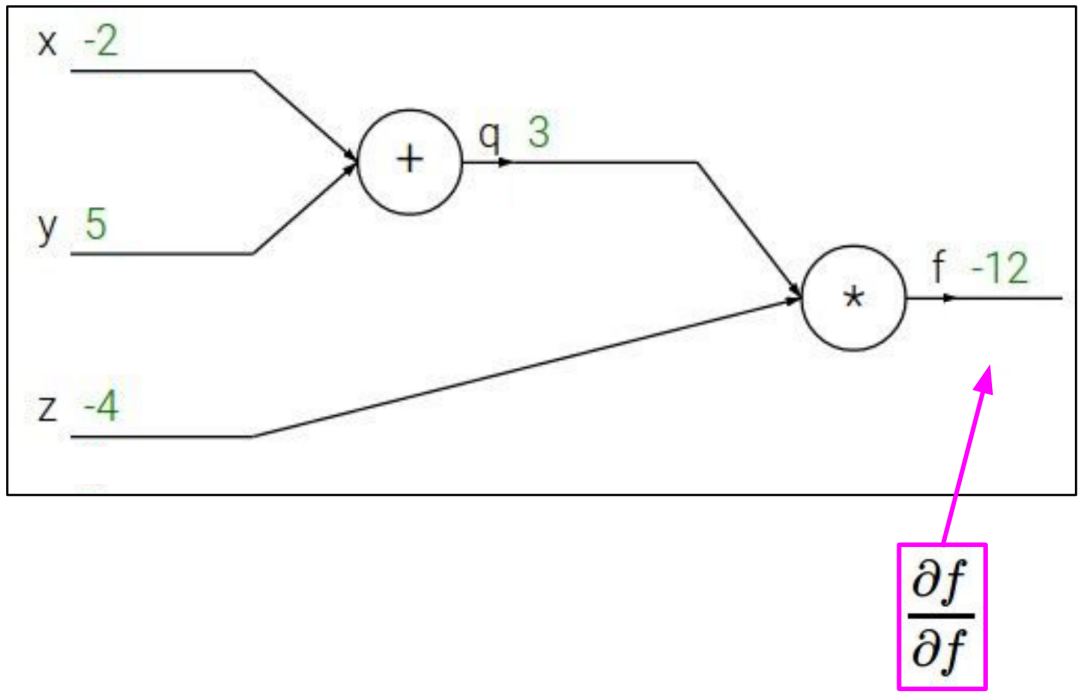
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropatation - a simple example

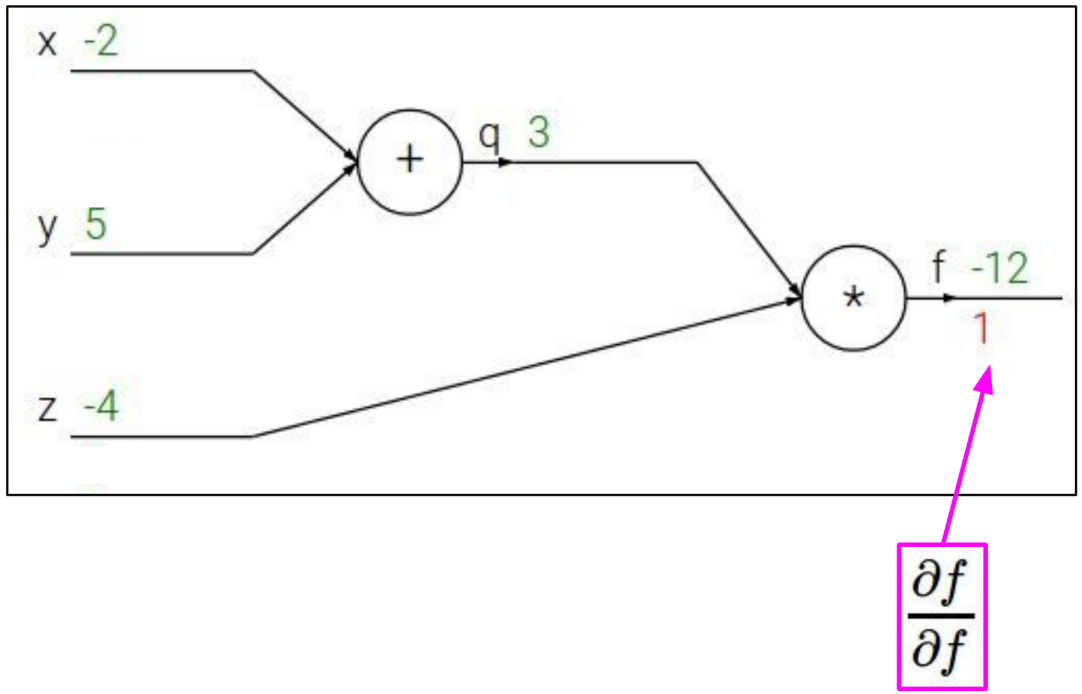
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropatation - a simple example

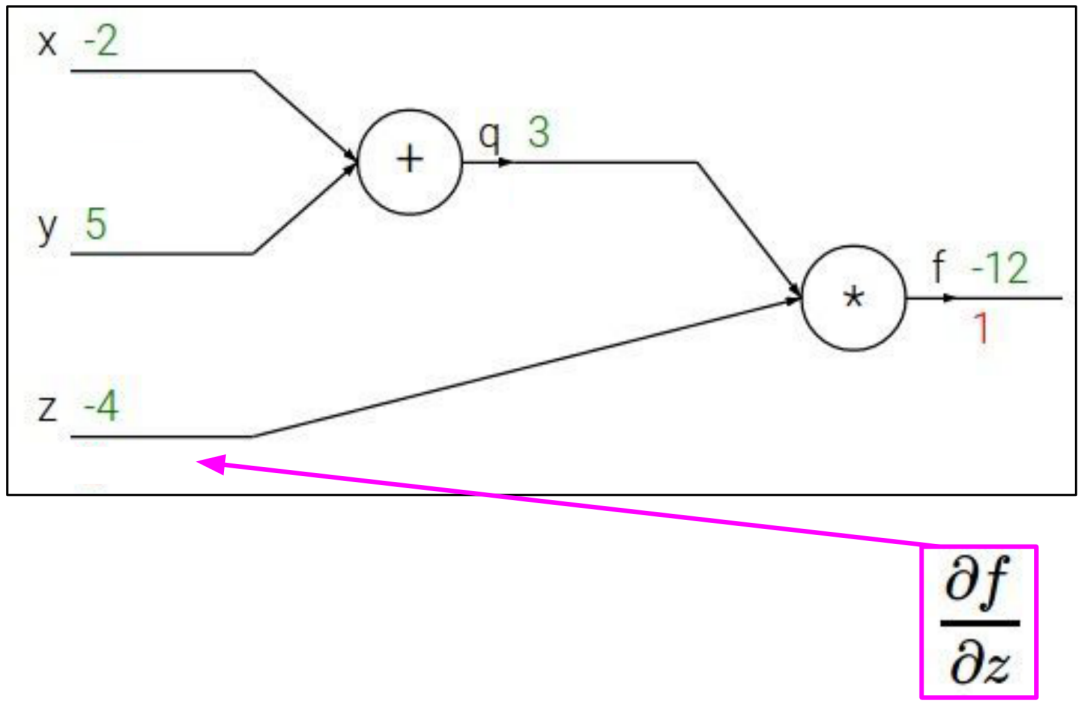
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropatation - a simple example

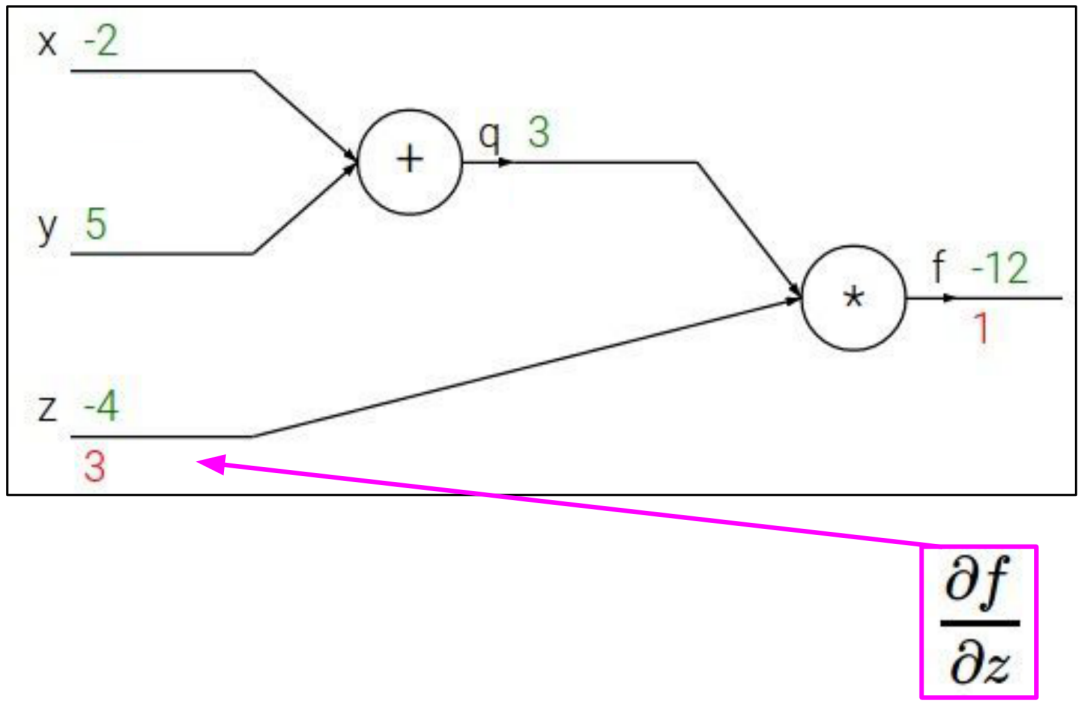
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropatation - a simple example

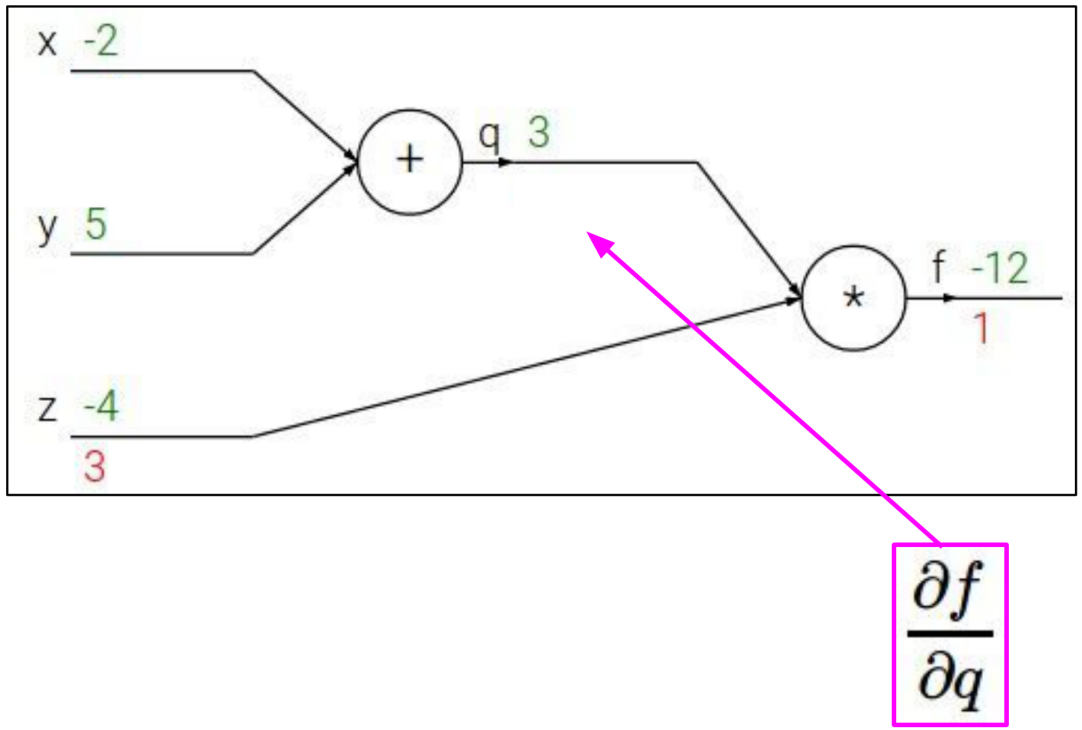
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropatation - a simple example

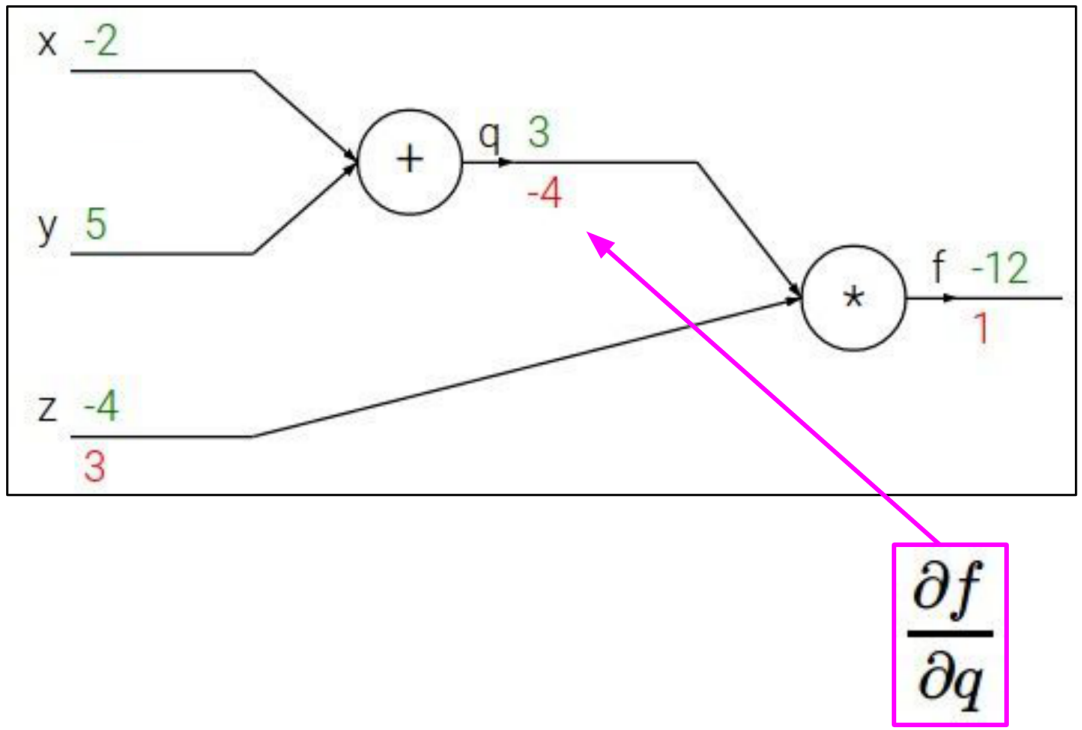
$f(x, y, z) = (x + y)z$

e.g. $x = -2, y = 5, z = -4$

$q = x + y$
$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

$f = qz$
$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropatation - a simple example

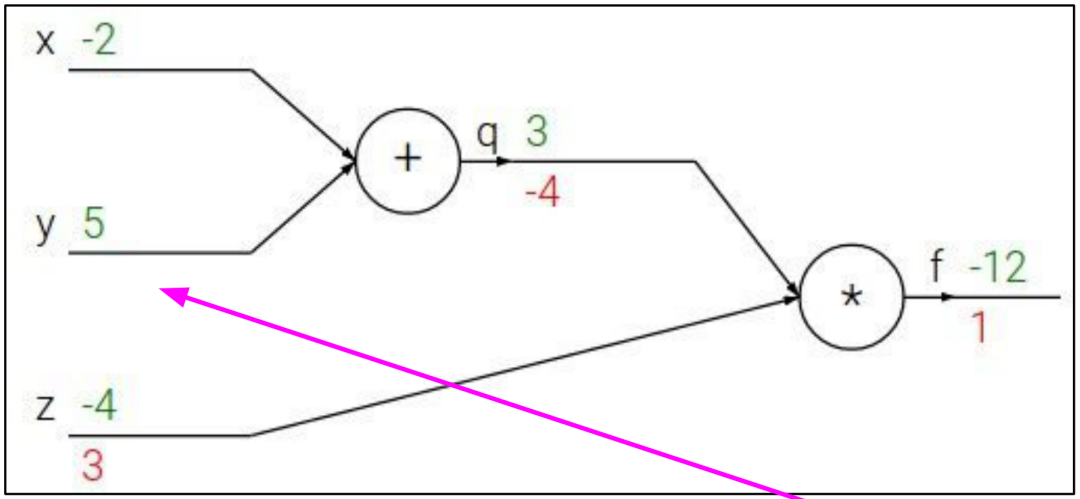
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropatation - a simple example

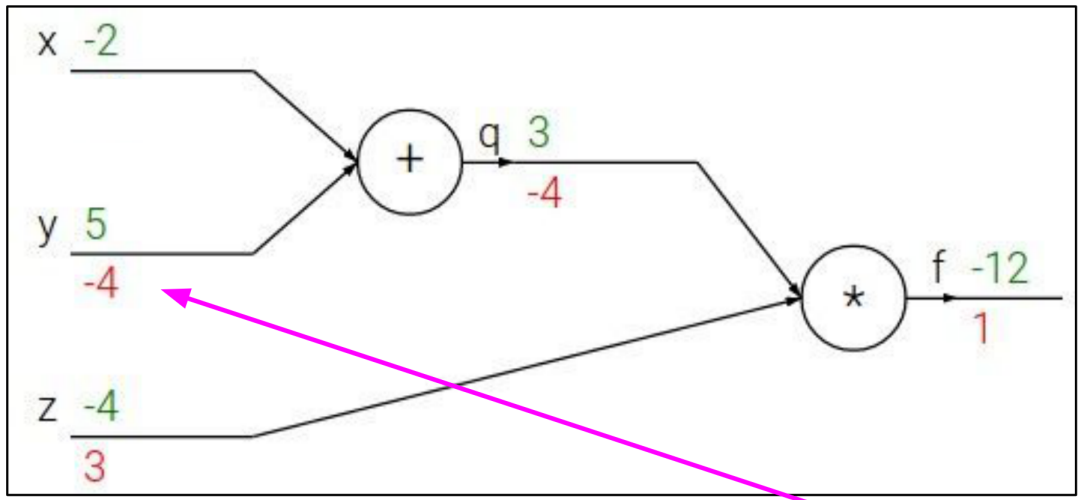
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y$$
$$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz$$
$$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropatation - a simple example

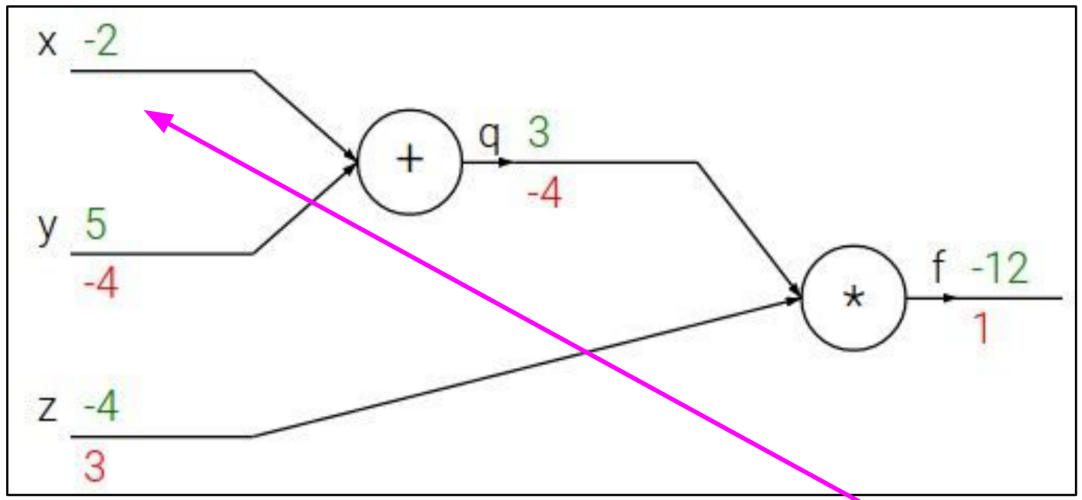
$f(x, y, z) = (x + y)z$

e.g. $x = -2, y = 5, z = -4$

$q = x + y$
$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

$f = qz$
$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$\frac{\partial f}{\partial x}$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

Backpropatation - a simple example

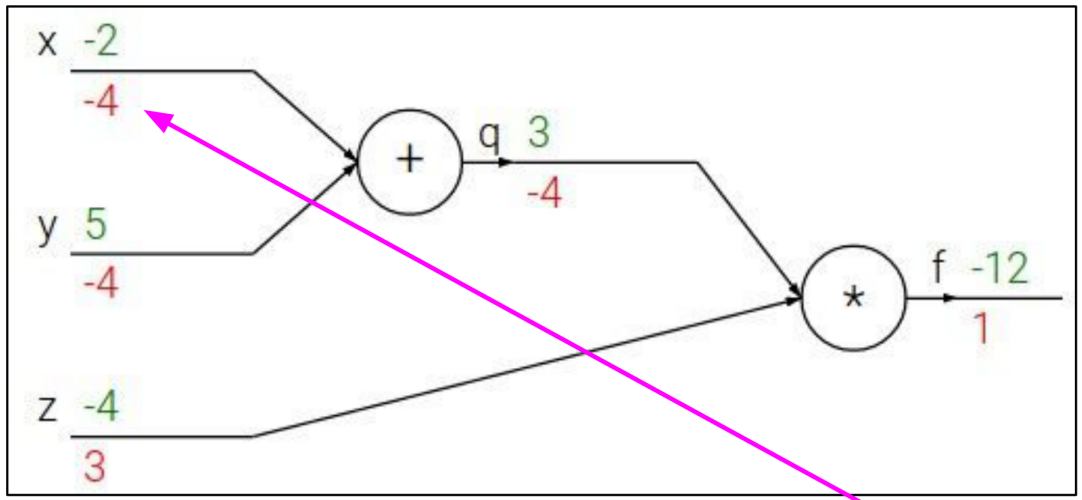
$f(x, y, z) = (x + y)z$

e.g. $x = -2, y = 5, z = -4$

$q = x + y$
$\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

$f = qz$
$\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$\frac{\partial f}{\partial x}$

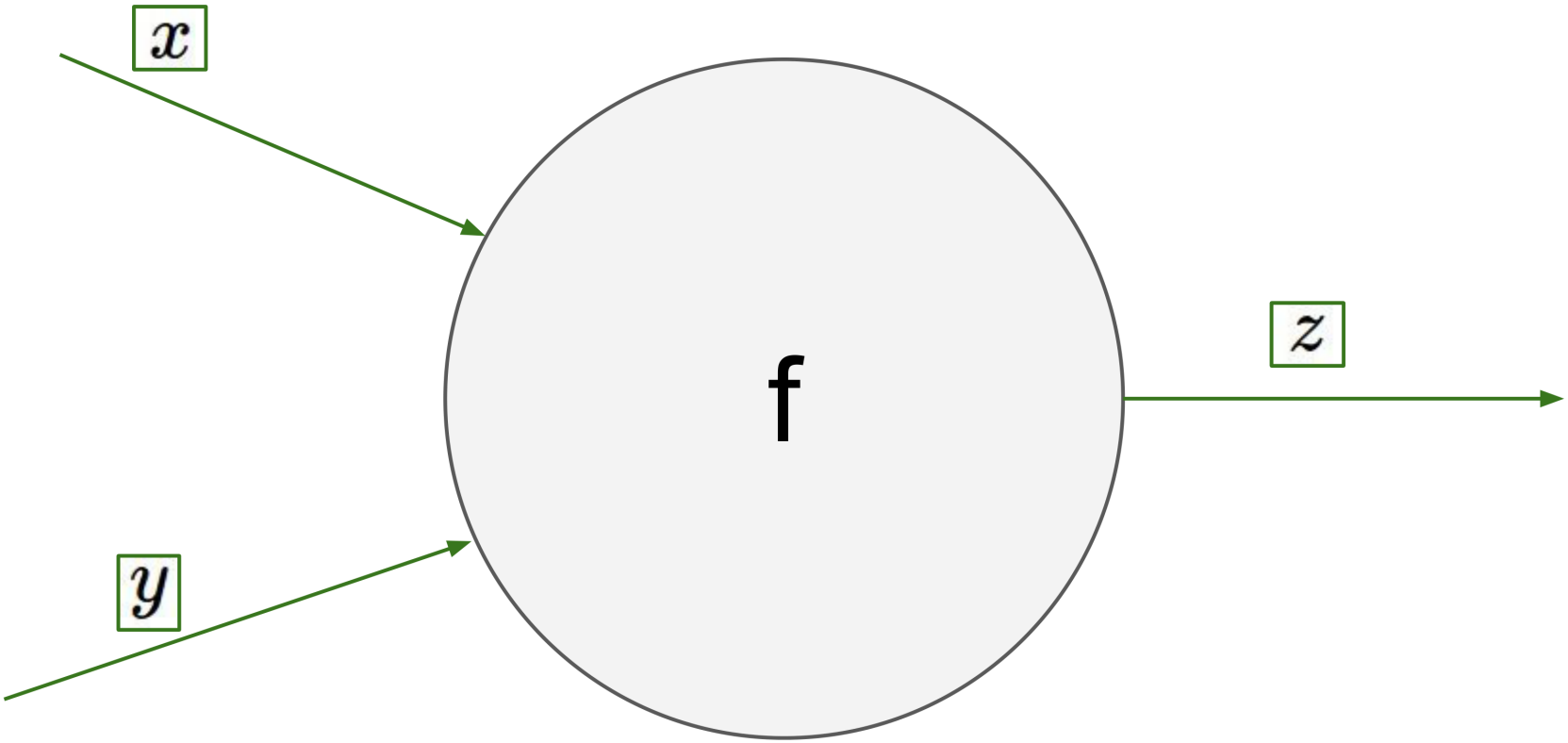
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

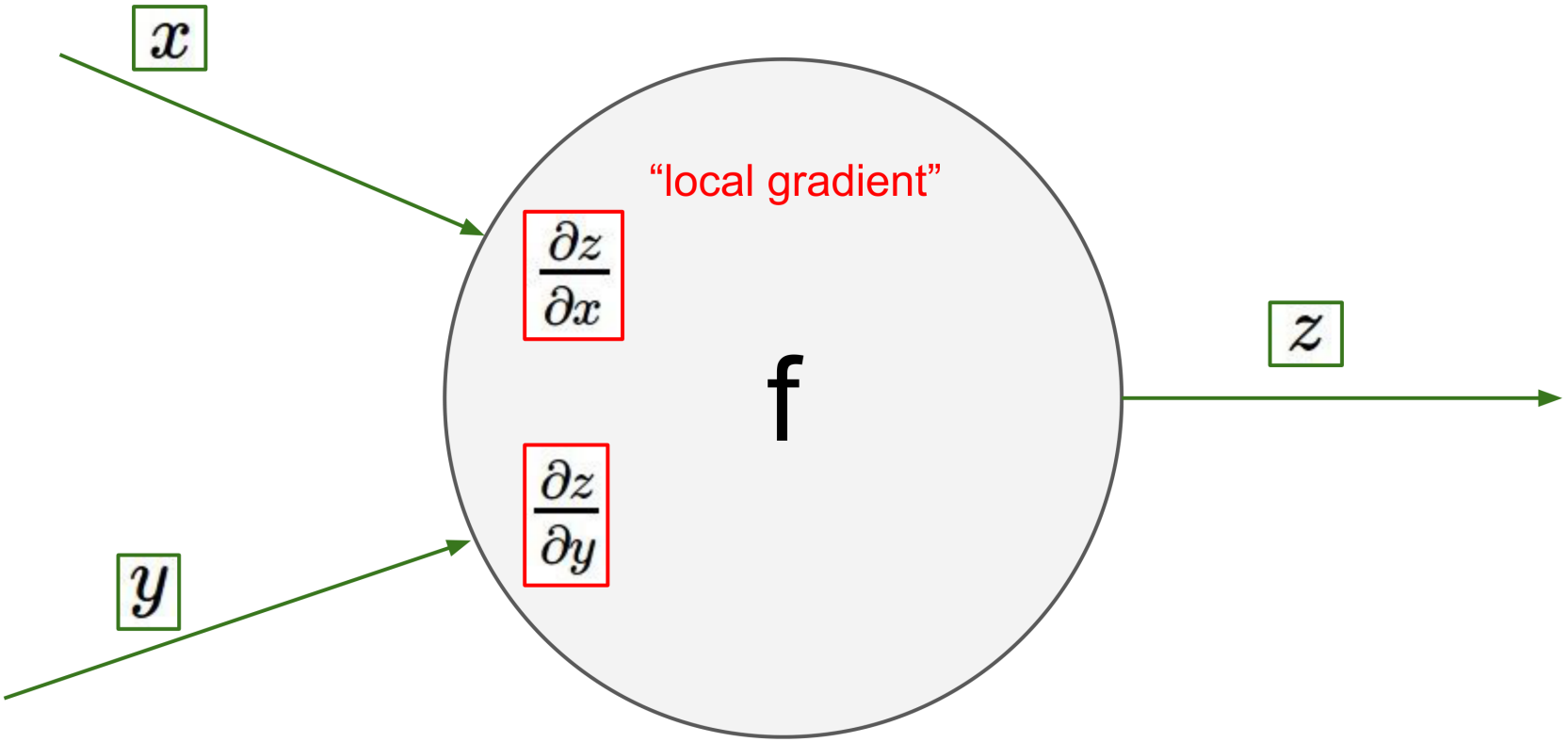
Upstream gradient

Local gradient

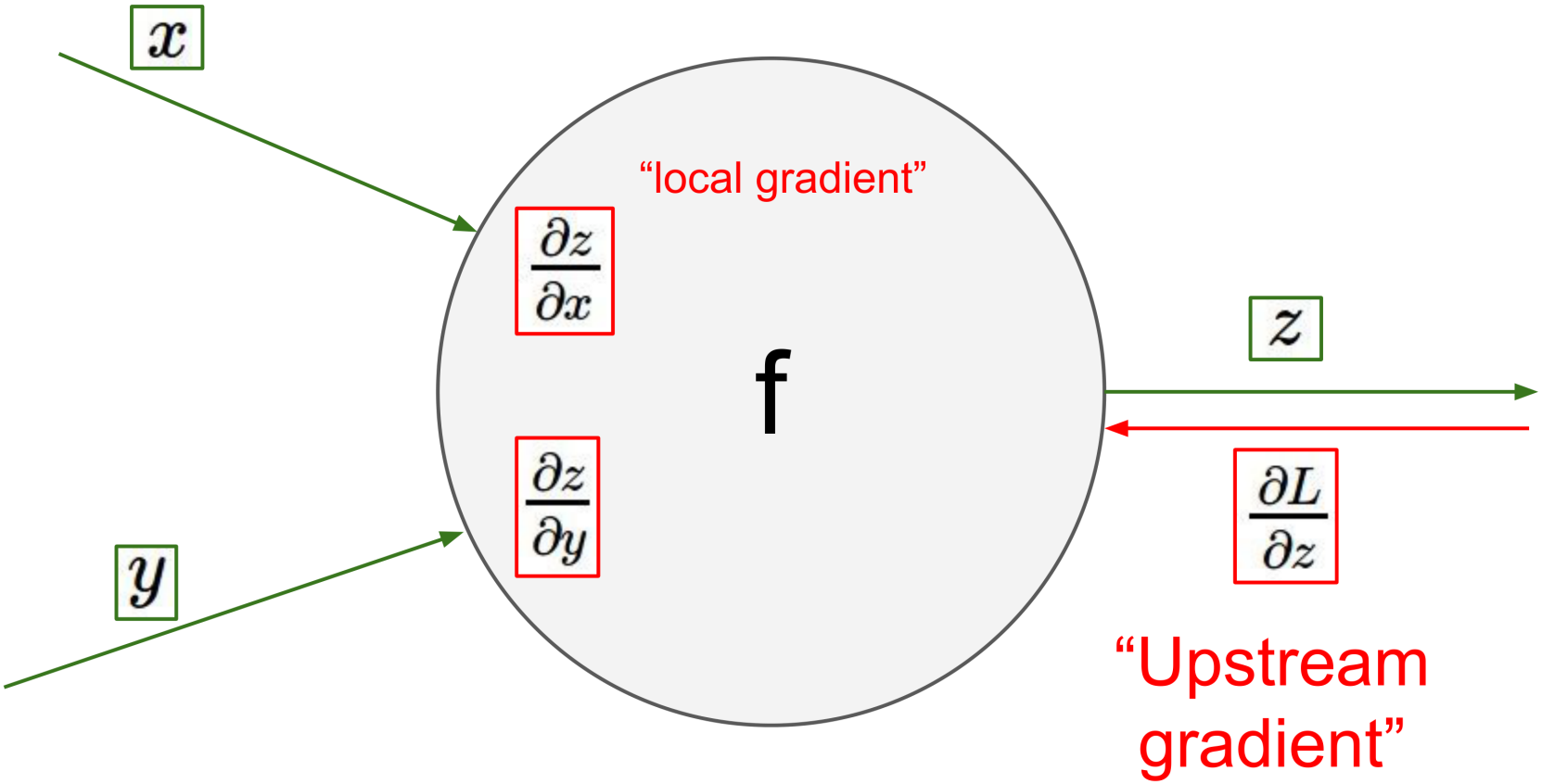
Gradient computing



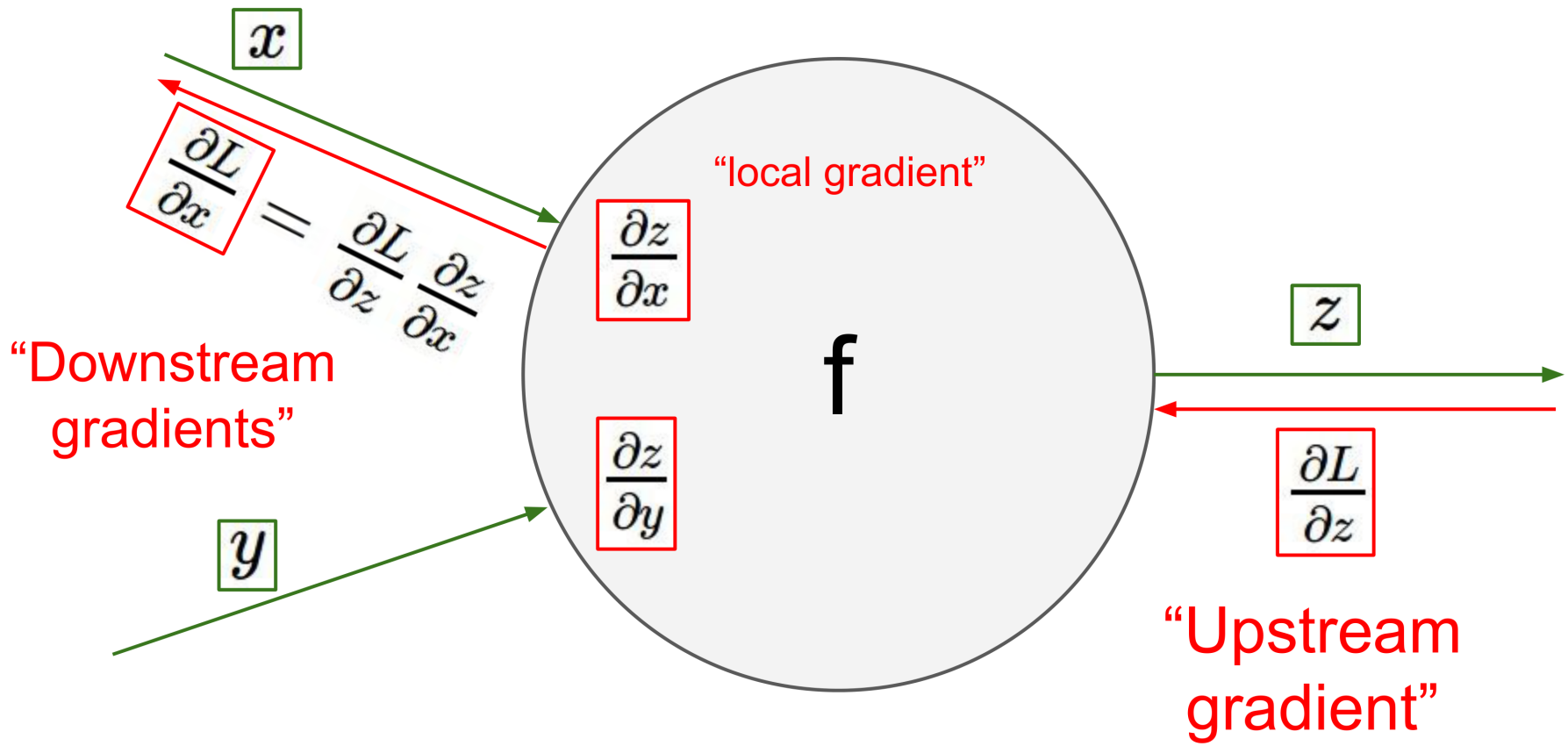
Gradient computing



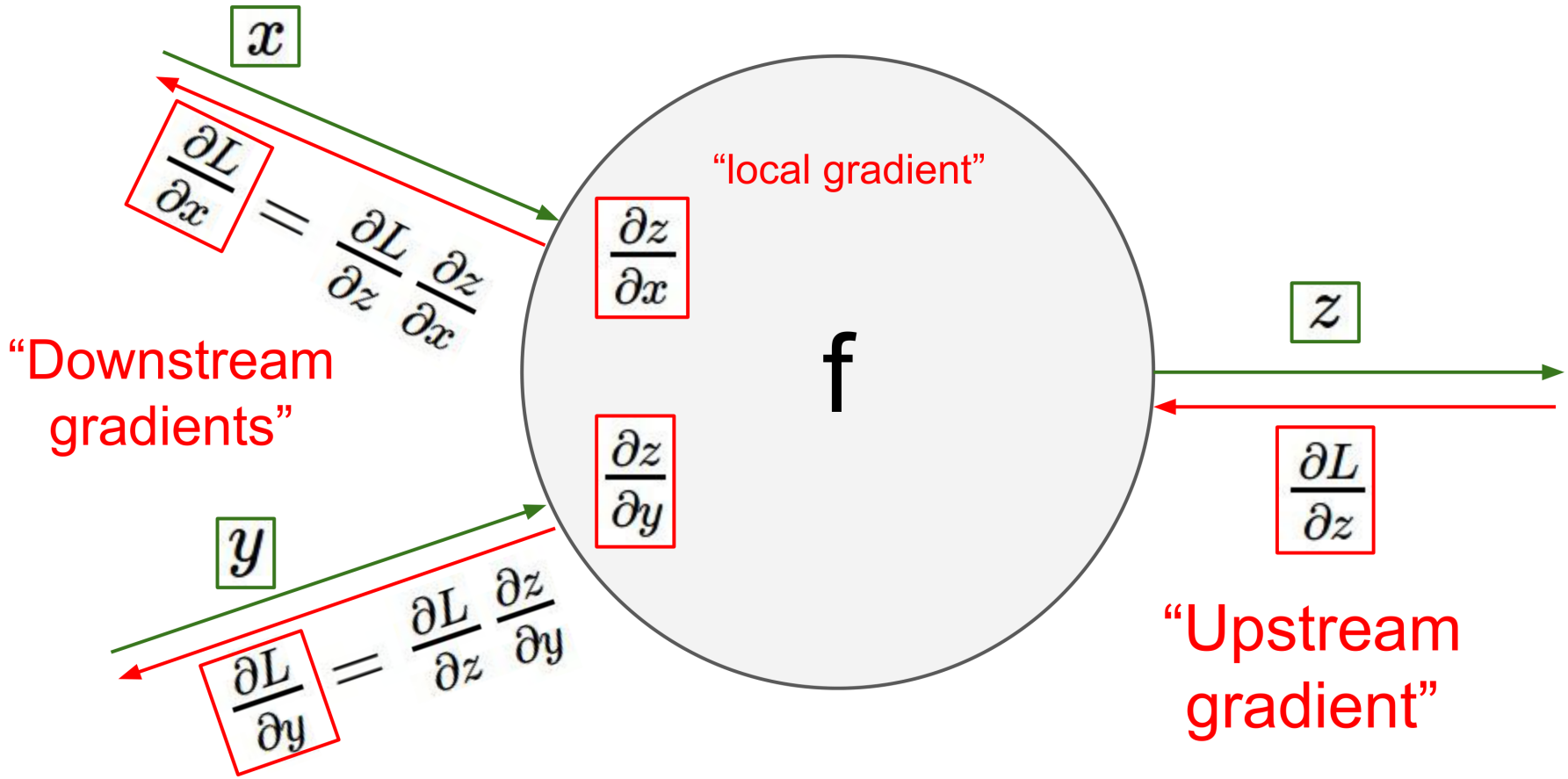
Gradient computing



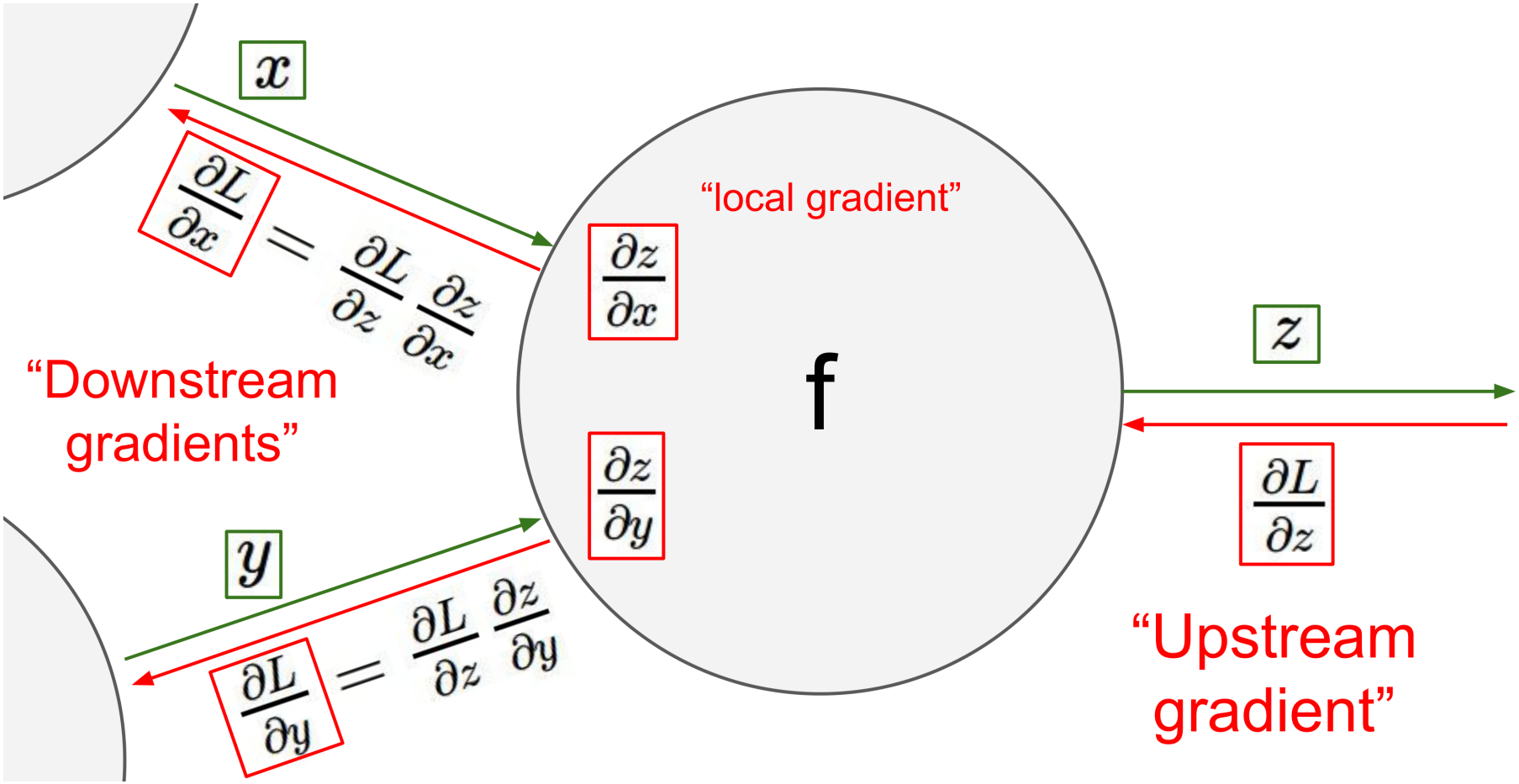
Gradient computing



Gradient computing



Gradient computing



Part 3. Network training and validation

Softmax function

- **Definition**: the softmax function is often used as the last activation function of a neural network to normalize the output of a network η to a probability distribution ϕ over predicted output classes.
- Its mathematical formulation is as following:

$$\phi_i = \frac{e^{\eta_i}}{\sum_{i=1}^{k} e^{\eta_i}}$$

- Here, we will derive this mathematical formula step by step.

Softmax function

- Let's consider a multi-class classification problem in which the response variable y can take on any one of k values, so $y \in \{1, 2, \ldots, k\}$.
- To parameterize a multinomial over k possible outcomes, we could use k parameters $\phi_1, \phi_2, \ldots, \phi_k$ specifying the probability of each of the outcomes.
- However, they would not be independent since knowing any $k-1$ of the ϕ_i's uniquely determines the last one, satisfying $\sum_{i=1}^{k} \phi_i = 1$.
- So we will instead parameterize the multinomial with only $k-1$ parameters, $\phi_1, \phi_2, \ldots, \phi_{k-1}$, where $\phi_i = p(y = i; \phi)$, and $p(y = k; \phi) = 1 - \sum_{i=1}^{k-1} \phi_i$. For notational convenience, let $\phi_k = 1 - \sum_{i=1}^{k-1} \phi_i$.

Softmax function

- Here, we introduce an indicator function $1\{\cdot\}$ takes on a value of 1 if its argument is true, and 0 otherwise. For example, $1\{2 = 3\} = 0$, and $1\{3 = (5 - 2)\} = 1$.
- We would like to define $T(y) \in \mathbb{R}^{k-1}$ as follows:

$$T(1) = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, T(2) = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \cdots, T(k-1) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}, T(k) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

  We will write $(T(y))_i$ to denote the i-th element of the vector $T(y)$.
- So, we can also write the relationship between $T(y)$ and y as $(T(y))_i = 1\{y = i\}$. Further, we have that $E\left[(T(y))_i\right] = P(y = i) = \phi_i$.

$$E\left[(T(y))_i\right] = 1 \times P(y = i) + 0 \times P(y \neq i) = \phi_i$$

Softmax function

- Now, we have:

$$
\begin{aligned}
p(y;\phi) &= \phi_1^{1\{y=1\}}\phi_2^{1\{y=2\}}\cdots\phi_k^{1\{y=k\}} \\
&= \phi_1^{1\{y=1\}}\phi_2^{1\{y=2\}}\cdots\phi_k^{1-\sum_{i=1}^{k-1}1\{y=i\}} \\
&= \phi_1^{(T(y))_1}\phi_2^{(T(y))_2}\cdots\phi_k^{1-\sum_{i=1}^{k-1}(T(y))_i} \\
&= e^{(T(y))_1\log\phi_1+(T(y))_2\log\phi_2+\cdots+(1-\sum_{i=1}^{k-1}(T(y))_i)\log\phi_k} \\
&= e^{(T(y))_1\log\frac{\phi_1}{\phi_k}+(T(y))_2\log\frac{\phi_2}{\phi_k}+\cdots+(T(y))_{k-1}\log\frac{\phi_{k-1}}{\phi_k}+\log\phi_k} \\
&= e^{\eta^T T(y)+a(\eta)},
\end{aligned}
$$

where $\eta = \begin{bmatrix} \log\frac{\phi_1}{\phi_k} \\ \log\frac{\phi_2}{\phi_k} \\ \vdots \\ \log\frac{\phi_{k-1}}{\phi_k} \end{bmatrix}$, $a(\eta) = \log\phi_k$

Softmax function

- For $i = 1, 2, \ldots, k$, $\eta_i = \log \frac{\phi_i}{\phi_k}$. So, we can get the following:

  $$e^{\eta_i} = \frac{\phi_i}{\phi_k}$$

  $$\phi_k e^{\eta_i} = \phi_i \qquad (1)$$

  Also:

  $$\phi_k \sum_{i=1}^{k} e^{\eta_i} = \sum_{i=1}^{k} \phi_i = 1$$

  $$\phi_k = \frac{1}{\sum_{i=1}^{k} e^{\eta_i}} \qquad (2)$$

- We substitute equation (2) back into equation (1) and get:

  $$\phi_i = \frac{e^{\eta_i}}{\sum_{i=1}^{k} e^{\eta_i}} \qquad (3)$$

- This function (3) mapping from η to ϕ is called the **softmax** function.

Loss function

- Definition
  - Q: What is loss function?
  - A: A loss function[2] is a function that maps an event or values of one or more variables onto a real number intuitively representing some "cost" associated with the event.
- General formulation
  - Given an input dataset $\mathbf{X} = \{x_1, x_2, \ldots, x_n\}$ with the corresponding label $\mathbf{Y} = \{y_1, y_2, \ldots, y_n\}$.
  - Built a neural network with parameters Θ.
  - Got prediction $\hat{\mathbf{Y}} = \{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n\}$, where $\hat{y}_i = f(x_i, \Theta)$.
  - Total loss: $\mathcal{L} = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i)$.

[2] https://en.wikipedia.org/wiki/Loss_function

Mean square error

- Mean squre error (MSE) is the average squared difference between the estimated values and the actual value.
- MSE can be expressed by

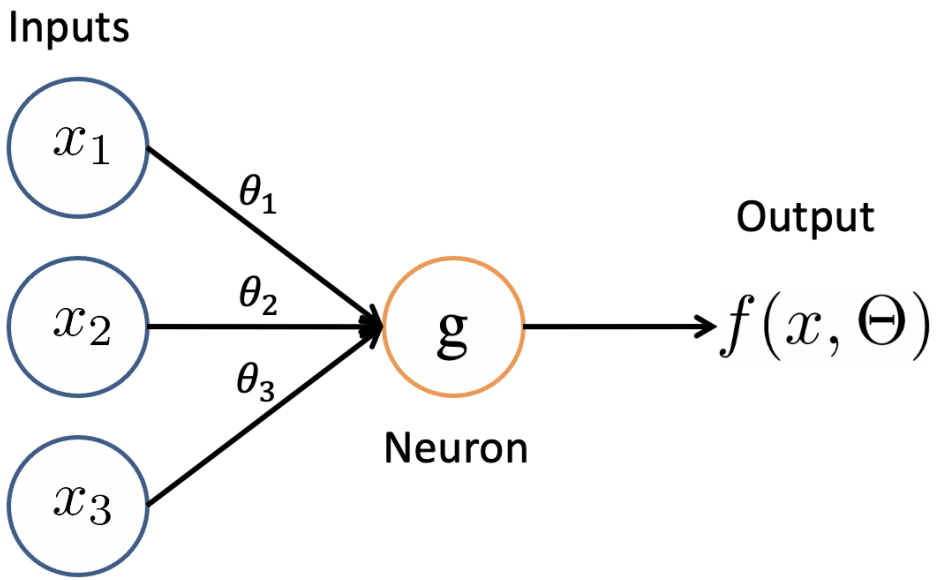
$$\ell_{MSE} = (y_i - \hat{y}_i)^2 = (y_i - f(x_i, \Theta))^2$$

- Further, we can compute the gradient of MSE by

$$\frac{\partial \ell_{MSE}}{\partial \Theta} = -2(y_i - f(x_i, \Theta)) \frac{\partial f(x_i, \Theta)}{\partial \Theta}$$

Mean square error

- To help better understand, let us go through a simple example with $f(x, \Theta) = g(x\Theta)$, where $x = [x_1, x_2, x_3]$, $\Theta = [\theta_1, \theta_2, \theta_3]^T$ and g is a relu activation function $g(z) = \max(0, z)$.



Mean square error

- So, the gradient of MSE can be computed by

$$\begin{aligned}\frac{\partial \ell_{MSE}}{\partial \theta_1} &= -2(y - f(x, \Theta))\frac{\partial f(x, \Theta)}{\partial \theta_1} \\ &= -2(y - \max(0, \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3))\frac{\partial \max(0, \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3)}{\partial \theta_1} \\ &= \begin{cases} -2x_1(y - \theta_1 x_1 - \theta_2 x_2 - \theta_3 x_3), & \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \geq 0 \\ 0, & otherwise \end{cases}\end{aligned}$$

- $\frac{\partial \ell_{MSE}}{\partial \theta_2}$ and $\frac{\partial \ell_{MSE}}{\partial \theta_3}$ can be computed in a similar way.

Cross entropy error

- Cross entropy error (CE) is the cross-entropy between two probability distributions over the same underlying set of events.
- CE can be expressed by

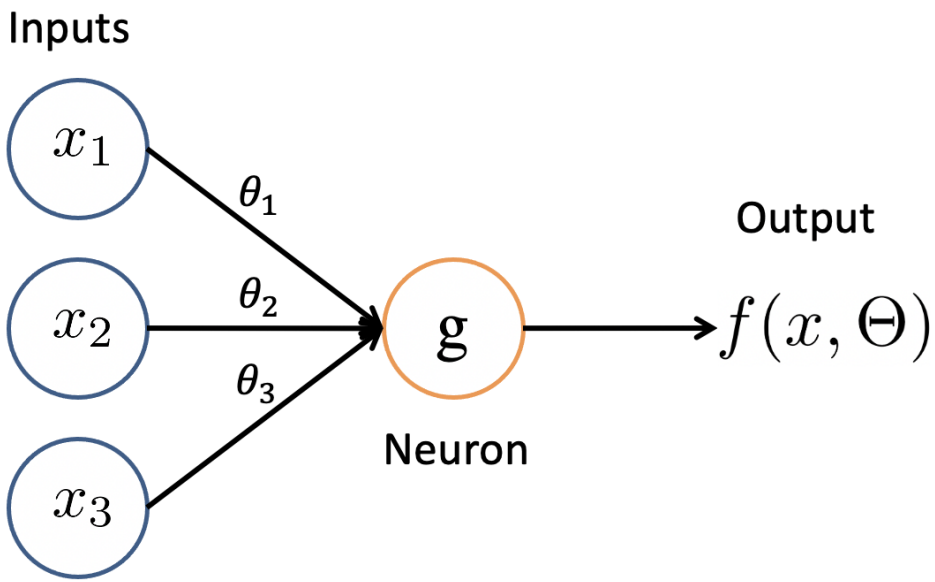
$$\ell_{CE} = -y_i \log(\hat{y}_i) = -y_i \log(f(x_i, \Theta))$$

- Further, we can compute the gradient of CE by

$$\frac{\partial \ell_{CE}}{\partial \Theta} = -\frac{y_i}{f(x_i,\Theta)} \frac{\partial f(x_i,\Theta)}{\partial \Theta}$$

Cross entropy error

- Similarly, for better understanding, let us go through a simple example with $f(x, \Theta) = g(x\Theta)$, where $x = [x_1, x_2, x_3]$, $\Theta = [\theta_1, \theta_2, \theta_3]^T$ and g is a relu activation function $g(z) = \max(0, z)$.



Cross entropy error

- So, the gradient of CE can be computed by

$$\begin{aligned}\frac{\partial \ell_{CE}}{\partial \theta_1} &= -\frac{y}{f(x,\Theta)}\frac{\partial f(x,\Theta)}{\partial \theta_1} \\ &= -\frac{y}{\max(0,\theta_1 x_1+\theta_2 x_2+\theta_3 x_3)}\frac{\partial \max(0,\theta_1 x_1+\theta_2 x_2+\theta_3 x_3)}{\partial \theta_1} \\ &= \begin{cases} -\frac{x_1 y}{\theta_1 x_1+\theta_2 x_2+\theta_3 x_3}, & \theta_1 x_1+\theta_2 x_2+\theta_3 x_3 > 0 \\ 0, & otherwise \end{cases}\end{aligned}$$

- $\frac{\partial \ell_{CE}}{\partial \theta_2}$ and $\frac{\partial \ell_{CE}}{\partial \theta_3}$ can be computed in a similar way.

Learning rate

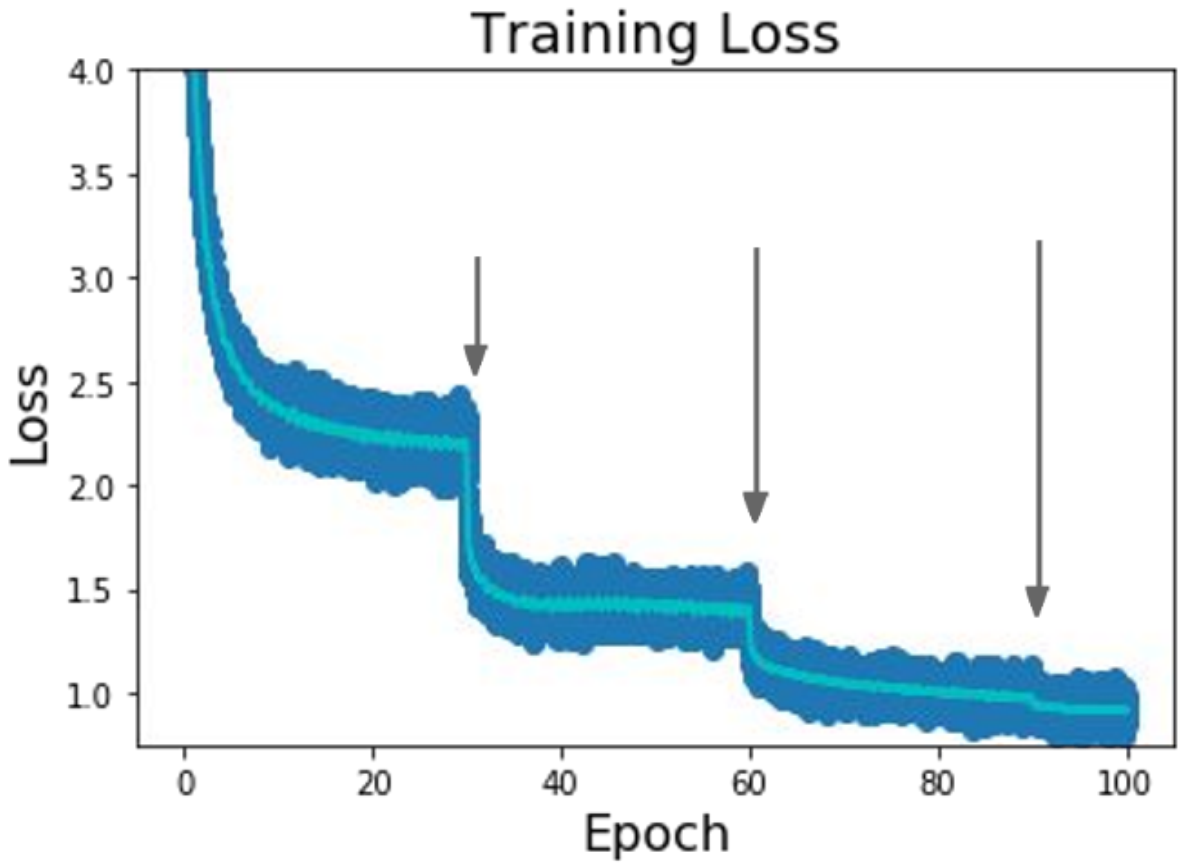
- In an optimization problem, we perform gradient descent with specific step size at every iteration as following:

$$\Theta = \Theta - \alpha \frac{\partial \ell}{\partial \Theta}$$

  Here, the step size α is called the **learning rate**.
- **Definition**: the learning rate is a tuning parameter in an optimization algorithm that determines the step size at each iteration while moving toward a **minimum** of a loss function.

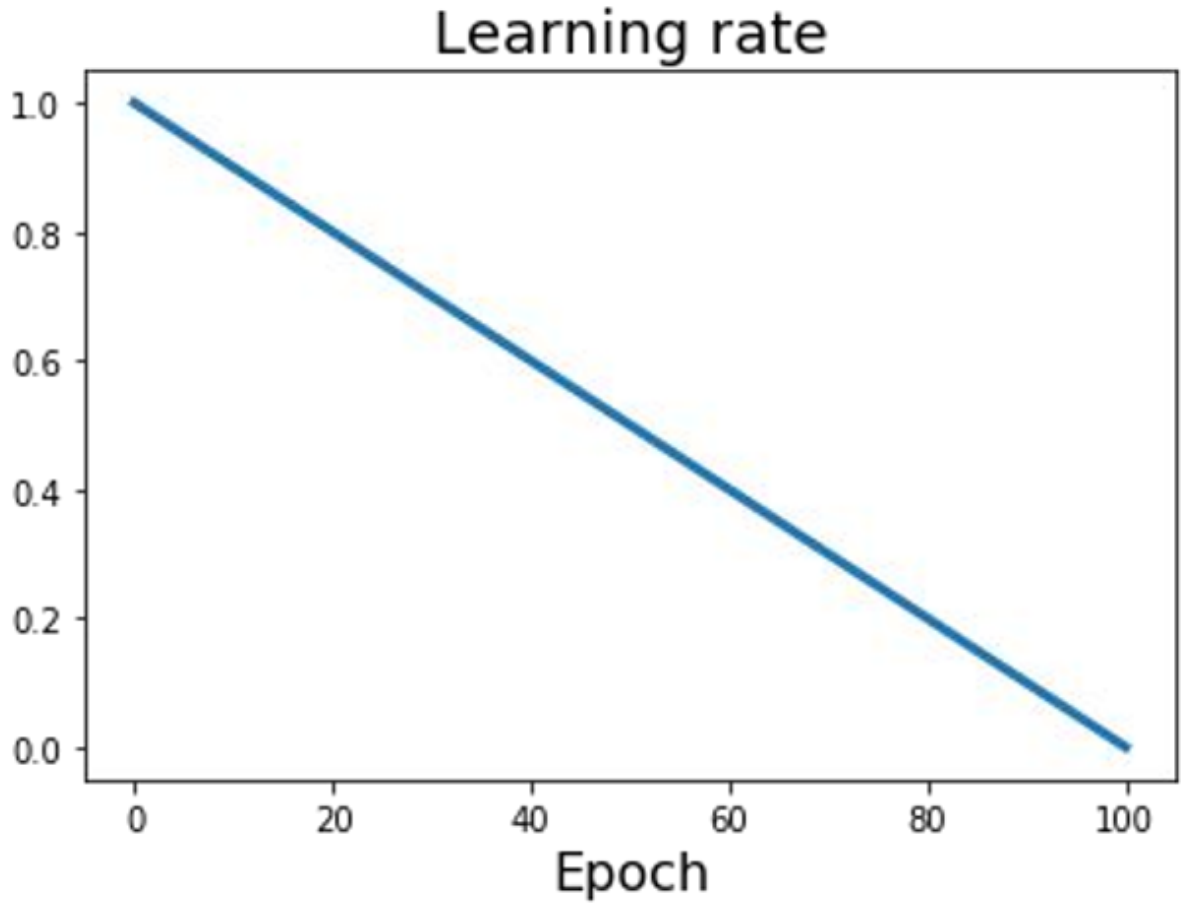
Learning rate decay

- Why should learning rate decay?
  - Reduce learning rate at a few fixed points, e.g. multiplying learning rate by 0.1 after epochs 30, 60, and 90.
  - Training loss significantly decreases after learning rate decay.



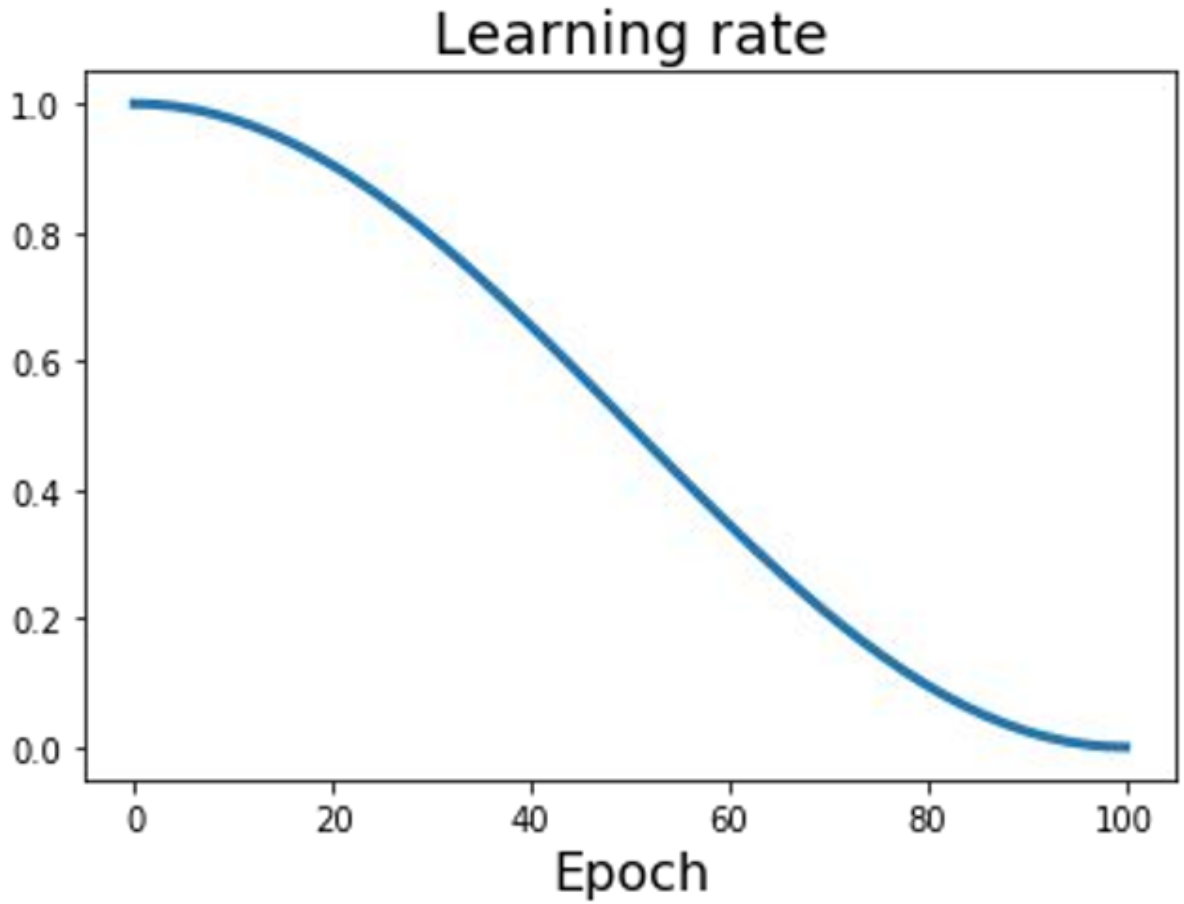
Learning rate decay

- Linear decay
  - $\alpha_t = \alpha_0(1 - t/T)$, where α_0 is the initial learning rate, α_t is the learning rate at epoch t, and T is total number of epochs.



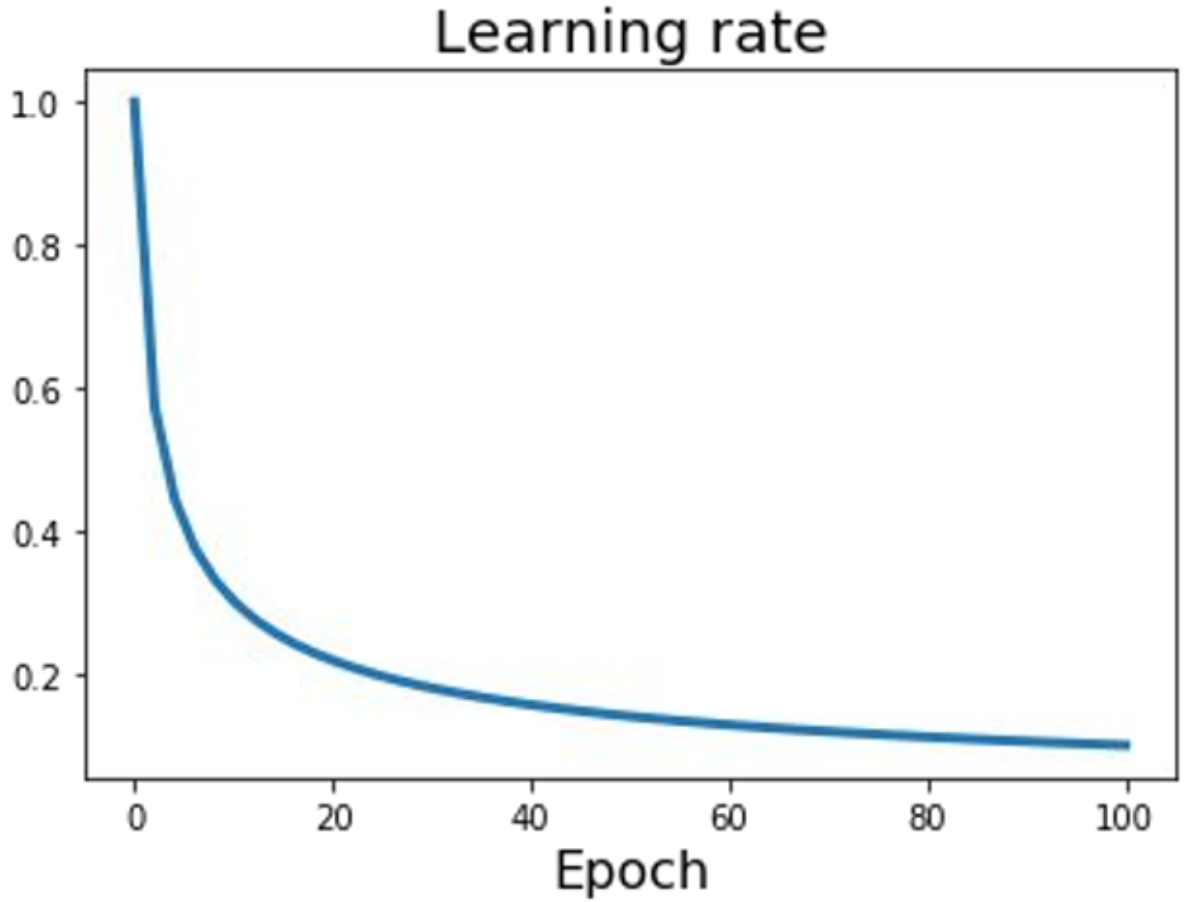
Learning rate decay

- Cosine decay
  - $\alpha_t = \frac{1}{2}\alpha_0(1 + \cos t\pi/T)$, where α_0 is the initial learning rate, α_t is the learning rate at epoch t, and T is total number of epochs.



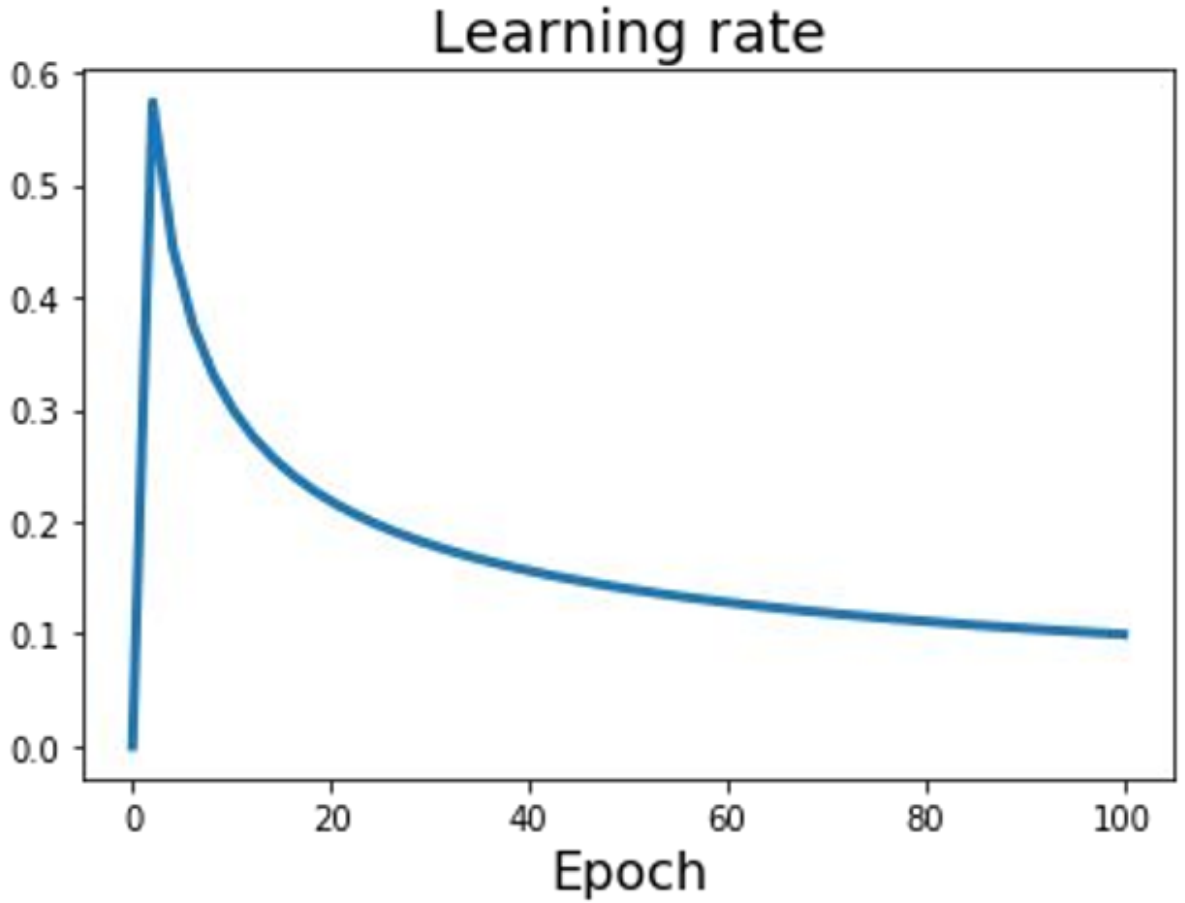
Learning rate decay

- Inverse sqrt decay
  - $\alpha_t = \alpha_0/\sqrt{t}$, where α_0 is the initial learning rate, α_t is the learning rate at epoch t, and T is total number of epochs.



Learning rate decay

- Warmup strategy
  - High initial learning rate can make loss explode, linearly increasing learning rate from 0 over the first 5000 iterations can prevent this.



Part 4. Summary

Summary

- Neural network architecture
  - Biological inspiration
  - Activation function
  - Feed forward neural network
  - How network model function
- Backpropagation
  - Gradient computing
- Network training and validation
  - Loss function
  - Learning rate decay
  - Softmax

Reading materials

- Stanford CS231n: Convolutional Neural Networks for Visual Recognition http://cs231n.stanford.edu/
- Stanford CS229: Machine Learning http://cs229.stanford.edu/

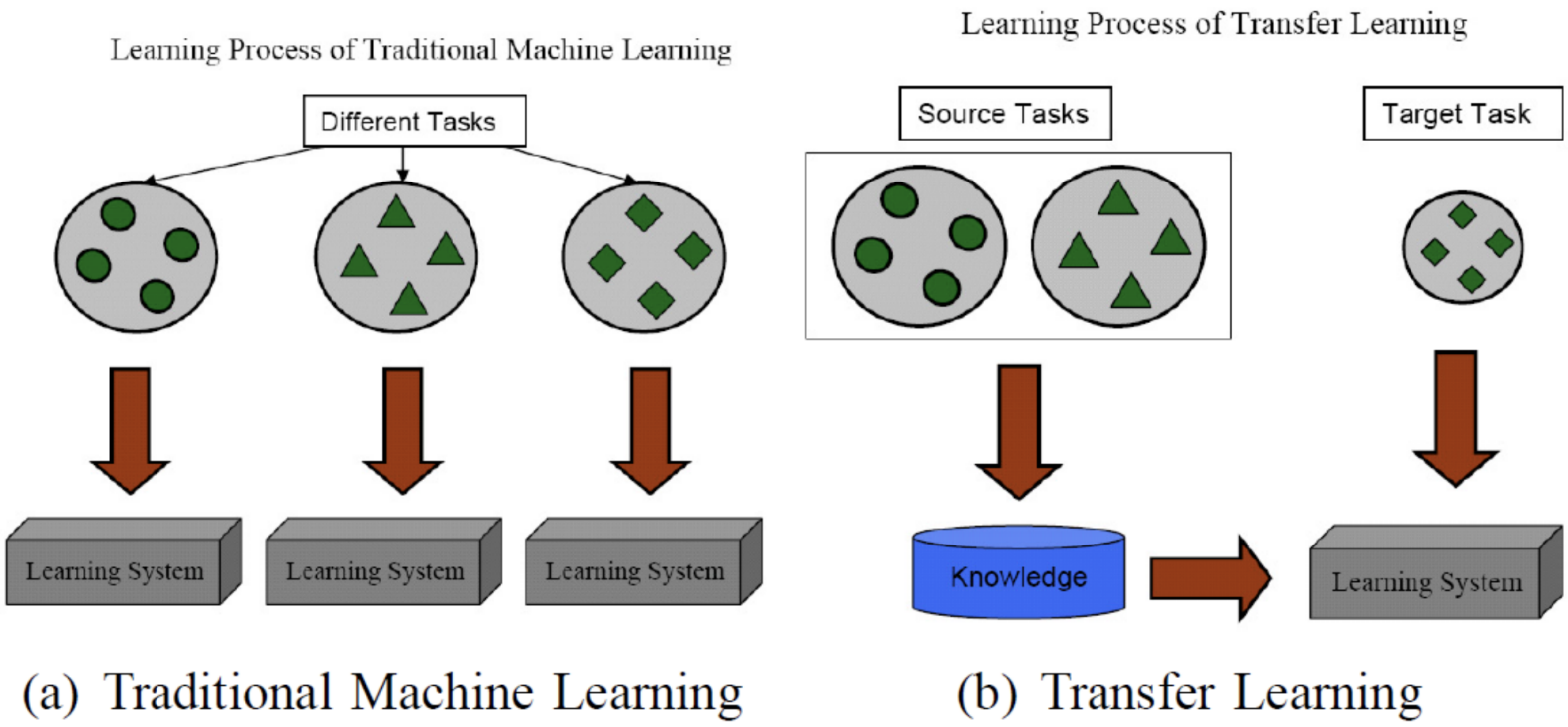
ESTR3108

Outline

- Transfer learning

Motivation for transfer learning

- Training every model from scratch is time-consuming and expensive.
- But, there are extensive existing knowledge. Can we reuse them?



(a) Traditional Machine Learning (b) Transfer Learning

Transfer learning

- The origin of transfer learning
  - Thorndike and Woodworth in 1901:
    how individuals **transfer** in one context to another context that share **similar** characteristics.

Transfer learning

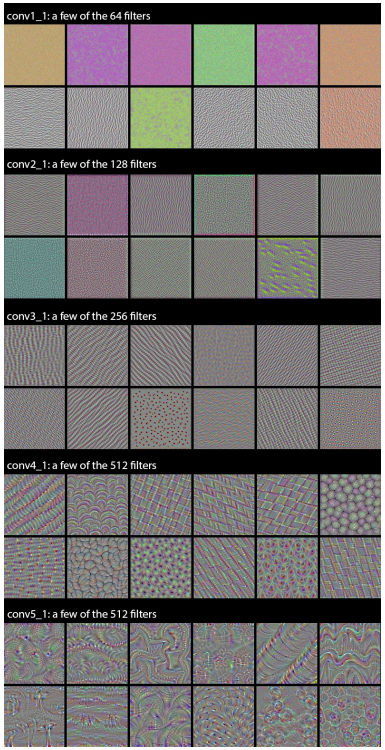
- The origin of transfer learning
  - Thorndike and Woodworth in 1901:
    how individuals **transfer** in one context to another context that share **similar** characteristics.
- Intuitive definition
  - Wikipedia: research problem in machine learning that focuses on storing knowledge gained while solving one problem and applying it to a **different** but **related** problem.

Transfer learning

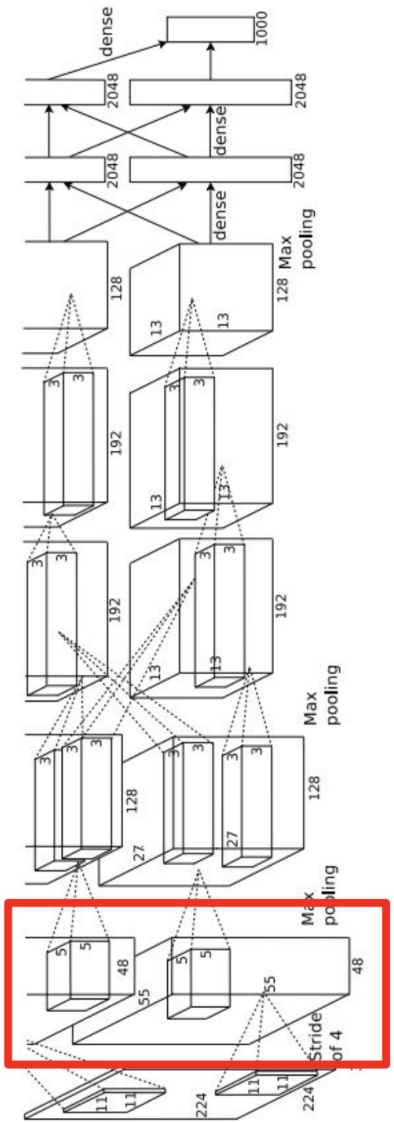
- The origin of transfer learning
  - Thorndike and Woodworth in 1901:
    how individuals **transfer** in one context to another context that share **similar** characteristics.
- Intuitive definition
  - Wikipedia: research problem in machine learning that focuses on storing knowledge gained while solving one problem and applying it to a **different** but **related** problem.
- Related fields
  - machine learning, AI, data mining, information engineering
  - applications (e.g., computer vision, natural language processing, healthcare, robotics, recommendation systems, etc.)

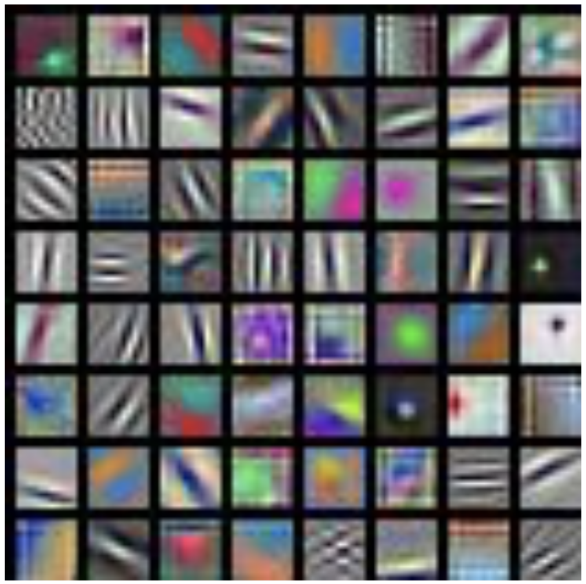
Transfer learning in CNNs

- Neural network layers: general to specific (e.g., VGG16 learned filters).
- Bottom/earlier layers: general learners ("low-level" concepts such as edges, colors, shapes)
- Top/later layers: specific learners ("high-level" features such as object parts, semantics, object categories)



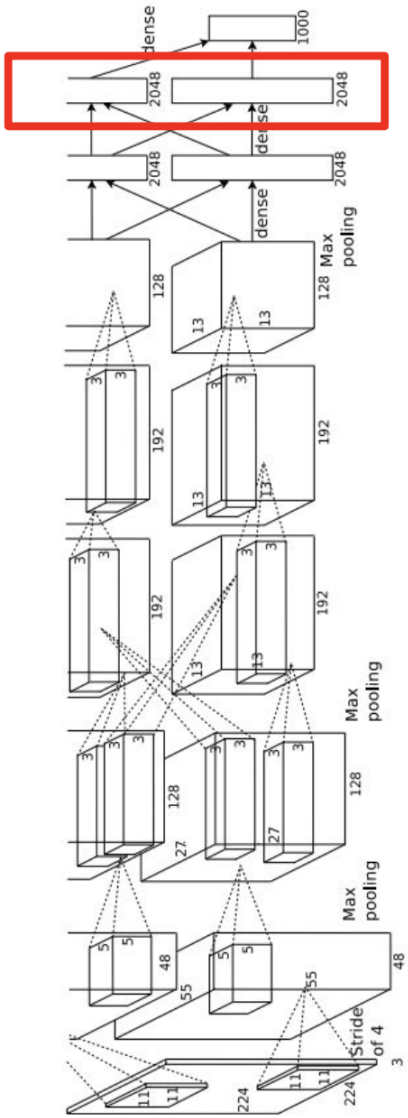
Transfer learning in CNNs

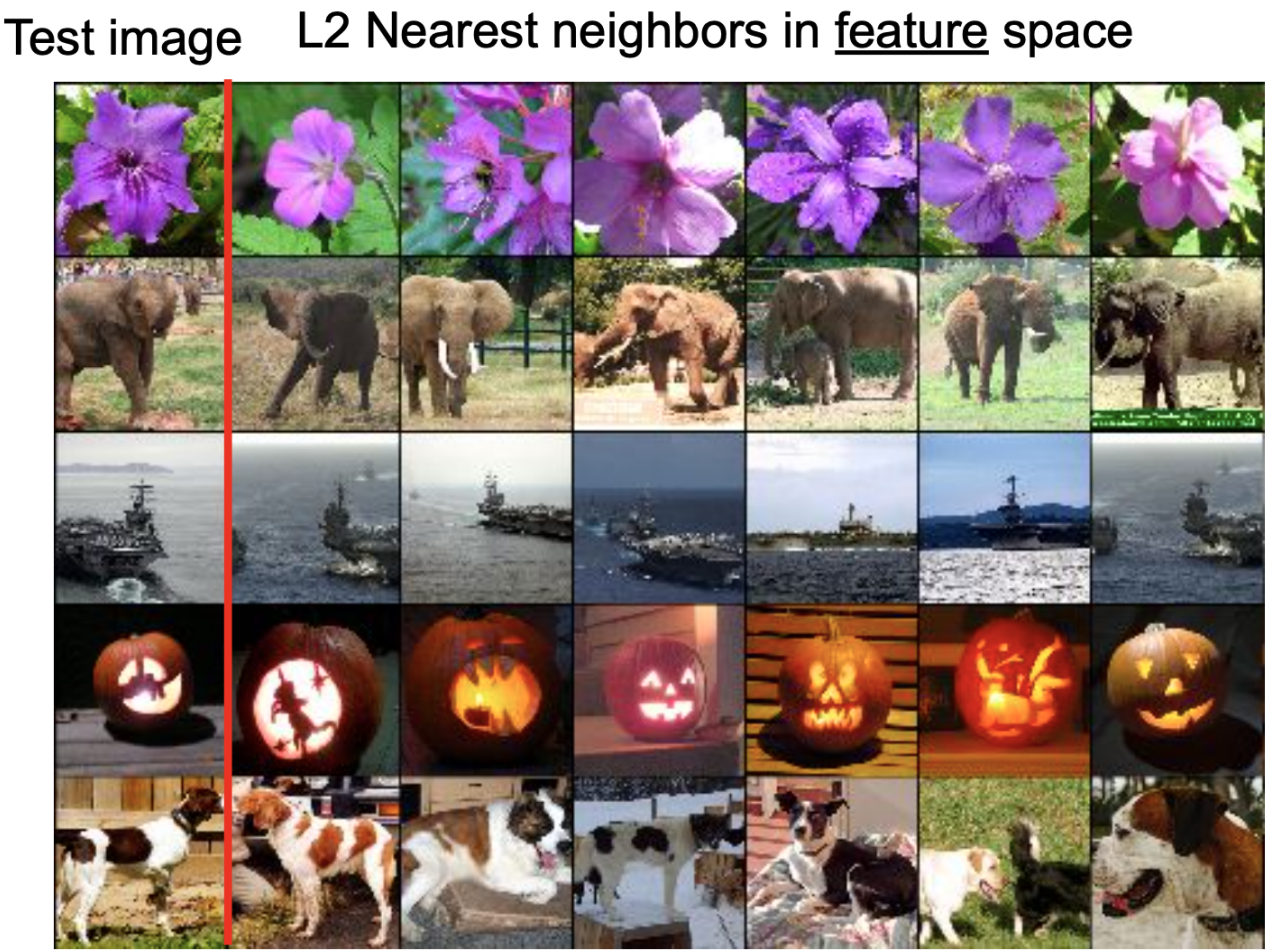




AlexNet:
64 x 3 x 11 x 11

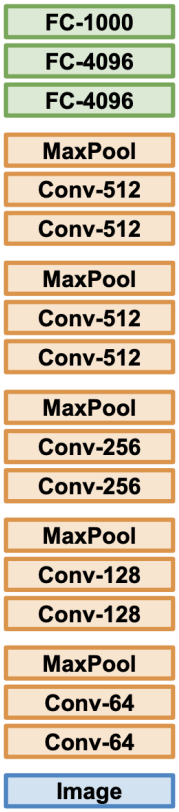
Transfer learning in CNNs



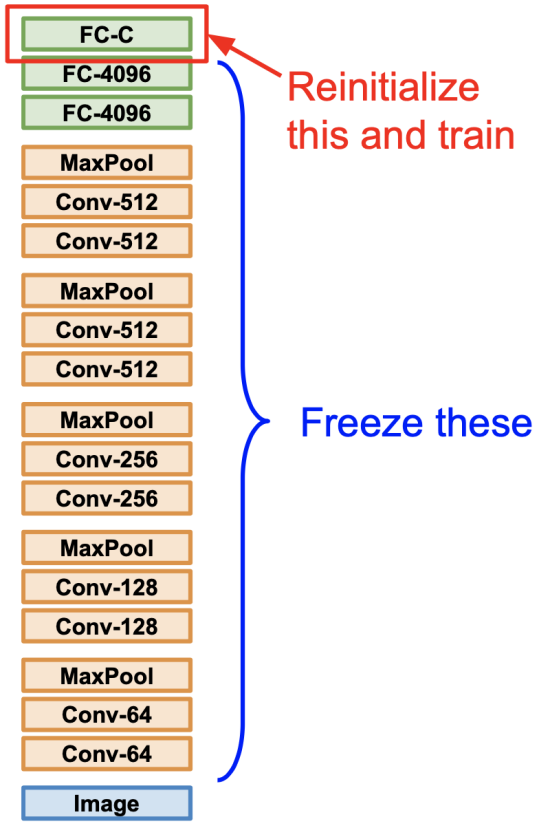


Transfer learning in CNNs

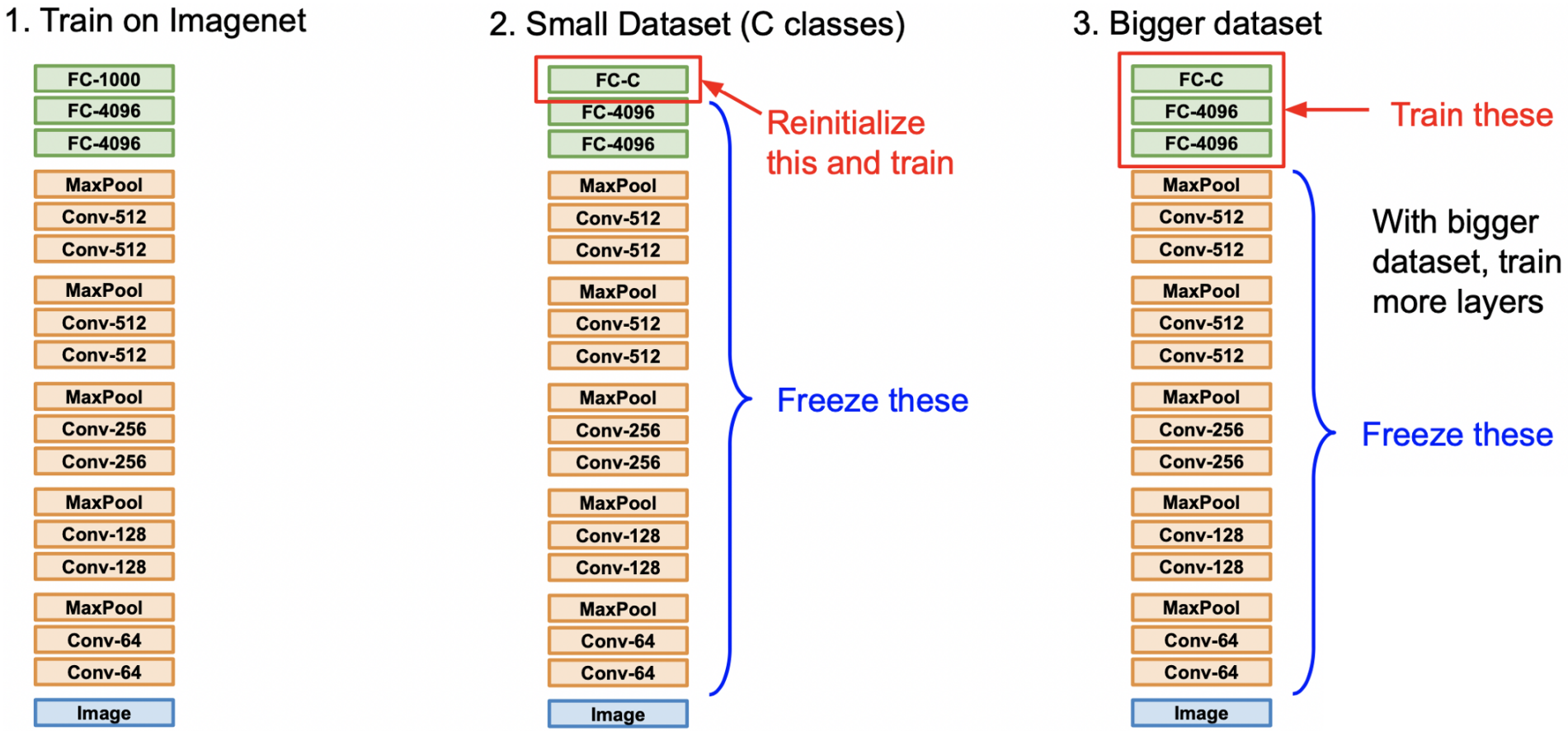
1. Train on Imagenet



2. Small Dataset (C classes)

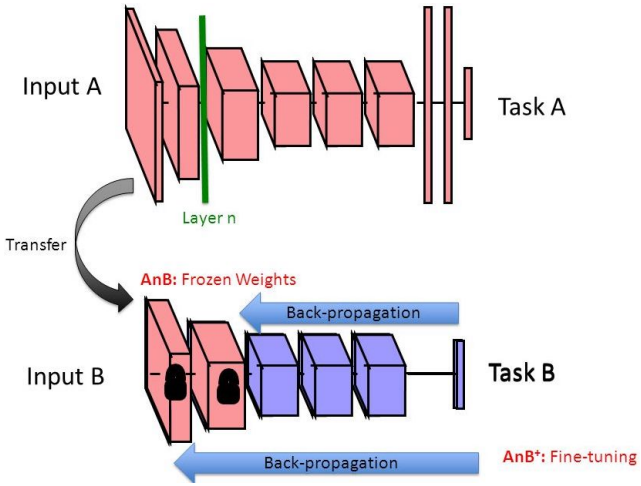


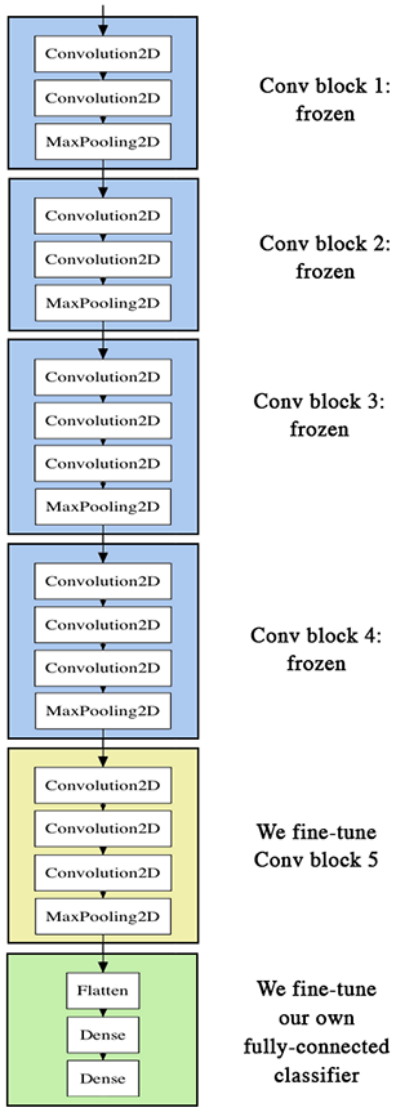
Transfer learning in CNNs



Transfer learning in CNNs

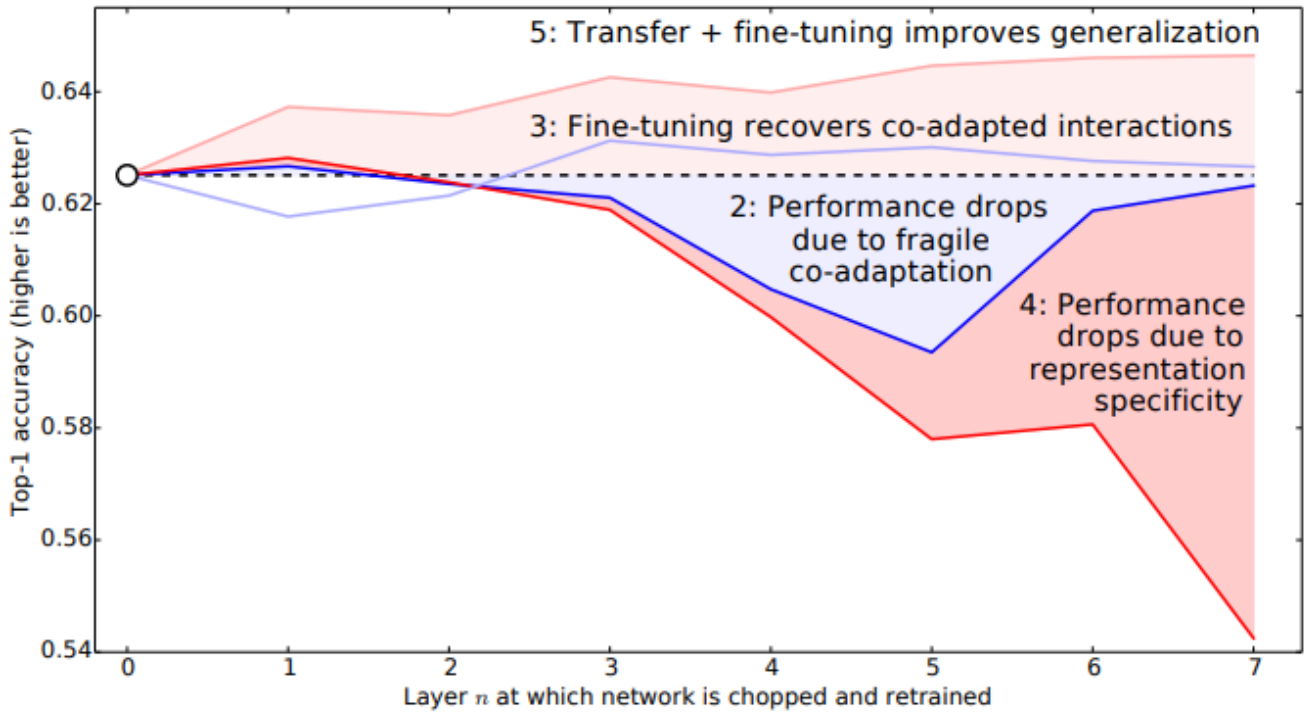
- Start with a pre-trained network (usually learned from a large dataset).
- Partition network into:
  - Featurizer: identify which layers to keep (frozen).
  - Classifier: identify which layers to update (learn).
- Re-train classifier layers with new data at hand.
- Unlock weights and fine-tune the whole network with a smaller learning rate.





Which layers to re-train?

- Really depends on the domain (or specific task at hand).
- Start by re-training the last layers (last FC and last Conv).
- Walk backwards if the performance is not satisfactory.



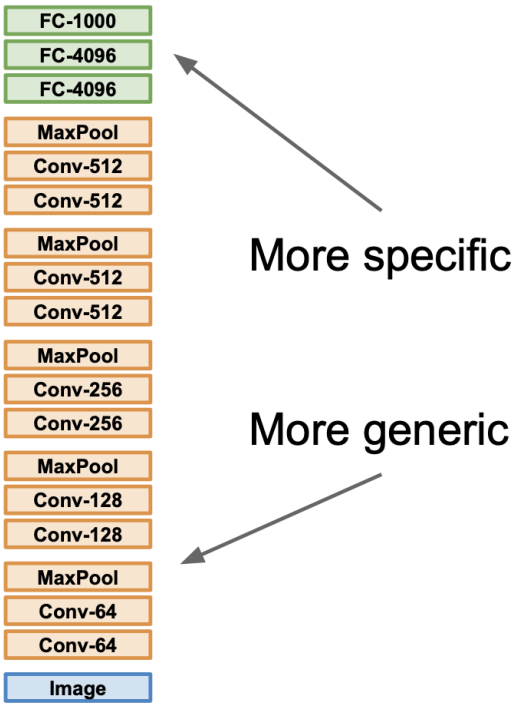
How transferable are features in deep neural networks? Jason Yosinski, Jeff Clune, Yoshua Bengio, Hod Lipson, in Advances in Neural Information Processing Systems (NeurIPS), 2014. http://arxiv.org/abs/1411.1792

When and how to fine-tune?

- Suppose we have model A, trained on dataset A.
- How do we apply transfer learning to dataset B to create model B?

When and how to fine-tune?

- Suppose we have model A, trained on dataset A.
- How do we apply transfer learning to dataset B to create model B?



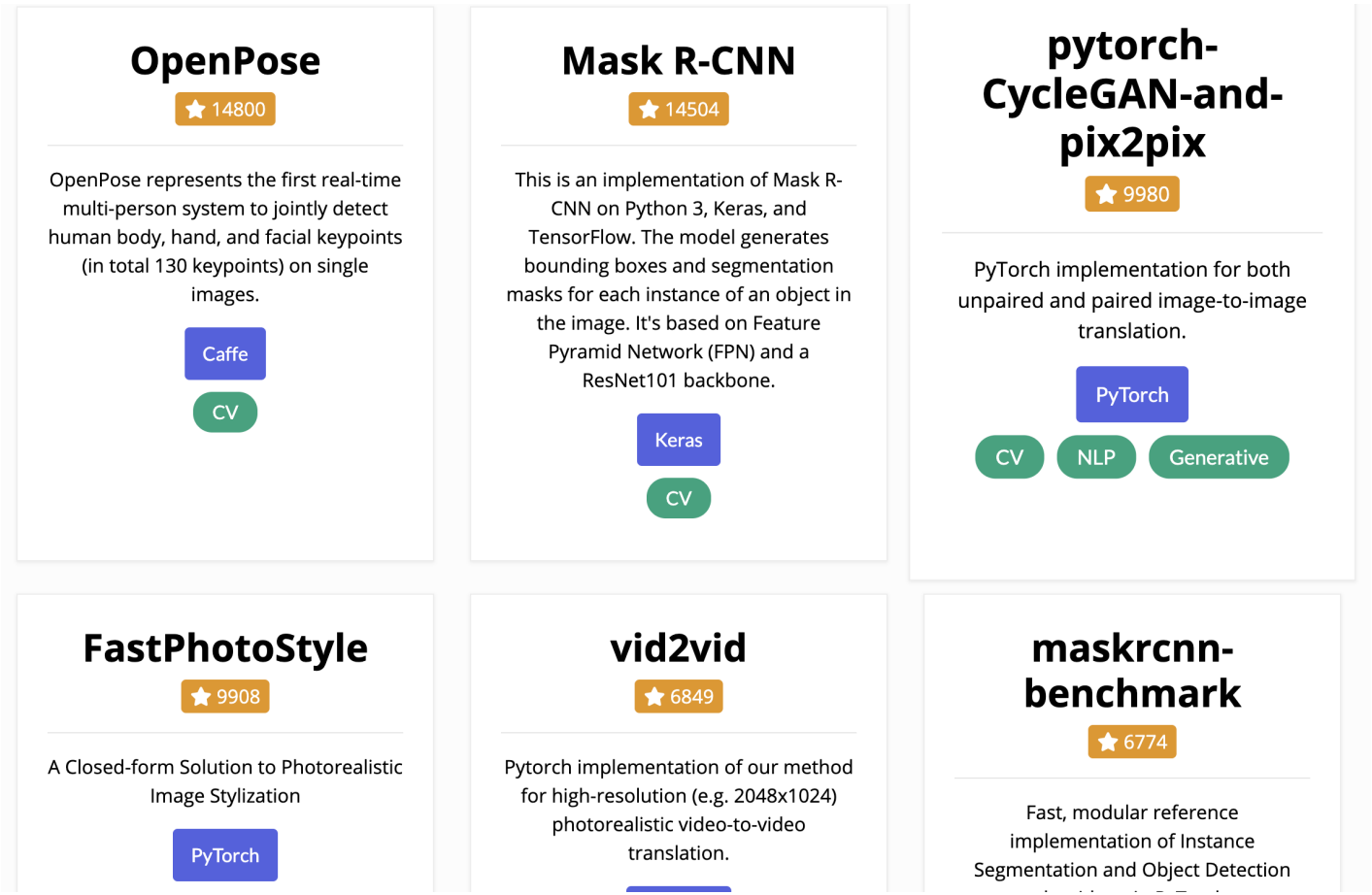
	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Fine-tuning learning rates

- Fine-tuning typically use smaller learning rates to avoid distorting the existing weights.
- Assuming pre-trained weights are close to "good".
- Typically, use $1/10$ of the original learning rate (i.e., for randomly initialized weights) is a reasonable starting point.

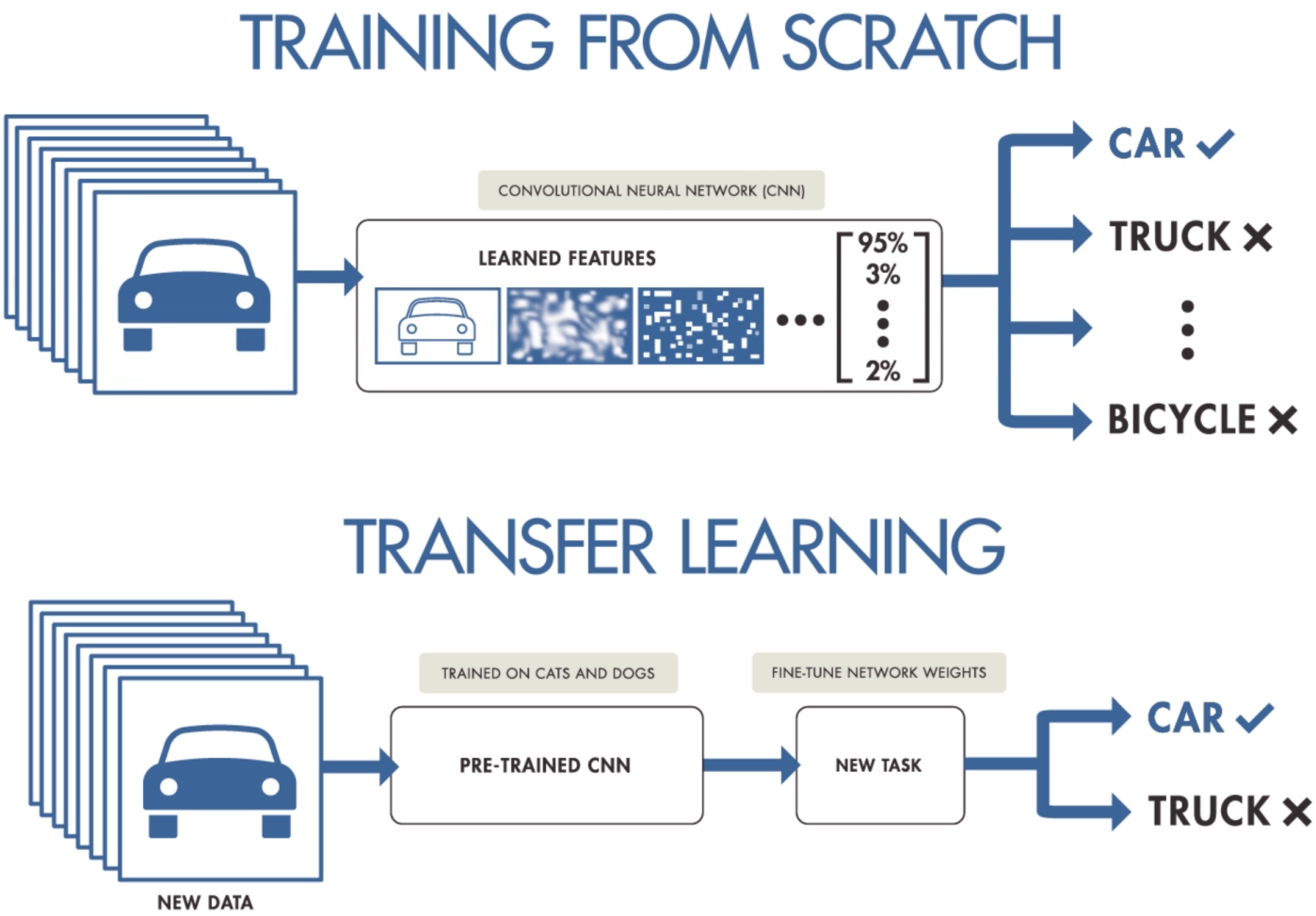
Model zoo

Publicly released and downloadable model zoo: https://modelzoo.co/



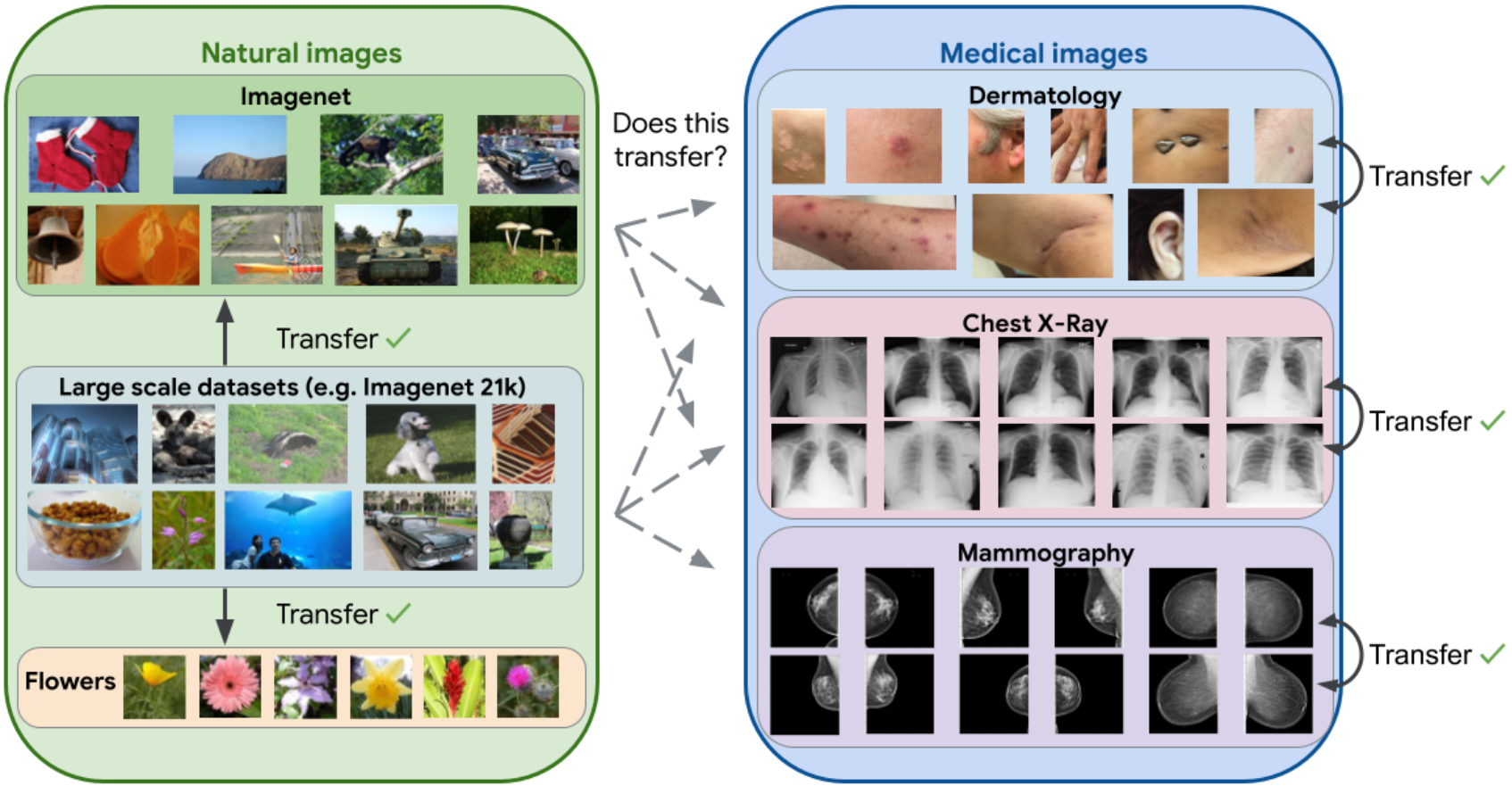
Applications

- Image classification (most common): learn new image classes



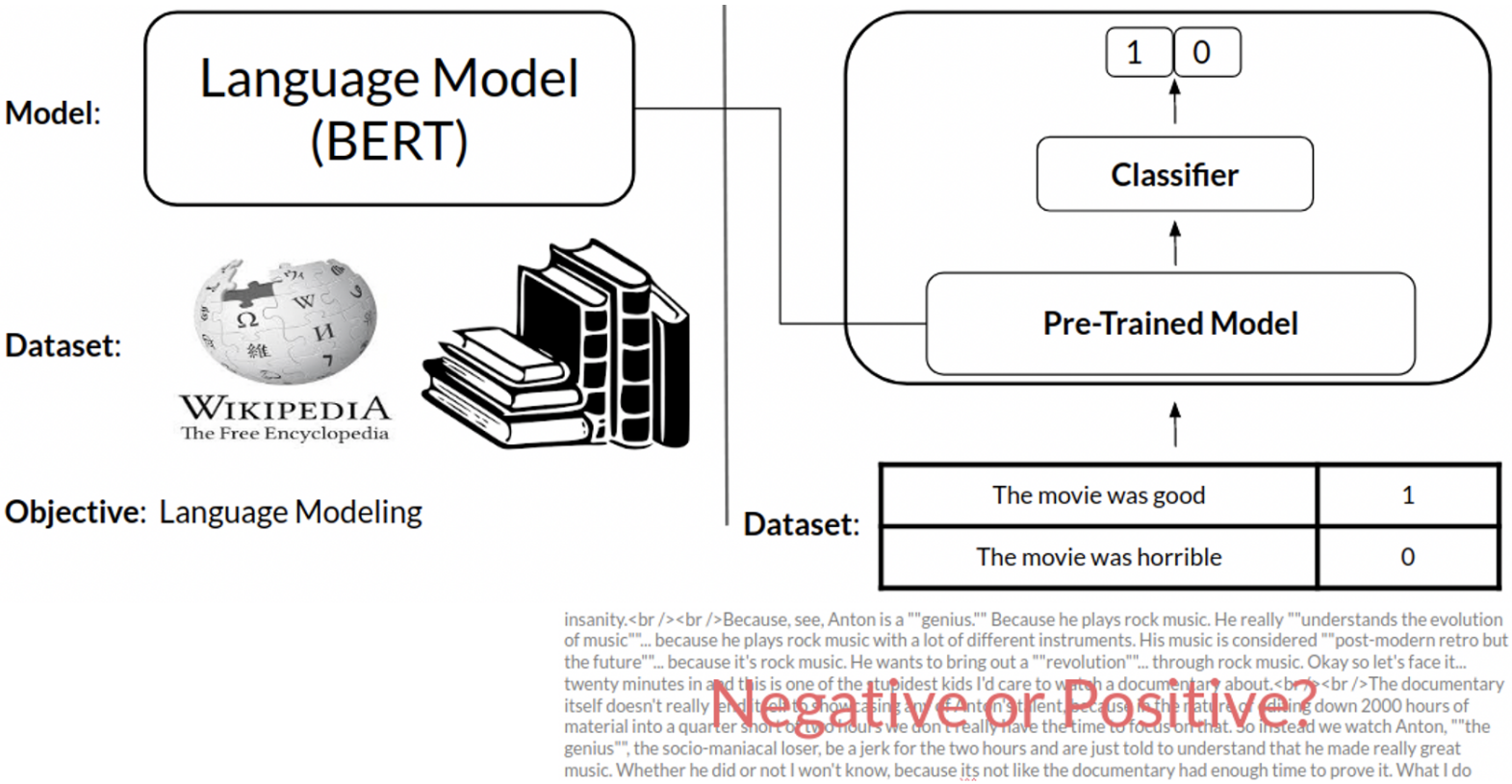
Applications

- Medical image diagnosis



Applications

- Text sentiment classification



Applications

- Speaker adaptation in speech recognition